Walrus-Cache-CDN

A Lifecycle-Aware Cache and CDN-like Edge Layer for Walrus Retrieval

Name: Raghav Awasthi[1]

BSc Computer Science

Supervisors: Philipp Jovanovic, Alberto Sonnino

Submission date: 22nd April 2026

[1]**Disclaimer:** This report is submitted as part requirement for the BSc Degree in Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

This dissertation presents **Walrus-Cache-CDN**, a systems prototype that introduces a cache-enabled retrieval layer in front of Walrus object retrieval endpoints. The aim is to reduce repeated origin fetches and improve latency while preserving correctness under object lifecycle changes such as certification updates, deletions, and epoch-based expiry.

The implemented system combines a Rust command-line interface, a RocksDB-backed persistent cache, an in-memory hot window, policy-aware admission including LRU-oriented and WTinyLFU-style behaviour, an indexer capable of both simulated and live Sui-backed event ingestion, and Prometheus-compatible metrics for evaluation and operational visibility. The system is evaluated using both end-to-end prototype demonstrations and a reproducible synthetic evaluation harness covering baseline-versus-cache comparisons, admission-policy comparisons, scan-resistance workloads, size-aware admission, cold-start warm-up, and concurrency scaling.

The results show that the cache/CDN-like layer materially improves hit-rate, lowers average latency, and increases throughput, while explicit invalidation and event-aware metadata updates preserve practical correctness. Overall, the project demonstrates that a lightweight lifecycle-aware edge layer can provide meaningful performance gains for decentralised object retrieval without requiring modifications to the underlying Walrus protocol.

Acknowledgements

I would like to sincerely thank Professor Philipp Jovanovic and Alberto Sonnino for all of their support throughout this project. From answering my many questions to guiding me through the specifics of Walrus and its surrounding technical context, their advice was consistently thoughtful, generous, and genuinely helpful throughout the entire process. I am especially grateful for how willing they were to make time to help. Whether I needed clarification on a detail, direction on a design decision, or simply a better understanding of how a particular part of Walrus worked, they were always supportive and encouraging. Their guidance made a substantial difference to both the direction and the quality of this work.

I am also immensely grateful to my family and friends, who continually supported me and created an environment in which I was comfortable and able to work to the best of my ability.

Contents

1 Introduction **1**

1.1 Motivation . . . 1

1.2 Problem Statement . . . 1

1.3 Project Objectives . . . 2

1.4 Main Contributions . . . 2

1.5 Scope and Assumptions . . . 2

1.6 Dissertation Structure . . . 3

2 Background, Context, and Related Work **4**

2.1 Introduction . . . 4

2.2 Blockchains, Decentralised Storage, and the Edge-Latency Problem . . . 4

2.2.1 Blockchain Overview . . . 4

2.2.2 Different Blockchain Models . . . 5

2.2.3 Decentralised Storage and the Need for an Edge Layer . . . 5

2.3 How Walrus Works . . . 6

2.3.1 Walrus as a Blob Storage System . . . 6

2.3.2 Walrus Client-Facing Components . . . 6

2.3.3 Walrus Read Path . . . 7

2.3.4 Why Walrus is a Good Fit for Caching . . . 7

2.4 Sui, Lifecycle State, and Correctness . . . 8

2.4.1 Sui Overview . . . 8

2.4.2 Walrus and Sui Together . . . 8

2.4.3 Object Identity and Lifecycle State . . . 9

2.5 Neighbouring Decentralised Storage Ecosystems . . . 10

2.5.1 IPFS . . . 10

2.5.2 Filecoin . . . 10

2.5.3 Arweave . . . 10

2.5.4 Why This Comparison Matters . . . 10

2.6 Caching, Admission, and Invalidation . . . 11

2.6.1 Why a Cache/CDN Layer is Non-trivial . . . 11

2.6.2 Caching Basics and Cache Operation Patterns . . . 11

2.6.3 Freshness and Invalidation Strategies . . . 12

2.6.4 Replacement Policies, Admission Control, and Size Awareness . . . 13

2.6.5 Persistent Local State and RocksDB . . . 14

2.6.6 Lifecycle, Events, and Invalidation . . . 15

2.6.7 Observability and Metrics . . . 16

2.7 Research Gap . . . 16

2.8 Chapter Summary . . . 17

3 Requirements and Analysis **18**
3.1 Introduction . . . 18
3.2 Refined Problem Statement . . . 18
3.3 Requirements Derivation . . . 19
3.4 System Scope and Boundary . . . 19
3.5 Functional Requirements . . . 19
3.6 Non-Functional Requirements . . . 21
3.7 Operational Scenarios . . . 21
3.8 Use-Case View . . . 22
3.9 Analysis of the Requirements . . . 22
3.10 Analysis Model of the System State . . . 23
3.11 Requirement Prioritisation . . . 24
3.12 Chapter Summary . . . 24

4 System Design and Architecture **25**
4.1 Introduction . . . 25
4.2 Design Goals . . . 25
4.3 Architectural Overview . . . 26
4.4 Data Plane and Control Plane . . . 27
4.5 Request Path Design . . . 28
4.6 Hybrid Cache Design . . . 29
4.7 Metadata Layout and State Representation . . . 29
4.8 Admission and Replacement Design . . . 30
4.9 Event Ingestion and Lifecycle Maintenance . . . 31
4.10 Invalidation Model . . . 32
4.11 Configuration and Operating Modes . . . 32
4.12 Extension for Multi-Node Placement . . . 32
4.13 Observability as a Design Concern . . . 33
4.14 Failure Modes and Architectural Limits . . . 33
4.15 Architecture Summary . . . 33

5 Implementation **34**
5.1 Implementation Overview . . . 34
5.2 CLI and Program Entry Point . . . 34
5.3 Configuration Handling . . . 35
5.4 Core Cache Data Structures . . . 35
5.5 RocksDB Layout and Opening Logic . . . 36
5.6 Read Path Implementation . . . 36
5.7 RAM Hits, Disk Hits, and Promotion Logic . . . 37
5.8 Admission Logic in Practice . . . 38

5.9 Frequency Sketch Implementation . . . 38
5.10 Write Path and Cache Population . . . 39
5.11 Metadata and Secondary Index Maintenance . . . 39
5.12 Invalidation Implementation . . . 39
5.13 Event Ingestion Implementation . . . 40
5.14 Cache Side-Effects from Events . . . 41
5.15 Metrics Implementation . . . 41
5.16 Demonstration Flow . . . 42
5.17 Sharding Helper . . . 42
5.18 Evaluation Harness Implementation . . . 42
5.19 Implementation Limitations . . . 43
5.20 Implementation Summary . . . 43

6 Evaluation **44**
6.1 Evaluation Goals . . . 44
6.2 Methodology . . . 44
6.3 Scenario Set . . . 45
6.4 How to Read Curated Labels . . . 45
6.5 Headline Results . . . 46
6.6 Overall Cache Effectiveness . . . 46
6.7 Baseline-Relative A/B Speedups . . . 47
6.8 Admission Policy Comparison . . . 49
6.9 Scan Resistance . . . 50
6.10 Size-Aware Admission Discussion . . . 50
6.11 Cold-Start Warm-up Behaviour . . . 51
6.12 Concurrency Scaling . . . 52
6.13 Latency Distribution . . . 53
6.14 Policy Trade-off Frontiers . . . 54
6.15 Correctness and Invalidation . . . 56
6.16 Why the Results Make Sense for This Project . . . 56
6.17 Threats to Validity . . . 57
6.18 Evaluation Summary . . . 57

7 Conclusions **58**
7.1 Achievements . . . 58
7.2 Evaluation of the Work . . . 58
7.3 Limitations . . . 59
7.4 Future Work . . . 59

A System Manual **63**
A.1 Purpose . . . 63
A.2 Repository Layout . . . 63

A.3 Build and Test Procedure . . . 64
A.4 Configuration and Environment . . . 64
A.5 Extending the System . . . 65

B User Manual **66**
B.1 Overview . . . 66
B.2 Metrics and Observability . . . 67
B.3 Demo Workflow . . . 67

C Project Plan and Interim Report **68**
C.1 Project Plan . . . 68
C.2 Interim Report . . . 71

D Selected Code Listing **74**
D.1 Selection Rationale . . . 74
D.2 CLI Surface and Runtime Wiring . . . 75
D.3 Walrus Retrieval and Upload Paths . . . 79
D.4 Cache Core, Metadata, and Invalidation . . . 82
D.5 Lifecycle Event Handling . . . 88
D.6 Selected Verification Code . . . 91

Chapter 1

Introduction

1.1 Motivation

Modern decentralised storage systems provide strong integrity and availability guarantees, but they can still exhibit variable retrieval latency at the application edge due to network distance, transient congestion, and repeated full-origin fetches. In systems where data objects are addressed by immutable identifiers and lifecycle metadata evolves over time, there is an opportunity to combine object-store semantics with CDN-style caching and event-driven invalidation.

This project presents **Walrus-Cache-CDN**, a systems prototype that sits in front of Walrus retrieval endpoints and introduces a cache layer with measurable hit/miss behaviour, explicit invalidation controls, and blockchain-event-aware metadata updates. Rather than proposing a new storage protocol, the work investigates how far a practical, modular edge layer can improve retrieval behaviour while remaining faithful to the underlying Walrus/Sui object lifecycle. In implementation terms, this means combining a Walrus client, a hybrid RAM and RocksDB cache, a metadata and tag-index layer, an event-driven indexer, as well as a metrics subsystem, all into one coherent prototype.

1.2 Problem Statement

The core problem addressed by this project is as follows:

> How can a cache-enabled retrieval layer for Walrus blob objects reduce repeated origin fetches while preserving correctness under object lifecycle changes (e.g., certification updates, deletion, and epoch-driven expiry)?

There are three main trade-offs that must be considered:

1. **Performance vs correctness**: having aggressive caching would massively improve/reduce latency, but would simultaneously run the risk of stale objects being fetched.

2. **Simplicity vs adaptivity**: a simple replacement/admission policy would be robust, but workload-aware admission is often necessary for skewed access patterns.

3. **Prototype realism vs scope**: integrating live on-chain signals would increase realism but would also introduce operational complexity, such as requiring tests to be repeated over long amounts of time in order to gain an accurate picture of the system.

1.3 Project Objectives

The main aim is to design and implement a command-line Walrus retrieval client with an integrated cache (which can optionally be disabled). This must support both a simulated indexer mode and a live Sui-backed mode for event ingestion which actually retrieves up-to-date data. In regards to the cache, multi-policy cache admission (None, LRU-style, WTinyLFU-style) should be implemented, alongside metadata-aware invalidation. The system should also expose observability metrics such that it is easy to demonstrate hit/miss transitions and latency distribution. Finally, the system should provide deterministic demonstrations of cache miss → hit → invalidation flows.

1.4 Main Contributions

The main concrete contributions of this dissertation lie in the following features:

1. **A modular Walrus edge prototype**: a Rust implementation of the system that combines retrieval, upload, indexing, invalidation, metrics, demo orchestration, and synthetic evaluation within a single executable.

2. **A hybrid cache architecture**: a two-tier design that combines a bounded in-memory hot window with a RocksDB-backed persistent cache that is organised into separate payload, metadata, and tag-index stores.

3. **Policy-aware cache admission**: the support for multiple admission modes, including unconditional admission, LRU-oriented behaviour, and a lightweight WTinyLFU-style mode backed by a Count-Min sketch and optional size-aware gating.

4. **Lifecycle-aware cache maintenance**: the integration of simulated and live Sui-backed event ingestion, including derived lifecycle signals such as *Extended* and *Expired*, which can update metadata and trigger invalidation as needed.

5. **Observability and reproducibility**: Prometheus-compatible metrics, deterministic demo flows, and a reproducible synthetic evaluation harness that, combined, support defensible performance, as well as correctness analysis.

1.5 Scope and Assumptions

This work is a systems prototype rather than a full production CDN deployment, due to time constraints, as well as limited access to various resources. The project assumes the following: the Walrus aggregator and publisher APIs are available and reachable; blob identifiers are immutable and suitable as cache keys for payload reuse; and lifecycle correctness can be approximated operationally using observed Walrus/Sui events together with local metadata.

Additionally, it is important to note that global POP placement, multi-region coherence, adversarial threat modelling, and fully decentralised replication protocols are taken to be out of scope, since this is merely a prototype.

1.6 Dissertation Structure

The remainder of this dissertation is organised as follows: Chapter 2 introduces the technical background, context, and related work. Chapter 3 defines the project requirements and analysis. Chapter 4 presents the system design and architecture used for the prototype. Chapter 5 explains the implementation in relation to the actual codebase. Chapter 6 evaluates the prototype and interprets the graphs and metrics. Finally, Chapter 7 concludes the dissertation and outlines future work.

Chapter 2

Background, Context, and Related Work

2.1 Introduction

This project sits at the intersection of decentralised storage, blockchain state, cache design, event-driven systems, persistent embedded storage, and systems performance engineering. For that reason, this chapter does more than simply define a few terms. It explains the storage model that Walrus builds on, how Sui represents and evolves state, why caching is useful but non-trivial in this setting, and how the most relevant prior work in caching, admission control, persistent state, and event-driven maintenance relates directly to the design of Walrus-Cache-CDN.

A key idea that runs through the dissertation is that Walrus-Cache-CDN should not be understood as a simple wrapper that stores recently seen bytes in a local map. Rather, it is more accurately understood as a small edge system that combines a retrieval **data plane** with a metadata and invalidation **control plane**. The data plane serves object bytes quickly, while the control plane decides whether those bytes are still valid to serve. Understanding that distinction is essential for understanding both the system design and the evaluation later in the dissertation.

The chapter is organised as follows. It first introduces the blockchain and decentralised-storage context relevant to the project, then explains Walrus and its relationship to Sui. It next develops the main caching, lifecycle, invalidation, and observability concepts that motivate the system design, while incorporating the most relevant related work as each concept is introduced. The chapter ends by identifying the specific gap that this dissertation addresses.

2.2 Blockchains, Decentralised Storage, and the Edge-Latency Problem

Before discussing Walrus specifically, it is important to summarise the basic concepts of blockchains and decentralised systems that this project depends on.

2.2.1 Blockchain Overview

At a high level, a blockchain is a replicated state machine maintained by a distributed set of participants that agree on the order and effect of transactions. Different systems implement this in different ways, but the common theme is that multiple nodes validate or execute transactions and agree on a shared state transition history. In simpler terms, a blockchain is a shared ledger that records transactions across many machines in a way that resists retroactive modification.

Bitcoin introduced this idea in the context of peer-to-peer electronic cash, focusing on preventing double-spending without a central intermediary [1]. Later blockchain systems generalised the idea from payments to arbitrary programmable state. For the purposes of this dissertation, the most important point is that a blockchain is not merely a storage layer. It is a state system with consensus, transaction ordering, and a specific data model. That data model affects what information is visible to external systems and how quickly lifecycle changes can be detected.

2.2.2 Different Blockchain Models

Different blockchain systems make different design choices depending on their goals.

Bitcoin is primarily a payment system and uses an unspent transaction output style model. It is not designed around general application objects in the same sense as modern smart-contract platforms [1].

Ethereum generalised the model by introducing programmable smart contracts and an account-based state model. In this model, accounts hold balances and can trigger execution, while smart contracts are programs stored and executed in shared state [2, 3]. This made decentralised applications practical, but it still organises state around accounts and contract storage rather than first-class objects.

Sui differs in a way that is especially important for this project. Its storage model is object-centric, meaning that objects are the fundamental units of on-chain state, each with a unique ID, versioned state, and ownership semantics [4]. Transactions operate over these objects, and applications can emit events that external systems later query and process [5]. Sui also operates in epochs, meaning network activity is partitioned into periods during which validator membership remains fixed [6].

This object-centric model is particularly important when compared with the account-based traditions of systems such as Ethereum. In an account-based setting, lifecycle-relevant state is often spread across contract storage and account interactions. In Sui, by contrast, it is much more natural to reason about state transitions at the level of identifiable objects, and this matters because it means that Walrus lifecycle and metadata logic can naturally be associated with objects and object-related events rather than only with accounts or contract storage slots. In addition, the fact that Sui operates in epochs means that this project can use epoch-related metadata as part of its local cache lifecycle model, specifically when worrying about expiring or invalidating certain blobs. This makes Sui an especially suitable control-plane source for an external cache that must react to object lifecycle changes.

2.2.3 Decentralised Storage and the Need for an Edge Layer

Decentralised storage systems improve durability and availability by distributing storage across multiple participants, but they do not automatically provide low-latency reads at the application edge. In practice, the application still faces network delays, repeated retrieval costs, and

variability due to load or path distance. This is where caching becomes useful.

If immutable or mostly stable objects are repeatedly requested, an edge cache can avoid repeated origin retrievals and reduce latency. However, because the storage system is decentralised and lifecycle state can evolve through chain-visible events, correctness cannot be reduced to a simple fixed expiry timer.

This practical motivation is closely aligned with the long-established literature on web caching and content distribution networks. Surveys such as Wang's review of web caching showed that locality, replication, and proximity to users are central reasons why caching improves performance [7]. The same first-order logic applies here: expensive or repeated remote retrievals should be moved closer to the request path where possible. The difference is that Walrus-Cache-CDN does not operate in an ordinary HTTP freshness model, so reuse must be tied to lifecycle-aware metadata rather than only standard cache headers.

2.3 How Walrus Works

Walrus is the decentralised storage network that this project targets. Understanding its basic data model, its client-facing components, and its relationship to Sui is necessary to understand both why a Walrus-facing cache is possible and why it is non-trivial.

2.3.1 Walrus as a Blob Storage System

Walrus is designed for storing and retrieving blobs rather than for running general smart-contract logic directly in storage nodes. The key abstraction for this project is the **blob**: a content object that can be written, certified, and read. The Walrus paper describes a write path in which stored sliver data and acknowledgements are combined into a write certificate, which is then published on-chain as a Point of Availability (PoA) [8]. The PoA signals that storage nodes are obliged to maintain the blob for reads over its retention period.

This matters for caching because it means there is a meaningful distinction between the blob bytes themselves, the fact that the blob has been made available and certified, and the lifecycle metadata associated with how long the blob should remain available.

2.3.2 Walrus Client-Facing Components

For an application developer, Walrus is typically not experienced as a set of storage nodes directly, but rather through higher-level services that expose upload and retrieval functionality. In the prototype implemented in this dissertation, two roles are especially important: the **publisher**, which is used to upload new blobs and obtain identifiers or related metadata, and the **aggregator**, which is used on the read path to retrieve blob contents by blob ID or related object identity.

This distinction matters architecturally because Walrus-Cache-CDN is placed in front of the retrieval side of the system and does not change how Walrus itself operates internally. In

other words, the prototype treats Walrus retrieval endpoints as the authoritative origin and optimises repeated reads at the client/edge side, while still allowing upload and lifecycle activity to happen through the normal Walrus/Sui mechanisms.

2.3.3 Walrus Read Path

Walrus reads are not identical to reading from a single central object store. The Walrus paper describes a read path in which the reader can ask storage nodes for the commitments and secondary slivers associated with a blob ID, collect enough valid replies, reconstruct the blob, and recompute the blob ID to verify correctness [8]. Repeated reconstruction or repeated origin-side retrieval work is therefore more expensive than serving an already validated object from a trusted local cache layer.

Current Walrus documentation reinforces this point by describing the read path as a sequence involving authenticated metadata retrieval, sliver collection, blob reconstruction, and result verification [8, 9, 10]. This directly motivates a cache layer: once a blob has been retrieved and validated, repeated local serving can avoid repeated origin-side retrieval and reconstruction costs.

As Walrus-Cache-CDN sits in front of retrieval endpoints and does not modify the Walrus protocol directly, it exploits the fact that once blob bytes are retrieved and identified by blob ID, repeated local serving can be much cheaper than repeated origin fetches. This is why the present dissertation focuses on a retrieval-side edge layer rather than on redesigning the underlying storage protocol itself.

2.3.4 Why Walrus is a Good Fit for Caching

Walrus is particularly well suited to a cache-enabled edge layer for two main reasons.

First, blob identity is content-oriented and stable enough to make a natural cache key. Once a blob has been retrieved and validated, subsequent requests for the same blob ID can often reuse the same bytes locally.

Second, the expensive part of retrieval is not merely the act of reading from somewhere else. It can involve remote coordination, sliver collection, and reconstruction work [8, 10]. That means the performance gap between a local cache hit and an origin retrieval can be substantial enough to matter in practice.

However, this does not mean Walrus can be treated as a static immutable archive from the cache's point of view. The payload may be immutable under a blob ID, but the surrounding lifecycle state still matters. This is where Sui becomes important.

2.4 Sui, Lifecycle State, and Correctness

2.4.1 Sui Overview

Sui is the blockchain environment that provides the object-centric state model and event mechanisms that this project relies on. Unlike account-based systems that organise state primarily around accounts and contract storage, Sui represents on-chain state as objects, each with a unique identifier, versioned state, and ownership semantics [4]. Transactions operate by creating, reading, transferring, or mutating objects, and the resulting state transitions are recorded by the network. For the purposes of this dissertation, three Sui features matter most.

Object-centric state. Objects are first-class entities in Sui. This fits well with Walrus because blob-related lifecycle information can be tied to specific objects and object IDs, making it easier for an external cache/control layer to reason about what changed and which cached state may need to be updated.

Events. Sui allows applications to emit events as part of transaction execution, and these events can then be queried through interfaces such as RPC and GraphQL [5]. Events are important because they provide an external, machine-readable signal that something relevant has happened. In this project, they form the basis of the cache control plane: rather than guessing only from elapsed time, the system can react to observed state transitions.

Epochs and reconfiguration. Sui operates in epochs, which are bounded periods during which the validator set remains stable before reconfiguration [6]. Epochs matter here because lifecycle metadata may include an `end_epoch` or related horizon after which a blob should be treated as expired or invalid to serve from cache. Even though the cache is not itself part of consensus, epoch-related information gives it a structured notion of lifecycle progression.

Together, these features make Sui useful not as a storage replacement for Walrus blob bytes, but as a **control-plane source of lifecycle truth** for an external cache.

2.4.2 Walrus and Sui Together

Walrus and Sui play different but complementary roles. Walrus is responsible for decentralised blob storage and retrieval, while Sui provides the surrounding object model, transaction context, and event-visible state transitions that allow external systems to observe lifecycle changes [8, 4, 5].

This distinction is crucial for the dissertation. The prototype does not assume that all correctness questions can be answered from the cached bytes themselves. Instead, it treats the payload bytes and the lifecycle state as related but distinct concerns: Walrus provides the blob data and the retrieval path, Sui provides observable state transitions and lifecycle signals,

and Walrus-Cache-CDN combines the two by caching payload bytes while separately tracking metadata and invalidation-relevant state.

In practical terms, this means that the cache can store and serve blob bytes by blob ID, but it should also react when Sui-visible information suggests that local metadata must change or that a cached entry should no longer be served. This is why the project includes a Sui-backed indexer mode and why lifecycle-aware invalidation is a central design feature rather than an optional add-on.

It is therefore most accurate to think of Walrus-Cache-CDN as a system with a **data plane**, which accelerates repeated blob retrievals from Walrus, and a **control plane**, which uses Sui/Walrus-visible lifecycle information to decide whether cached state remains safe and appropriate to serve.

2.4.3 Object Identity and Lifecycle State

One of the most important background ideas for this dissertation is the distinction between **object identity** and **object lifecycle state**. This distinction explains both why caching is feasible and why correctness cannot be reduced to simply storing bytes locally.

Immutable identity. The project uses the blob ID as the main cache key. This is appropriate because the payload bytes associated with a given blob ID are treated as immutable once identified and retrieved. In practical terms, if the same blob ID is requested twice, then, unless some lifecycle rule prevents reuse, the same payload bytes should be returned. This property is what makes a Walrus-facing cache technically plausible. Without a stable object identity, repeated requests could not safely reuse a previously stored payload.

Mutable lifecycle state. Although the payload bytes are treated as immutable, the metadata associated with the object is not necessarily fixed. In this project, lifecycle-related metadata may include object size, timestamps, certification or end-epoch information, deletability or invalidation-related flags, and semantic tags used for purge operations. This means that the bytes for a blob ID are stable enough to cache, while the surrounding state that determines whether the cached bytes are still appropriate to serve may change over time. That is exactly why Walrus-Cache-CDN separates payload storage from metadata storage.

This distinction appears directly in the codebase in the separate handling of blob bytes and the `BlobMeta` structure, which stores fields such as `size`, `end_epoch`, `ts_ms`, `deletable`, and `tags`. This separation is therefore not merely an implementation convenience; it is a direct consequence of the Walrus/Sui lifecycle model described here.

This separation between stable identity and changing lifecycle state also helps explain why ordinary cache-freshness ideas are only partially applicable. In conventional HTTP caches, correctness is often determined through timers and validators. In the Walrus/Sui setting,

correctness depends more heavily on externally visible lifecycle change, which makes event-informed metadata maintenance especially important.

2.5 Neighbouring Decentralised Storage Ecosystems

It is also useful to position Walrus relative to neighbouring decentralised storage ecosystems, because this helps clarify what this dissertation is and is not claiming.

2.5.1 IPFS

IPFS is built around content-addressed data and retrieval by Content Identifier (CID). Its documentation emphasises content addressing, peer-to-peer transfer, and the lifecycle of data from local content-addressing through distribution and retrieval [11, 12, 13]. This is relevant because it shows a closely related design tradition in which stable content identity makes reuse and verification natural. However, IPFS by itself does not provide the same Walrus/Sui-style lifecycle and event context that this project uses for cache invalidation and metadata maintenance.

2.5.2 Filecoin

Filecoin extends decentralised storage with providers, retrieval, economic incentives, and cryptographic proofs. Current documentation presents Filecoin as a peer-to-peer storage and retrieval network and describes a storage model built around providers, deals, and sectors, alongside explicit retrieval flows [14, 15, 16, 17]. This is relevant because it highlights a different but related systems model in which durable storage and retrieval are tightly connected to incentives and proof-based assurance. Compared with Filecoin, the contribution of this dissertation is narrower: it does not propose a new storage market or proof mechanism, but rather studies how a lifecycle-aware edge cache can sit in front of an existing decentralised retrieval layer.

2.5.3 Arweave

Arweave is commonly positioned around permanent data storage and the permaweb model [18, 19]. This makes it a useful contrast case. Whereas Arweave is typically discussed in terms of permanence, Walrus-Cache-CDN is concerned with improving repeated retrieval performance while remaining responsive to lifecycle-relevant state changes in a Walrus/Sui environment.

2.5.4 Why This Comparison Matters

These systems are relevant not because Walrus-Cache-CDN tries to unify them, but because they illustrate a common systems reality: decentralised storage networks optimise for durability, verifiability, and distribution, while applications still benefit from fast, local, edge-oriented

retrieval layers. The dissertation therefore addresses a practical systems gap at the interface between decentralised storage guarantees and application-facing performance.

2.6 Caching, Admission, and Invalidation

2.6.1 Why a Cache/CDN Layer is Non-trivial

The central challenge is not just to store hot objects locally. It is to design a cache layer that is fast, safe, and operationally manageable at the same time.

Performance goal. A cache should reduce repeated origin fetches, reduce average and tail latency, and improve throughput for workloads with temporal locality. This is the familiar part of the problem.

Correctness goal. A cache must not continue serving entries whose lifecycle state has changed in a way that makes them inappropriate to serve. In ordinary HTTP systems, this problem is often handled with freshness metadata, validators, and revalidation rules [20]. In this project, correctness depends more heavily on event-derived lifecycle information such as deletion-like transitions, certification changes, and epoch-related expiry logic.

Operational goal. A useful prototype must also be observable and debuggable. It is not enough for the system to work internally. The project must be able to show, with evidence, when a request was a miss, when it became a hit, and when an invalidation caused the next request to fall back to origin.

These three concerns together explain why the system has to be treated as an edge **system**, not merely as a local optimisation.

2.6.2 Caching Basics and Cache Operation Patterns

A cache is a smaller, faster storage layer that keeps copies of data from a slower or more expensive source. When a request can be answered from the cache, it is called a **cache hit**. When the cache does not contain the requested item and the system must fetch it from the original source, it is called a **cache miss**. A cache is beneficial when the same objects are requested repeatedly or when recently or frequently used objects are likely to be reused.

Caches help because memory and local disk are usually faster than network retrieval from remote or distributed origins. They also reduce load on the origin system by avoiding duplicate work. In a decentralised storage setting, they are especially useful when origin reads involve network hops, reconstruction, or cryptographic verification steps that need not be repeated for every request.

There are several broad cache categories, including client-side caches, proxy or gateway caches, edge/CDN caches, and storage-engine caches. Walrus-Cache-CDN is best understood as a proxy or edge-style cache, since it sits in front of the retrieval origin and serves repeated requests more efficiently.

The standard cache operation patterns are also useful background. Cache-aside is where the application checks cache first, then fetches from origin on miss, and then inserts into cache. Read-through is where the cache layer itself fetches from origin on miss and returns the result. Write-through is where writes are propagated through the cache to backing storage immediately. Write-back is where writes are absorbed by cache and flushed later. Write-around is where writes bypass cache to avoid polluting it with data that may never be read. This project is closest to a cache-aside/read-through hybrid on the read path: the edge layer checks local state, falls back to origin on miss, persists the fetched object into local state, and then applies policy logic to determine whether it should also be promoted into the in-memory hot tier.

2.6.3 Freshness and Invalidation Strategies

Not all caches use the same strategy for freshness, insertion, or eviction. These choices matter because they affect both performance and correctness.

TTL-Driven Caching. Time-to-Live (TTL) driven caching is a relatively simple strategy where, after a TTL expires, the object is treated as stale and must be refreshed or removed. TTL-based caching is simple and widely used, but it assumes that freshness can be approximated by elapsed time. In Walrus-Cache-CDN that is not sufficient by itself, since lifecycle changes are often event-driven rather than strictly time-driven.

Validation-Based Caching. Validation-based caching is common in HTTP systems. Validators such as `ETag` or conditional requests allow a cache to confirm whether a cached object is still fresh [20]. This reduces staleness risk but requires cooperation from the origin protocol. Walrus retrieval does not naturally expose identical HTTP-style validation semantics, so this project relies more on lifecycle metadata and invalidation events.

Event-Driven Caching. An event-driven cache updates or invalidates local state when it sees relevant external signals. This is the strategy with the closest matching use-case to this project. Instead of assuming objects expire only because a timer elapsed, the system reacts to deletion, extension, or epoch-related signals when they are observed.

This part of the design is strongly related to event-driven systems work more broadly. In distributed systems, it is common to consume events from an external source of truth and maintain a derived local view that makes the read path cheaper or simpler. Kleppmann's treatment of logs, derived state, and materialised views is highly relevant here [21]. Walrus-Cache-CDN follows exactly this pattern: origin retrieval remains authoritative, while local

cache state is a materialised and maintainable view whose correctness depends on externally observed lifecycle change.

2.6.4 Replacement Policies, Admission Control, and Size Awareness

Caching performance does not depend only on whether a system stores objects locally. It also depends on which objects are allowed into the cache, which ones are retained when space is limited, and how size and workload structure influence those decisions. For this reason, Walrus-Cache-CDN does not treat the cache as a passive byte store; it treats it as a policy-driven system.

When a cache is full, it needs a rule for deciding which entries to evict. That rule is called an **eviction** or **replacement** policy, and there exist many kinds of eviction policies.

FIFO. First-In, First-Out (FIFO) evicts the oldest inserted entry first. It is extremely simple, but it ignores both recency and frequency, making it a weak choice for performance-sensitive workloads.

LRU. Least Recently Used (LRU) evicts the item that has not been accessed for the longest time. The intuition is that recently used items are more likely to be used again soon. LRU is popular because it is conceptually simple and works well when workloads have strong temporal locality. Redis documentation still presents LRU as one of the standard eviction options in practical cache systems [22]. The main advantage of LRU is that it adapts quickly to bursts of recent interest. Its main weakness, however, is scan sensitivity: if a workload performs a long one-time scan over many distinct objects, LRU may evict genuinely useful hot objects in favour of items that will never be reused.

LFU. Least Frequently Used (LFU) evicts the item with the lowest access frequency. The idea is that frequently accessed items are more valuable than infrequently accessed ones. LFU is strong on stable popularity distributions, and Redis also supports LFU-style eviction in production systems [22]. The main weakness of pure LFU is that it can be too sticky unless some kind of ageing or decay mechanism is introduced.

ARC. Adaptive Replacement Cache (ARC) is historically important because it demonstrated that dynamically balancing recency and frequency can outperform simple one-signal policies under changing workloads [23]. Low Inter-reference Recency Set (LIRS) is also important because it highlighted the weaknesses of naive LRU and pushed the field toward more workload-sensitive management [24]. These systems are useful background, but they are more complex than is necessary for the current prototype.

A more directly relevant development is the move from pure eviction design toward **admission control**. Eviction decides what leaves the cache, whereas admission control decides what

gets into the cache in the first place. Without admission control, every miss becomes a new insertion. That is simple, but it allows one-time or scan-heavy accesses to consume cache space and displace genuinely valuable objects.

TinyLFU is especially important here because it reframed the problem in exactly these terms [25]. Instead of assuming every miss deserves a place in the cache, TinyLFU uses a compact approximate frequency sketch to estimate whether a newly accessed candidate is likely to be more valuable than the item it might displace. Window TinyLFU, or W-TinyLFU, combines a small recency-focused window with a larger main space protected by TinyLFU-style admission. Caffeine's design notes explain this clearly, and modern libraries such as Moka in Rust and Ristretto in Go show that TinyLFU-derived approaches are now part of practical systems design rather than only academic proposals [26, 27, 28, 29].

This is directly relevant to Walrus-Cache-CDN because the codebase includes an `AdmissionMode` with `WTinyLFU`, `Lru`, and `None`, and the evaluation compares these directly. The use of W-TinyLFU is therefore not a theoretical aside; it is one of the practical design choices implemented and measured in the system.

A further refinement is **size-aware admission**. Objects may have very different sizes. A cache that stores one very large object may lose the opportunity to store many smaller objects that collectively produce more hits. AdaptSize is one influential example, designed for CDN memory caches with variable object sizes [30]. The core principle is that cache space should be understood in terms of bytes, not only object count. This idea is highly relevant to Walrus-Cache-CDN because Walrus blobs can vary significantly in size. A large blob admitted into persistent cache consumes disk capacity, affects local block-cache behaviour, and may crowd out many smaller objects that would together generate greater reuse. The current prototype therefore includes a simple probabilistic size-aware gate that biases admission against oversized objects when capacity is constrained.

2.6.5 Persistent Local State and RocksDB

A pure in-memory cache is very fast but loses state on restart. A persistent cache survives restart and can store more metadata, but it is typically slower than RAM. This project intentionally combines both: a hot in-memory structure for speed and a RocksDB-backed persistent store for durability and structured metadata.

RocksDB is an embeddable persistent key-value store that is widely used in systems that need high-performance local storage with durability. Its support for **column families** is especially relevant here: RocksDB documentation explains that each key-value pair belongs to exactly one column family and that column families provide a logical partitioning of the database [31, 32]. Official RocksDB material also emphasises practical capabilities such as write-ahead logging, logical partitioning, and efficient point lookups, which make it a strong fit for lookup-heavy systems with structured local state.

Walrus-Cache-CDN needs more than a single dictionary of key-to-bytes. It needs payload

storage, metadata storage, secondary records for tags or invalidation support, and persistence across process restarts. RocksDB is therefore a good fit because it supports structured local state without requiring a separate external database service. The project's use of distinct stores for blobs, metadata, and tag-related records maps naturally onto the column-family concept.

This matters not only as background, but also in relation to prior work. Much cache-policy literature focuses primarily on admission and eviction behaviour over abstract cached objects, which is appropriate for analysing replacement quality and hit-rate trade-offs [25, 23, 24]. However, a practical edge system must also manage several kinds of local state simultaneously, such as payload bytes, lifecycle metadata, auxiliary indexes, invalidation or tag mappings, and restart-persistent policy state. RocksDB is well suited to this kind of design because it is an embeddable persistent key-value store and supports logical partitioning through column families, which are widely used to separate different classes of data within one database [32, 31, 33].

Persistent local state is therefore part of the contribution of this project, since it allows the system to preserve cache contents across restart and maintain lifecycle-aware metadata independently from payload bytes. This also matches the distributed-systems view of derived state, where a local representation is maintained to make future reads cheaper while remaining tied to an external source of truth [21].

2.6.6 Lifecycle, Events, and Invalidation

A cache is only correct if it knows when to stop serving an entry – this is the invalidation problem.

A cache improves performance by keeping old copies of data, but that same behaviour creates risk if the old copy is no longer appropriate to serve. In a decentralised storage setting, invalidation is difficult because the cache is not the source of truth; it must infer validity from external state or explicit rules.

Sui events provide a practical mechanism for observing object-related activity and triggering local workflows [5]. In this project, the indexer consumes either simulated or live events and maps them to local cache actions such as metadata updates or purges. This turns chain-visible activity into cache-maintenance logic.

The prototype supports three main invalidation modes: purge by blob ID, purge by tag, and purge by epoch threshold. These operations map directly to the kinds of correctness risks the cache faces. A direct object invalidation can be handled by blob ID. Grouped or semantic objects can be handled by tag. Time- or certification-horizon-style invalidation can be handled by epoch thresholds.

This design also aligns with the broader event-driven systems literature. Traditional web caches frequently rely on TTLs, freshness lifetimes, or validation mechanisms such as conditional requests and validators [20]. By contrast, event-driven systems often maintain derived or materialised local state by reacting to externally observed state changes rather than assuming that freshness degrades only as a function of elapsed time [21]. This perspective is

especially appropriate for Walrus because lifecycle transitions such as certification, extension, deletion, and expiry are visible through Walrus/Sui mechanisms, including object-related events and epoch progression [5, 6, 10]. In that sense, Walrus-Cache-CDN can be understood as an application-specific instance of **event-informed cache maintenance for a decentralised storage read path**, where cached payload reuse is governed not only by request patterns but also by externally visible lifecycle signals.

2.6.7 Observability and Metrics

Systems work must be measured, not only described. In the context of caching, useful metrics include hit rate, miss rate, byte hit rate, latency, tail latency, throughput, and purge counts. This project exposes Prometheus-compatible metrics because observability is part of the design, not merely a helpful addition.

The key metrics are useful for different reasons. Hit rate measures how often the cache prevents an origin request. Byte hit rate measures how much traffic is offloaded from origin. Latency measures user-visible speed. Throughput measures system capacity under load. Purge and update counters measure whether lifecycle maintenance is happening.

This observability focus is also well grounded in prior work. Prometheus documentation continues to emphasise counters, histograms, and instrumentation practices for service-level monitoring, including the importance of histograms for latency distributions rather than only averages [34, 35]. More generally, standards work such as RFC 9211 reflects the growing recognition that caches are operational systems whose behaviour must be inspectable and explainable [36]. This is especially important in the present project because lifecycle-aware correctness is partly operational: it is not enough for the cache to be fast; the system must also make it visible when invalidation has happened and what effect it had.

2.7 Research Gap

Taken together, the background and literature discussed above suggest that Walrus-Cache-CDN should be understood as a synthesis of five different strands:

1. **web/CDN caching**: motivates locality, hit-rate, latency reduction, as well as reuse;
2. **cache policy research**: motivates admission-aware and size-aware caching rather than indiscriminate insertion;
3. **persistent embedded storage**: motivates the use of RocksDB as durable edge state rather than only volatile memory;
4. **event-driven materialisation**: motivates maintaining a local view of remote lifecycle state through an indexer;
5. **decentralised storage and blockchain state**: motivates why correctness depends on object lifecycle and chain-visible events rather than on simple TTLs alone.

Each of these areas has substantial prior work. The gap addressed by this dissertation lies in their integration: existing literature gives strong foundations for caching, admission control, embedded storage, and blockchain-visible state, but offers relatively little implementation-focused evidence on how these ideas work together for a Walrus-facing retrieval layer in particular.

The practical gap addressed by this dissertation can therefore be stated more precisely as follows:

> There is limited implementation-level evidence on how to build a Walrus-specific cache/edge layer that simultaneously improves repeated-read performance, preserves practical correctness under lifecycle changes, and remains observable and reproducible enough for systems evaluation.

This is not mainly a gap in cache theory, nor is it mainly a gap in blockchain consensus design. It is a **systems integration gap**. The dissertation addresses that gap by combining a Walrus-facing read path, persistent local payload and metadata storage, policy-aware cache admission, event-driven invalidation and metadata updates, and benchmarkable and inspectable metrics.

2.8 Chapter Summary

This chapter has established the context, background, and related work for the rest of the dissertation. It has shown why Walrus retrieval is a good candidate for edge caching, why Sui/Walrus lifecycle information matters for correctness, why payload identity and lifecycle state must be treated separately, and why the project should be understood as an event-aware and policy-aware edge layer rather than as a simple local byte cache.

It has also shown that the design of Walrus-Cache-CDN is grounded in several mature lines of prior work, including web caching, cache-policy research, persistent embedded state, event-driven systems, and decentralised storage. At the same time, there remains a clear implementation-level gap in showing how these ideas can be combined in a Walrus-specific prototype. The next chapter turns this contextual material into a concrete set of project requirements and analysis.

Chapter 3

Requirements and Analysis

3.1 Introduction

Chapter 2 established the technical context for the project: Walrus retrieval is expensive enough to benefit from caching, Sui provides lifecycle-relevant state and events, and prior work suggests that useful edge caching in this setting must combine performance optimisation with explicit correctness maintenance. This chapter translates that contextual material into a concrete statement of what the prototype is required to do and what design implications follow from those requirements.

Because Walrus-Cache-CDN is a systems prototype rather than a conventional end-user application, the requirements are expressed primarily in terms of system behaviour, lifecycle correctness, cache policy, observability, and evaluation support. The aim of the chapter is not to produce heavyweight software-oriented requirements, but to define the constraints and goals that directly shaped the architecture presented in Chapter 4.

3.2 Refined Problem Statement

Walrus provides decentralised blob storage and retrieval with strong durability and verifiability properties, but repeated reads may still incur substantial latency because origin retrieval can involve remote coordination, metadata lookup, sliver collection, and reconstruction. A client- or edge-side layer that repeatedly accesses the same blobs can therefore benefit from local reuse.

However, simple byte reuse is not enough. Cached payloads remain useful only if the system can also preserve practical correctness when lifecycle state changes. In the Walrus/Sui setting, this includes certification updates, explicit invalidation or deletion signals, and epoch-driven expiry conditions. The project problem can therefore be stated as follows:

> Design, implement, and evaluate a Walrus-facing cache/CDN-like edge layer that reduces repeated origin fetches and improves retrieval performance, while preserving practical correctness through lifecycle-aware metadata management, event-informed invalidation, and explicit observability.

This problem statement is broader than building a simple cache, but narrower than proposing a new decentralised storage protocol. It frames the dissertation as a systems integration problem in which performance, correctness, and evaluation must all be addressed together.

3.3 Requirements Derivation

The requirements for the prototype are derived from four main sources. The first is the project aims set out in Chapter 1, especially the goals of improving repeated-read performance, supporting multiple operating modes, and producing measurable evidence of benefit. The second is the contextual analysis of Chapter 2, which established that payload identity and lifecycle state must be treated separately, and that correctness maintenance must be event-informed rather than timer-only. The third is the practical scope of the dissertation as a systems prototype: the system must be realistic enough to be convincing, but also bounded enough to be implemented, demonstrated, and evaluated within the time available. The fourth is the evaluation ambition of the project, since the prototype must support reproducible comparison between variants rather than only informal demonstrations of functionality.

3.4 System Scope and Boundary

Walrus-Cache-CDN sits at the retrieval edge. It treats Walrus-facing endpoints as the authoritative origin and operates entirely as a client-side or edge-side enhancement layer. It does not modify the Walrus protocol, redefine Walrus storage semantics, or replace Sui as a source of lifecycle information.

Within this boundary, the prototype is responsible for retrieving blobs through Walrus-facing endpoints, maintaining local payload and metadata state, reacting to lifecycle-relevant signals through invalidation or metadata updates, and exposing enough metrics and deterministic behaviour to support rigorous evaluation.

Equally important is what lies outside scope. The project does not attempt to build a globally deployed production CDN, prove strong distributed consistency across many cache nodes, redesign Walrus storage economics, or perform a full adversarial security analysis. These exclusions define the level of ambition of the prototype and prevent the dissertation from making claims beyond what is actually implemented.

3.5 Functional Requirements

To keep the requirements concrete and traceable to the implemented prototype, the main functional requirements are summarised in Table 3.1. These requirements are phrased at the system level rather than as user-interface requirements, since Walrus-Cache-CDN is primarily a command-line systems prototype.

Table 3.1: Structured functional requirements for Walrus-Cache-CDN. The requirement set is aligned with the implemented command surface and code modules rather than with a graphical user interface.

ID	Area	Requirement
FR1	Retrieval	The system shall retrieve blobs by blob ID through Walrus-facing endpoints using the `get` command.
FR2	Retrieval	The system shall support retrieval by object ID through the `get-by-object` command. This path is functionally supported but is not the primary cached path.
FR3	Baseline mode	The system shall support cache-enabled and cache-disabled execution so that direct-origin and cache-assisted behaviour can be compared.
FR4	Persistent state	The system shall maintain a persistent RocksDB-backed store for payload bytes, metadata, and secondary tag records.
FR5	Hot tier	The system shall maintain a bounded in-memory hot tier that accelerates repeated accesses to recently or repeatedly requested objects.
FR6	Metadata	The system shall store lifecycle-aware metadata separately from payload bytes.
FR7	Invalidation	The system shall support purge-by-blob-ID, purge-by-tag, and purge-by-epoch-threshold operations through `cache-purge`.
FR8	Event ingestion	The system shall support deterministic simulated event ingestion through `index` in simulated mode.
FR9	Event ingestion	The system shall support live Sui-backed event ingestion when compiled with the `sui-live` feature.
FR10	Lifecycle maintenance	The system shall translate observed lifecycle events into metadata updates or invalidation side-effects.
FR11	Upload support	The system shall support blob upload through the `put` command so that test objects and demonstrations can be created within the prototype itself.
FR12	Demonstration	The system shall support deterministic miss→hit→invalidate flows through the `demo-flow` command.
FR13	Policy comparison	The system shall support multiple admission or promotion modes, including `None`, `Lru`, and `WTinyLFU`.
FR14	Evaluation	The system shall support reproducible synthetic evaluation through the `evaluate` command.
FR15	Observability	The system shall expose counters and latency metrics so that serving behaviour and invalidation activity can be inspected.

These requirements can be grouped conceptually into retrieval, local-state management,

lifecycle maintenance, observability, and evaluation support. The most important detail is that the prototype is required not only to cache data, but to maintain and expose the state needed to cache it responsibly.

3.6 Non-Functional Requirements

Beyond functionality, several non-functional properties are central to the value of the prototype.

The most important is **practical correctness**. The system is not expected to prove formal distributed consistency, but it must remain operationally sensible: if lifecycle state changes in a way that should affect cached reuse, the system must reflect that change in a visible and defensible way.

The second is **performance improvement**. The prototype should materially improve repeated-read behaviour compared with direct origin retrieval. This includes lower average latency, meaningful hit-rate, and higher effective throughput under workloads with reuse.

The third is **durability**. Useful local state should survive process restart where practical, especially metadata and cached payloads, so that warm-start behaviour is possible.

The fourth is **reproducibility**. The system should support deterministic demonstrations and repeatable evaluation, making it possible to explain and defend its behaviour.

The fifth is **configurability**. The prototype should support multiple operating modes through configuration rather than code changes, including simulated versus live event sources and cache-enabled versus cache-disabled variants.

3.7 Operational Scenarios

For a system such as Walrus-Cache-CDN, operational scenarios are more useful than classical end-user use cases. The most important scenarios are as follows. In a **cold retrieval**, the requested blob is not present in local state, so the system must fetch it from origin, persist the result, and possibly promote it into the hot in-memory tier. This establishes the miss path. In a **warm retrieval**, a previously requested blob is served from either the in-memory hot tier or the persistent RocksDB-backed store. This establishes that the cache actually provides reuse and lower-latency service. In a **lifecycle update**, an observed event alters retention-related metadata associated with a cached object. This distinguishes the prototype from a naive payload-only cache and demonstrates the existence of a real control plane. In an **invalidation scenario**, a blob becomes inappropriate to serve from cache because of explicit purge, grouped maintenance, or lifecycle expiry. The system must remove or invalidate the relevant local state so that later accesses no longer observe a stale hit. In a **deterministic demonstration**, a controlled miss→hit→invalidate sequence is executed through the CLI so that both data-plane and control-plane behaviour can be explained clearly. Finally, in a **policy comparison scenario**, the same or closely comparable workload is executed under different promotion modes so that differences in hit-rate, latency, and scan resistance can be measured.

3.8 Use-Case View

Although the project is not strongly user-interface oriented, it is still useful to represent the main interactions with the prototype at a high level. Figure 3.1 summarises the principal operational use cases supported by the current codebase.

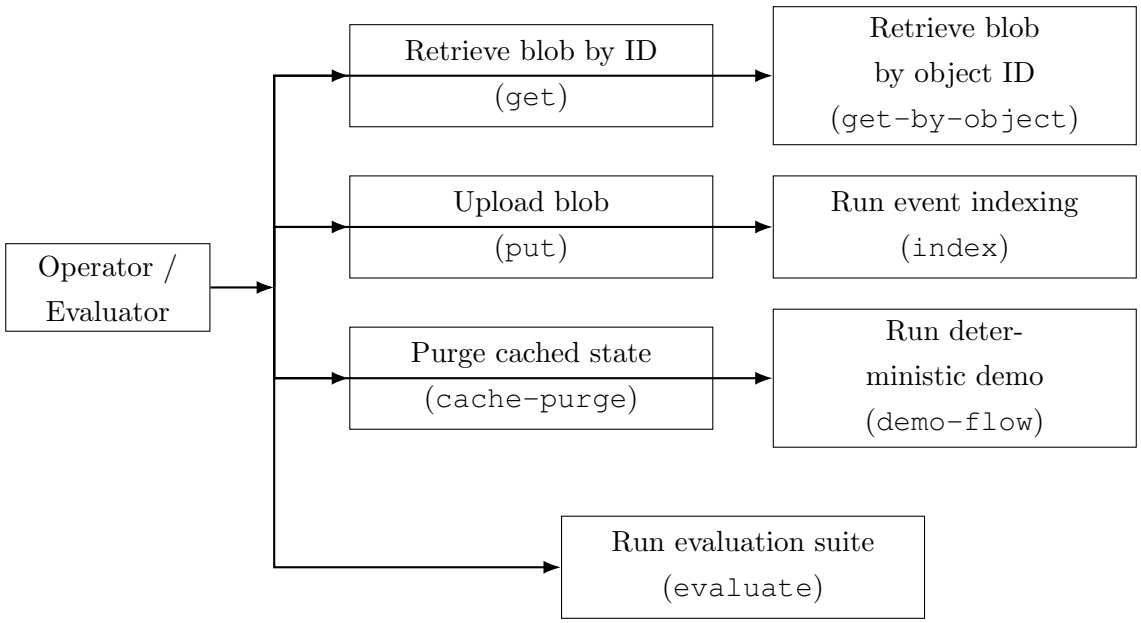

Figure 3.1: High-level operational use cases for Walrus-Cache-CDN. The prototype is driven primarily through CLI commands rather than through a graphical user interface, so the main use cases correspond directly to the implemented command surface.

The most important use cases are retrieval, event indexing, invalidation, deterministic demonstration, and evaluation. Upload is also included because it makes the prototype self-contained: the same executable can create test data as well as retrieve and cache it. The object-ID retrieval path is shown separately because it is supported by the prototype, but unlike blob-ID retrieval it is not the main cached path in the current implementation.

3.9 Analysis of the Requirements

The requirements have several direct design consequences.

First, the requirement to preserve correctness under lifecycle change implies that payload bytes and lifecycle metadata cannot be treated as the same kind of state. Payload identity is what makes caching possible, while lifecycle metadata determines whether cached payload remains appropriate to serve. This implies distinct storage and update paths for bytes, metadata, and invalidation-related indexing information.

Second, the requirement to improve latency while preserving useful state across restart implies that a single-tier cache would be insufficient. A purely in-memory cache would provide fast hits but would lose all state on restart, while a purely persistent cache would preserve state

but would not provide the lowest-latency path for the hottest objects. This motivates a hybrid RAM-plus-persistent design.

Third, the requirement to remain correct under lifecycle change implies that invalidation must be treated as a first-class design concern rather than an implementation detail. This leads directly to explicit support for purge-by-ID, purge-by-tag, and epoch-threshold invalidation, and to an event-ingestion path that can trigger cache-maintenance actions.

Fourth, the requirement to compare admission strategies implies that the cache is part of the research object rather than merely a storage convenience. Policy configuration must therefore be explicit in the design, and evaluation must be able to isolate the effect of different policy choices.

Finally, the requirement for reproducible evaluation implies that observability must be built into the system from the start. Metrics, deterministic demo flows, and benchmarkable execution paths are not optional support features – they are part of the prototype's core purpose.

3.10 Analysis Model of the System State

The requirements analysis can also be summarised as a conceptual state model. Figure 3.2 shows the main entities and relationships that the architecture must support.

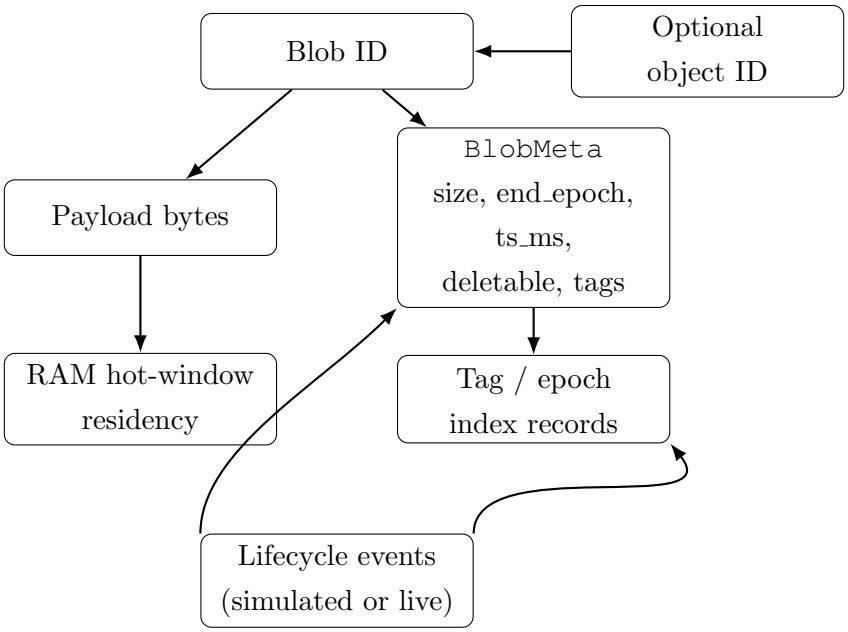

Figure 3.2: Conceptual analysis model of the main system state. Blob ID is the central identity used for the primary cached path; payload bytes, lifecycle metadata, RAM residency, and auxiliary index records are related but distinct forms of local state.

This model captures the central analytical point of the dissertation – the fact that the prototype is not merely caching a mapping from key to bytes. It is maintaining a small local state system in which payload data, lifecycle-aware metadata, hot-tier residency, optional object-ID linkage, and invalidation-oriented indexes are represented explicitly and can also be affected by observed lifecycle events.

3.11 Requirement Prioritisation

Not all requirements carry the same weight. The essential requirements are those without which the dissertation's core claim would fail: Walrus-facing retrieval, hybrid local caching, lifecycle-aware metadata, explicit invalidation, observability, and reproducible evaluation. Important but secondary requirements strengthen the prototype and deepen the evaluation, including multiple admission modes, size-aware admission, live Sui-backed event ingestion in addition to simulated mode, and grouped invalidation through tags. Extension-oriented requirements, such as request coalescing, more sophisticated prefetching, broader distributed routing behaviour, and full multi-node deployment, are natural continuations of the work but not necessary for this dissertation's main contribution.

3.12 Chapter Summary

This chapter has translated the goals of the dissertation and the contextual findings of Chapter 2 into a concrete set of requirements and design-driving observations. The core point is that Walrus-Cache-CDN is not merely required to cache data; it is required to cache data correctly, under observable lifecycle change, with enough structure and instrumentation to support meaningful systems evaluation.

The analysis also showed that these requirements directly motivate the key architectural choices of the project: separation of payload and metadata, a hybrid cache rather than a single-tier design, explicit invalidation support, policy-aware admission, and built-in observability. The next chapter presents the resulting system architecture as a direct response to those requirements.

Chapter 4

System Design and Architecture

4.1 Introduction

The previous chapter identified the main requirements of the prototype: repeated-read performance should improve relative to direct origin retrieval, lifecycle-aware correctness should be preserved, multiple cache policies should be comparable, and the resulting system should remain observable and reproducible enough for credible evaluation. This chapter explains how those requirements were turned into an actual design.

The overall design of Walrus-Cache-CDN is intentionally modest in deployment scope but non-trivial in systems structure. It is modest in the sense that it is a single-process, primarily single-node prototype rather than a globally distributed CDN. At the same time, it is non-trivial because it is not merely a key-value cache inserted in front of a network call. It combines a retrieval path, a persistent metadata path, an invalidation path, an event-ingestion path, and an observability layer. The design problem is therefore one of coordination: each subsystem must remain simple enough to be implemented clearly, while still fitting into a coherent whole.

4.2 Design Goals

Five design goals guided the architecture.

The first was to improve repeated retrieval performance without modifying Walrus itself. This meant the system had to be positioned entirely at the client or edge side and had to treat Walrus retrieval endpoints as the authoritative origin.

The second was to preserve practical correctness under lifecycle change. Because Walrus-facing objects may become inappropriate to serve due to deletion, expiry, or updated retention state, invalidation and metadata maintenance had to be part of the core design rather than optional support logic.

The third was to support both deterministic experimentation and more realistic operation. For that reason, the architecture had to accommodate both a simulated event source and a live Sui-backed event source behind a common interface.

The fourth was to remain observable. A system that improves latency but cannot show whether a request came from RAM, persistent cache, or origin is much harder to evaluate and defend in a dissertation setting.

The fifth was to remain implementable within the scope of this final-year project. This ruled out a full distributed control plane, but did not rule out modularity, persistence, or policy-awareness.

4.3 Architectural Overview

At the highest level, Walrus-Cache-CDN can be understood as a command-line driven edge retrieval system built from six cooperating subsystems: a CLI and orchestration layer, a Walrus client, a hybrid cache, a metadata and invalidation subsystem, an event-ingestion subsystem, and a metrics subsystem. These are not independent programs; they are coordinated parts of one executable. Their separation is architectural rather than deployment-based. In particular, the cache and indexer are not external services: they are internal subsystems whose interactions are mediated by shared runtime state inside the prototype process.

Figure 4.1 shows the top-level coordination structure of the prototype. The figure should be read as a coordination view rather than as a deployment diagram: the modules are separated by responsibility, but they execute within one prototype binary and interact through shared in-process state and control flow.

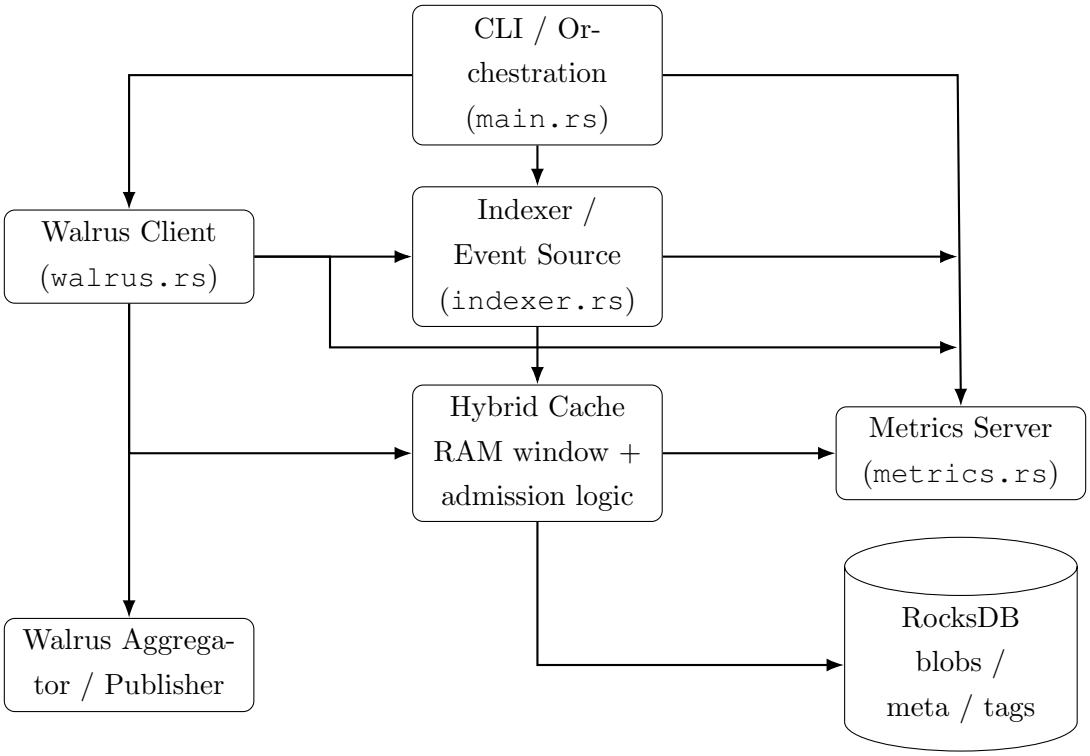

Figure 4.1: Top-level architecture of Walrus-Cache-CDN. The CLI coordinates retrieval, upload, indexing, purge, demo, and evaluation commands; the data plane uses a hybrid cache in front of Walrus origin endpoints; the control plane updates cache state via metadata and event-driven invalidation.

Although this figure shows the main components, it is also helpful to view the architecture in terms of layers rather than only modules. Figure 4.2 presents that layered interpretation.

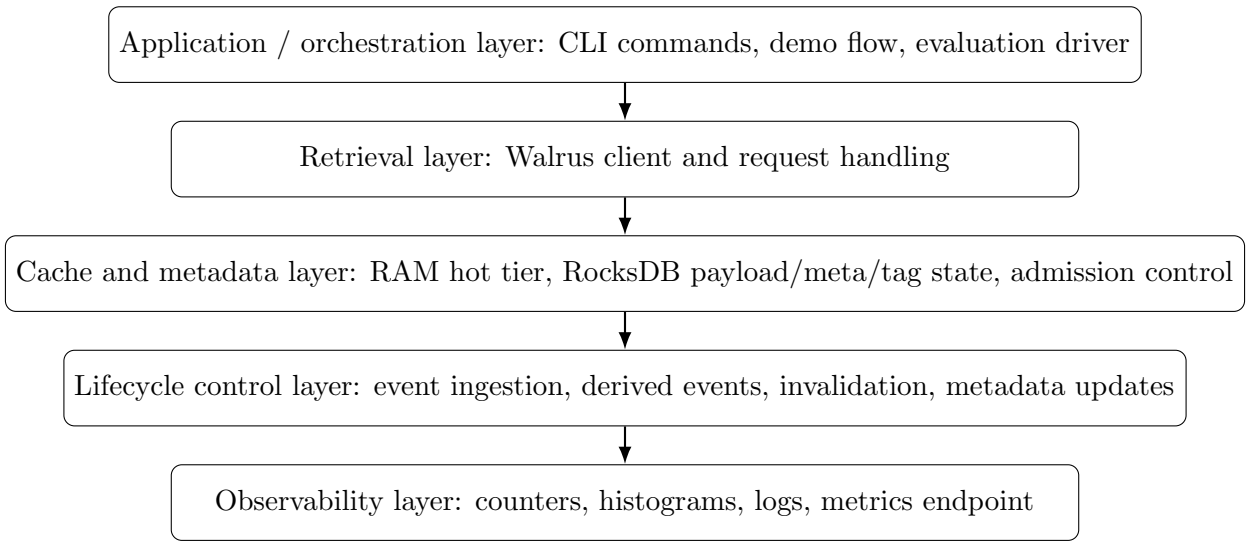

Figure 4.2: Layered interpretation of the architecture. The prototype is not only a retrieval cache; it combines orchestration, retrieval, local state, lifecycle control, and observability.

This layered view makes the central architectural claim clearer: Walrus-Cache-CDN is best understood as a small edge system rather than a single cache object.

4.4 Data Plane and Control Plane

The architecture is easiest to reason about when separated into a data plane and a control plane, but this distinction must be made in a way that reflects the implemented prototype accurately.

The **data plane** is responsible for serving user-visible blob retrievals. When a blob is requested by blob ID, the system first checks the in-memory hot window, then the persistent RocksDB-backed store, and only then falls back to Walrus origin retrieval. On an origin miss, the returned payload is written into persistent local state together with its metadata, after which the configured policy determines whether it should also be promoted into the in-memory hot tier. The runtime design is therefore best described as **persistent write-through on miss, plus selective hot-tier promotion**. This is more precise than saying that admission alone decides whether the object enters cache at all.

The **control plane** is responsible for maintaining correctness and operational state. It includes metadata storage, tag-index maintenance, purge operations by blob ID, tag, and epoch threshold, event ingestion from simulated or live sources, derived lifecycle signals such as **Extended** and **Expired**, and metrics exposure. The performance of the dissertation is therefore mainly to do with the data-plane, while the correctness is mainly related to the control-plane.

4.5 Request Path Design

The main cached retrieval path is keyed by blob ID and implements a strict serving precedence:

RAM hot window → RocksDB persistent store → Walrus origin.

This precedence reflects a deliberate trade-off. RAM is the fastest serving layer, but it is intentionally small and volatile. RocksDB is slower than RAM but survives restart and supports structured metadata and secondary indexes. Walrus origin retrieval is authoritative but comparatively expensive. Figure 4.3 shows this request path, including the post-miss steps of persistent storage and selective hot-tier promotion.

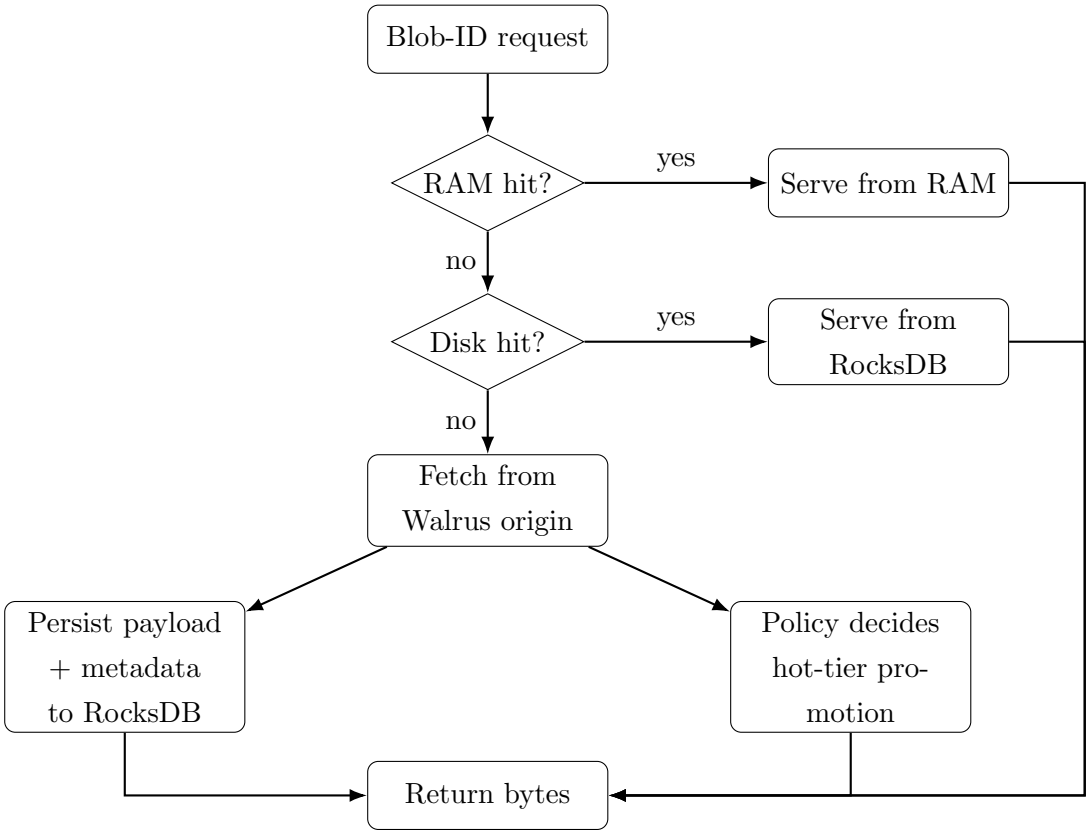

Figure 4.3: Blob-ID retrieval path. The system first checks the RAM hot window, then the persistent RocksDB-backed local store, and only then falls back to Walrus origin retrieval. On an origin miss, the runtime path persists the object in RocksDB and then applies policy logic to decide whether the object should also be promoted into the in-memory hot tier.

This figure clarifies an important aspect of the implemented runtime path: policy does not primarily decide whether an object enters persistent local state at all. Instead, on an origin miss, the object is first written into the RocksDB-backed store together with its metadata, after which policy mainly influences whether the object is also promoted into the bounded RAM hot tier.

The object-ID retrieval path should also be treated separately. The prototype supports retrieval by object ID through the `get-by-object` command, but in the current implemen-

tation this path bypasses the cache and fetches directly from the Walrus aggregator. For that reason, blob-ID retrieval remains the main cached path discussed throughout the dissertation.

4.6 Hybrid Cache Design

A central architectural decision is the use of a hybrid cache rather than a purely in-memory or purely persistent design. This follows directly from the requirements analysis of Chapter 3: the system needs both a very fast path for the hottest objects and persistent local state that survives restart and supports lifecycle-aware maintenance.

The in-memory tier is best described as a **bounded hot window** rather than as a strict LRU cache. In the implementation it is represented using a deque-like structure that stores a small subset of promoted objects. The purpose of this tier is not to hold the full cache working set, but to accelerate the hottest portion of it.

The persistent tier is the real backbone of the runtime cache. It stores payload bytes, lifecycle metadata, and tag-related secondary records in RocksDB. This means that even when an object is not present in the RAM hot window, it may still be available locally from persistent storage without requiring origin retrieval.

The consequence is that the architecture is not best understood as a small RAM cache backed by a remote origin. It is better understood as a persistent local cache with a bounded RAM accelerator layered on top of it. Figure 4.4 summarises that relationship.

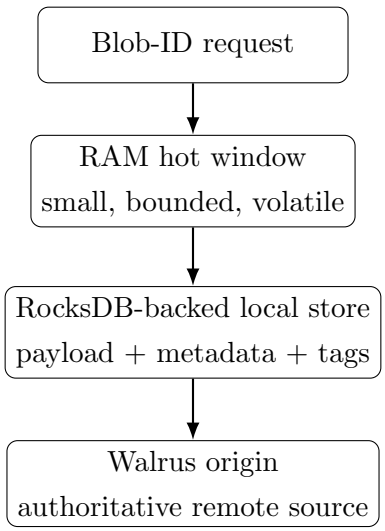

Figure 4.4: Hybrid cache tiering. The persistent RocksDB-backed store is the central local cache, while the RAM hot window acts as a bounded accelerator tier above it.

4.7 Metadata Layout and State Representation

A particularly important architectural choice is the separation of payload bytes from metadata and tag-index state. In the persistent cache, these are stored in three logical categories: cached object bytes, serialised lifecycle metadata, and secondary tag records used for grouped

invalidation.

This separation is not merely stylistic. It allows lifecycle metadata to be updated without rewriting payload bytes, enables grouped purge operations through secondary-index scans, and supports epoch-threshold invalidation without inspecting object bodies. Figure 4.5 summarises the internal state layout.

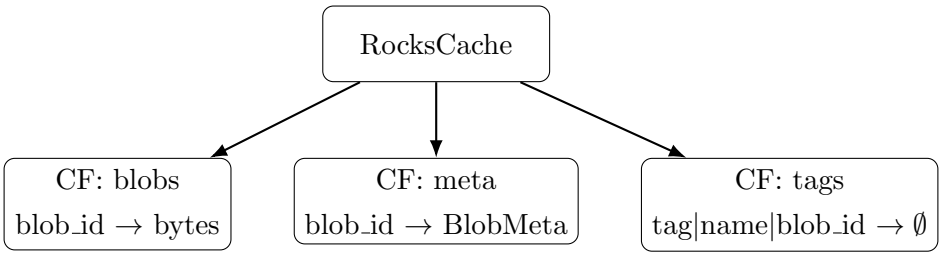

Figure 4.5: Logical cache state layout. Payload bytes, lifecycle metadata, and secondary tag records are stored separately so that serving, update, and invalidation paths can remain simple and explicit.

Conceptually, this state model reflects the distinction established earlier between immutable payload identity and mutable lifecycle state. The architecture therefore encodes one of the project's central ideas directly into storage layout.

4.8 Admission and Replacement Design

The architecture does not treat policy as an abstract optimisation layer detached from implementation reality. Instead, policy is applied in a way that reflects how the runtime cache actually behaves.

The system supports three policy modes: `None`, `Lru`, and `WTinyLFU`. These should not be interpreted as three complete cache implementations of equal scope. In the current runtime design, objects fetched on a miss are first persisted into RocksDB, subject to hard limits such as maximum admissible object size. The configured policy then mainly influences whether the object is promoted into the in-memory hot tier, and how aggressively repeated access leads to RAM residency.

This means that `AdmissionMode::None` is not the same as disabling the cache. Cache disablement is controlled separately through configuration. In runtime code, `None` still allows persistent local storage, while omitting the stronger policy heuristics used in the other modes.

Similarly, the `WTinyLFU` path is best described as **WTinyLFU-inspired** or **frequency-aware promotion**, rather than as a full production implementation of W-TinyLFU. The code uses a Count-Min sketch and victim-frequency comparison, but it does not implement the full protected-region structure of a mature industrial cache.

The architectural importance of these modes is therefore two-fold. First, they shape hot-tier behaviour in the real runtime system. Second, they provide policy variants for the synthetic evaluator, where policy effects can be studied more explicitly under controlled workloads.

4.9 Event Ingestion and Lifecycle Maintenance

Correctness in this project depends on more than cache lookup order. The system also needs a way to react when lifecycle state changes. This is handled by the event-ingestion subsystem.

A common `EventSource` abstraction allows both simulated and Sui-backed live streams to be consumed using the same maintenance logic. This keeps the control plane structurally uniform: deterministic testing and more realistic live operation differ mainly in event source, not in the high-level update path.

Simulated mode emits a fixed and controlled sequence of events, which is valuable for demonstrations, debugging, and reproducibility. Live Sui-backed indexing is available only when the prototype is compiled with the `sui-live` feature. In that mode, the system polls real event data through Sui RPC and converts matching observations into the project's internal event representation.

The design also derives higher-level lifecycle signals. In particular, `Extended` is derived when a later certification raises the object's end epoch, while `Expired` is derived when observed epoch progression overtakes stored retention state. This allows the control plane to operate on semantically meaningful lifecycle transitions rather than on a raw and potentially heterogeneous event stream.

Figure 4.6 shows how external lifecycle observations are translated into local cache side-effects.

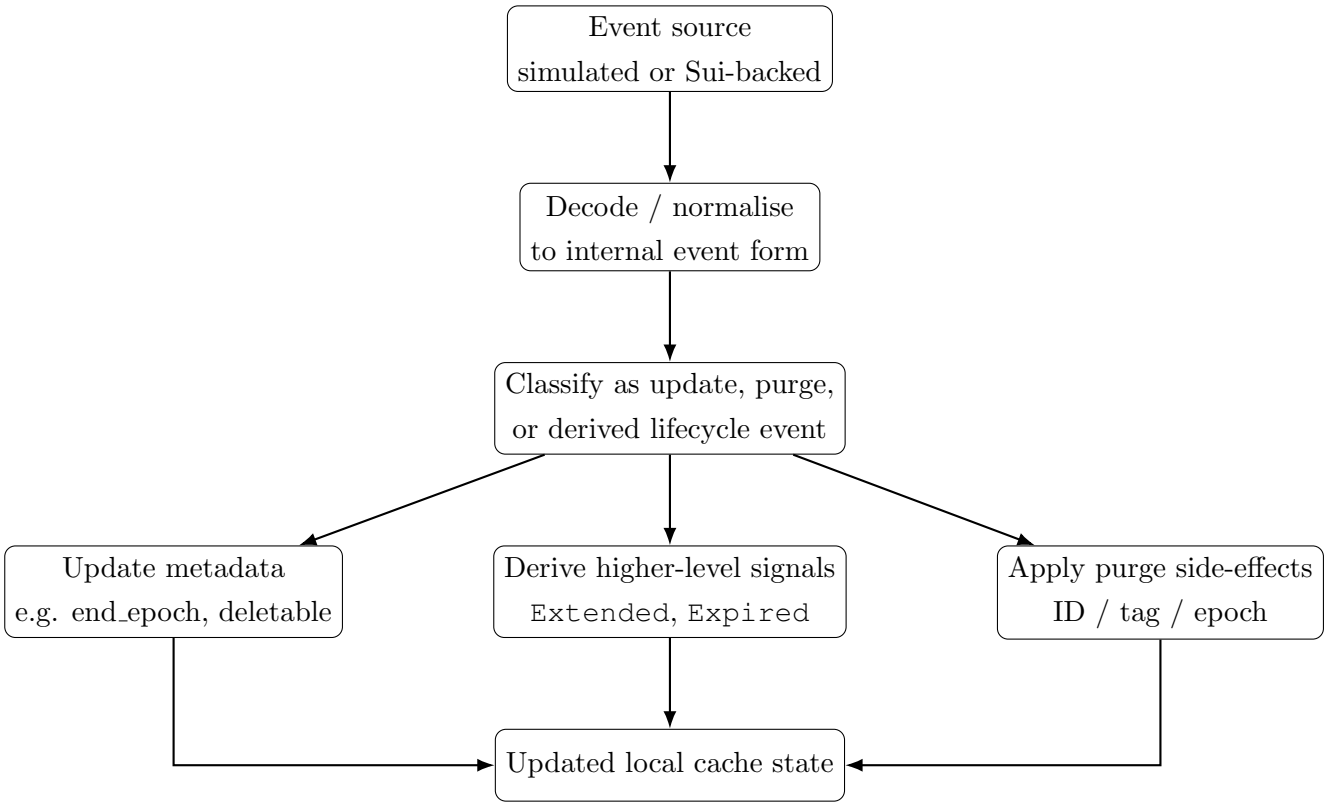

Figure 4.6: Lifecycle-control path. External events are normalised, classified, and turned into metadata updates, derived lifecycle transitions, or invalidation side-effects.

4.10 Invalidation Model

The architecture supports three invalidation modes: purge by blob ID, purge by tag, and purge by epoch threshold. This is stronger than a simple TTL-based design because it allows invalidation to be expressed in forms that correspond directly to the correctness risks identified in Chapters 2 and 3.

Exact invalidation by blob ID addresses direct object-level removal. Group invalidation by tag addresses collections of related objects. Epoch-threshold invalidation addresses lifecycle and retention-horizon maintenance. Together, these operations form the operational core of the control plane's correctness story.

4.11 Configuration and Operating Modes

The system is configured through environment variables rather than through source-level changes. This includes network selection, aggregator and publisher base URLs, indexer mode, Sui RPC endpoint, cache enablement, persistent cache path, block-cache capacity, admission mode, optional size-aware gating, metrics listen address, and optional node lists for sharding experiments.

This design choice matters because it allows the same executable to run in multiple roles: purely local testing, testnet-backed experimentation, cache-disabled baseline mode, cache-enabled edge mode, deterministic demo mode, and evaluation mode. Configuration therefore supports both reproducibility and fair comparison across variants.

4.12 Extension for Multi-Node Placement

Although the dissertation prototype is primarily single-node, the architecture includes an extension point for multi-node placement through rendezvous-hash-style node selection. This functionality is implemented through `sharding::pick_node`.

It is important, however, to characterise this feature accurately. In the current prototype, sharding support is advisory and lightly integrated rather than part of a full distributed placement subsystem. The architecture therefore does not claim to provide a real distributed cache fabric, replicated control plane, or global edge coordination mechanism. Instead, it demonstrates that the keying model and per-object node-selection logic are already compatible with future horizontal extension.

This extension point was worth discussing here because it shows that the system was built with a plausible forward path toward multi-node deployment, even though that deployment model lies beyond this dissertation's scope.

4.13 Observability as a Design Concern

Observability is not bolted on after the fact. The architecture includes counters for cache hits, misses, writes, bytes served, bytes written, and purge activity, along with latency histograms labelled by serving source. This is crucial because the dissertation needs to show not only that performance improves, but also how and where requests are being served, and whether invalidation has had the expected operational effect. In this sense, observability belongs equally to the data plane and the control plane: it explains performance behaviour while also exposing correctness maintenance.

4.14 Failure Modes and Architectural Limits

The architecture makes practical assumptions rather than idealised ones. It assumes that Walrus origin endpoints are reachable on miss, that the local RocksDB database remains available, that observed events are sufficiently timely to support practical correctness, and that the cache operates within one local process.

These assumptions imply clear limitations. Delayed event visibility can delay invalidation. The design does not provide global multi-edge coherence. Correctness is operational and event-informed rather than formally strong-consistency preserving. Finally, the prototype does not implement a full distributed control plane. These limits are acceptable in this dissertation prototype, but they matter because they define the boundary of the claims made later in evaluation.

4.15 Architecture Summary

The architecture of Walrus-Cache-CDN is intentionally modest but technically meaningful. It combines a fast retrieval path, durable local state, explicit metadata, lifecycle-aware invalidation, event ingestion, and observability into one coherent system. The key design decision is not any single cache policy or storage choice in isolation, but the integration of all of these elements into a Walrus-facing edge layer that improves repeated retrieval performance without discarding lifecycle-aware correctness concerns.

Chapter 5

Implementation

5.1 Implementation Overview

Walrus-Cache-CDN is implemented as a Rust command-line application. Rust is a strong fit for this project because the system combines several concerns that benefit from systems-level control: byte-oriented retrieval, embedded persistent storage, structured metadata handling, asynchronous network access, concurrency control, and low-overhead instrumentation.

The implementation is delivered as a single executable, but it is internally modular. The main source files are `main.rs`, which defines the CLI surface and orchestration logic; `walrus.rs`, which implements origin-facing Walrus operations and the main cached retrieval path; `cache.rs`, which implements the persistent store, hot-tier logic, metadata handling, tag indexes, and invalidation methods; `indexer.rs`, which provides simulated and live event ingestion; `metrics.rs`, which exposes Prometheus-compatible metrics over HTTP; `config.rs`, which centralises environment-driven configuration; `sharding.rs`, which contains the rendezvous-style node-selection helper; and `eval.rs`, which contains the reproducible synthetic evaluation harness.

This modular structure matters because it mirrors the design distinction introduced in Chapter 4. The project is not simply one function that fetches and caches bytes, but a coordinated implementation of both a data plane and a control plane.

5.2 CLI and Program Entry Point

The executable is structured as a multi-command CLI, which makes the prototype directly usable both for experimentation and for dissertation evaluation. The implemented command surface includes retrieval commands, upload commands, lifecycle-maintenance commands, deterministic demo commands, and evaluation commands.

The most important commands are `get`, which retrieves a blob by Walrus blob ID; `get-by-object`, which retrieves by object ID; `put`, which uploads a blob through a publisher endpoint; `index`, which runs event ingestion in either simulated or live mode; `cache-purge`, which invalidates by blob ID, tag, or epoch threshold; `cache-set-epoch`, which sets deterministic epoch metadata for invalidation demos; `demo-flow`, which runs a guided miss→hit→invalidate sequence; and `evaluate`, which runs the synthetic dissertation evaluation suite.

This CLI surface is more than a convenience layer. It is part of the implementation strategy for reproducibility. The same binary supports interactive exploration, deterministic demonstrations, live event observation, and controlled evaluation, which is exactly what a dissertation-scale systems prototype requires.

5.3 Configuration Handling

Runtime configuration is centralised in `AppConfig` and built from environment variables in `config.rs`. The configuration covers network selection, Walrus aggregator and publisher endpoints, indexer mode, Sui RPC parameters, cache enablement, RocksDB cache path, block-cache size, admission mode, optional size-aware gating, metrics bind address, and optional cache-node lists.

This approach keeps the executable flexible without forcing configuration logic into each subsystem independently. It also supports fair comparison between modes. For example, cache enablement is configured separately from admission mode, which is important because disabling the cache entirely is not the same thing as running with `AdmissionMode::None`. Likewise, simulated and live indexing are selected through configuration, while live Sui indexing additionally depends on compilation with the `sui-live` feature.

In practice, this configuration model supports the full range of modes needed by the dissertation: cache-disabled baseline runs, cache-enabled edge runs, deterministic demo flows, synthetic evaluation runs, and optional live indexing experiments.

5.4 Core Cache Data Structures

The cache implementation in `cache.rs` makes the architectural decisions of the previous chapter concrete. The most important starting point is the explicit admission-policy enum:

Listing 5.1: Admission modes supported by the cache implementation.

```
#[derive(Clone, Copy, Debug)]
pub enum AdmissionMode {
    WTinyLFU,
    Lru,
    None,
}
```

This enum is important because the prototype exposes policy as a first-class configuration choice rather than hard-coding one cache behaviour. In the runtime cache, they do not act as three fully separate cache implementations. Instead, they mainly affect promotion into the in-memory hot tier after persistent insertion, subject to hard limits such as maximum admissible object size.

The second core structure is the lifecycle-aware metadata record stored separately from payload bytes:

Listing 5.2: Lifecycle-aware metadata stored alongside cached payloads.

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct BlobMeta {
    pub size: u64,
    pub end_epoch: Option<u64>,
```

```
    pub ts_ms: Option<u64>,
    pub deletable: Option<bool>,
    pub tags: Vec<String>,
}
```

This structure is important because it encodes one of the project's central ideas directly in code: the cached payload and the lifecycle metadata are not the same thing. The bytes are what the caller wants to retrieve, while the metadata is what the system needs in order to serve, update, or invalidate those bytes responsibly.

The cache itself is implemented in the `RocksCache` structure, which combines a RocksDB instance, a bounded in-memory hot window, a small map used to track repeated disk hits, a Count-Min sketch for approximate frequency estimation, policy configuration, and optional size-aware gating. This structure mirrors the architectural model quite closely: persistent state, hot-tier acceleration, and policy state are brought together in one module, but remain logically distinct.

5.5 RocksDB Layout and Opening Logic

The persistent cache is opened using explicit RocksDB options and three named column families: `blobs`, `meta`, and `tags`. This structure reflects the design decision to separate payload bytes, lifecycle metadata, and secondary invalidation records.

The implementation does not use RocksDB as a naive append-only backing store. Instead, it configures point-lookup optimisation, a configurable block cache, bloom filtering, index and filter block caching, and missing-column-family creation. These options are appropriate for a retrieval-oriented workload in which repeated point reads are common and metadata lookups matter almost as much as payload retrieval.

This section of the implementation is important because it shows that persistence is not treated as an afterthought. RocksDB is part of the actual runtime cache path, not just a recovery mechanism for rare cases.

5.6 Read Path Implementation

The concrete read path spans both `walrus.rs` and `cache.rs`. The public-facing retrieval function is `WalrusClient::get_blob_by_id`, which first checks the cache and only then falls back to origin.

A simplified excerpt is shown below:

Listing 5.3: Simplified blob retrieval flow with cache lookup and origin fallback.

```
pub async fn get_blob_by_id(&self, blob_id: &str) -> Result<Vec<u8>> {
    if let Some(c) = &self.cache {
        match c.lock().unwrap().get(blob_id)? {
            CacheGet::RamHit(b) => return Ok(b.to_vec()),
```

```
            CacheGet::DiskHit(b) => return Ok(b.to_vec()),
            CacheGet::Miss => {}
        }
    }

    let url = self.base.join(&format!("v1/blobs/{blob_id}"))?;
    let res = self.http.get(url).send().await?.error_for_status()?;
    let body = res.bytes().await?;

    if let Some(c) = &self.cache {
        let meta = BlobMeta {
            size: body.len() as u64,
            end_epoch: None,
            ts_ms: None,
            deletable: None,
            tags: vec!["module:walrus".into()],
        };
        c.lock().unwrap().put_if_admit(blob_id, body.clone(), &meta)?;
    }

    Ok(body.to_vec())
}
```

The cached path described here applies to blob-ID retrieval. By contrast, the current `get_blob_by_object_id` path retrieves directly from the aggregator and does not currently pass through the cache, which is why blob-ID retrieval remains the main cached path discussed throughout the dissertation. This code fragment is useful because it demonstrates the precise control flow that the architecture chapter described abstractly. It also makes clear that the cache behaves as an edge layer in front of Walrus rather than as a replacement for Walrus.

5.7 RAM Hits, Disk Hits, and Promotion Logic

The actual cache lookup occurs in `RocksCache::get`. The function first increments the frequency sketch, then checks the in-memory window, then checks the RocksDB blob store.

A noteworthy implementation detail is the promotion rule after a disk hit. Disk-resident objects are promoted into the in-memory window after repeated disk hits or once their estimated access frequency crosses a small threshold. This is a useful design choice because it prevents every single disk-resident object from immediately entering RAM, while still allowing genuinely hot objects to migrate to the fastest tier.

In other words, the RAM layer behaves as a bounded promotion-based acceleration tier rather than as either a simple mirror of persistent state or a strict LRU cache in its own right.

5.8 Admission Logic in Practice

The admission decision is implemented in `RocksCache::admit`. In simplified form:

Listing 5.4: Admission logic for the three cache modes.

```
fn admit(&mut self, key: &str, size: usize, victim_freq: u32) -> bool {
    match self.admission {
        AdmissionMode::None => size <= MAX_ADMIT_BYTES,
        AdmissionMode::Lru => size <= MAX_ADMIT_BYTES && self.adaptsize_pass(
   size),
        AdmissionMode::WTinyLFU => {
            let cand = self.sketch_est(key);
            size <= MAX_ADMIT_BYTES && cand >= victim_freq && self.
   adaptsize_pass(size)
        }
    }
}
```

Several things are happening here:

1. In the concrete cache implementation, `None` mode behaves as a minimal admission baseline subject only to hard limits such as maximum object size. In the separate synthetic evaluator, however, the `None` policy is used as a deliberately weak comparison point and therefore should be interpreted as an evaluation baseline rather than as a byte-for-byte reproduction of the runtime cache path.

2. `Lru` uses recency-oriented promotion plus optional size-aware gating.

3. `WTinyLFU` compares candidate frequency against the resident victim frequency and only admits if the candidate is competitive.

4. Optional size-aware gating can bias against oversized objects.

This implementation is intentionally lightweight. It is not a fully general cache-policy framework, but it is sufficiently explicit to support meaningful policy comparison in the evaluation chapter.

5.9 Frequency Sketch Implementation

The implementation uses a compact Count-Min sketch with 8-bit counters:

Listing 5.5: Compact Count-Min sketch used for approximate frequency estimation.

```
struct CountMin8 {
    width: usize,
    depth: usize,
    table: Vec<u8>,
    salts: Vec<u64>,
}
```

The sketch is updated on every lookup and queried during admission decisions. This is important because it provides the practical mechanism behind the dissertation's WTinyLFU-style policy. Rather than tracking exact counts for all objects, the implementation uses a compact approximate structure, which is entirely consistent with the engineering motivation behind TinyLFU.

5.10 Write Path and Cache Population

The implementation supports uploading blobs through a publisher endpoint using `put_blob` and `put_blob_with_details`. These methods parse Walrus publisher responses and extract not only the `blobId`, but also optional object IDs and transaction digests where present.

This upload path is useful for two reasons. First, it makes the prototype self-contained: the same executable can create its own test objects. Second, it powers the deterministic `demo-flow` command, which uploads a blob and then walks through repeated gets and invalidations in a single process.

5.11 Metadata and Secondary Index Maintenance

The metadata and invalidation model becomes concrete in a small number of cache methods that update local state in place.

The function `update_meta_epoch` reads an existing metadata record, updates lifecycle-related fields such as `end_epoch` and `deletable`, adds an epoch tag where needed, and writes the updated record back to the metadata column family. This makes it the main bridge between externally observed lifecycle information and durable local cache state.

The function `add_tags` materialises secondary records of the form

```
tag|<tag>|<blob_id>
```

in the `tags` column family. These records are deliberately redundant with respect to the metadata object, but that redundancy is useful: it allows grouped invalidation to be implemented as a prefix scan over tag keys rather than as a full scan of all metadata records.

Together, these methods show that the control plane is not separate from persistence. Lifecycle updates are written into the same local state system that later serves requests.

5.12 Invalidation Implementation

The cache supports three main invalidation paths, and each corresponds directly to a design choice discussed in Chapter 4.

The first is exact invalidation through `purge_by_blob_id`. This removes the payload, metadata, associated tag records, and any matching hot-tier entry. It is the most direct cor-

rectness action and corresponds to object-level invalidation.

The second is grouped invalidation through purge_by_tag. This scans tag-prefix records and removes every associated object. This path is particularly useful for grouped maintenance and deterministic demonstrations, because it allows several related cached objects to be invalidated together.

The third is lifecycle-oriented invalidation through purge_epoch_leq. This scans epoch-tag records and removes all entries whose end epoch is at or below a supplied threshold. This is the mechanism that makes epoch-based lifecycle invalidation operational in the prototype.

These three paths are enough to demonstrate that invalidation is not a rhetorical feature of the dissertation, but an implemented part of the cache-control plane.

5.13 Event Ingestion Implementation

The event subsystem is implemented around a common EventSource trait:

Listing 5.6: Common event-stream abstraction used by simulated and live indexers.

```
pub trait EventSource: Send {
    fn stream(&mut self) -> Pin<Box<dyn Stream<Item = Result<WalrusEvent>> +
   Send + '_>>;
}
```

This is an important implementation choice because it keeps the maintenance logic agnostic to the source of lifecycle observations. The same downstream logic can therefore be used with either deterministic simulated input or live Sui-backed input.

The simulated source produces a short fixed event trace containing examples such as blob registration, certification, extension, epoch progression, and expiry. This is especially valuable for the purposes of this project because it makes the invalidation path demonstrable even when live chain activity is sparse or inconvenient to reproduce.

The live Sui-backed source is available behind the sui-live feature. When enabled, it connects to a Sui RPC endpoint, queries events using several module-filter strategies, decodes Walrus-related event types, and enriches them with transaction, sequence, and timestamp information. This part of the implementation is necessarily more operationally complex, but it shows that the same cache-maintenance logic can also operate in a more realistic environment.

A further useful detail is the generation of higher-level derived events. When the indexer sees a later certification with a larger end epoch, it emits an Extended event. When observed epoch progress overtakes stored end epochs, it triggers expiry-oriented purge behaviour. This makes the lifecycle model easier to reason about at cache level than a raw stream of low-level events would be.

5.14 Cache Side-Effects from Events

The indexer is not merely a passive consumer of lifecycle events. It also performs cache side-effects through a shared cache handle. In broad terms, deletion-like events such as `BlobDeleted`, `InvalidBlobId`, and derived `Expired` events trigger purge-by-ID behaviour. Certification-related events update metadata with end-epoch and deletability information, while `Extended` updates the stored retention horizon.

This is one of the most important implementation choices in the project because it operationalises the dissertation's central correctness claim: cache maintenance is informed by lifecycle state rather than only by request frequency or timer expiry.

Figure 5.1 summarises this mapping.

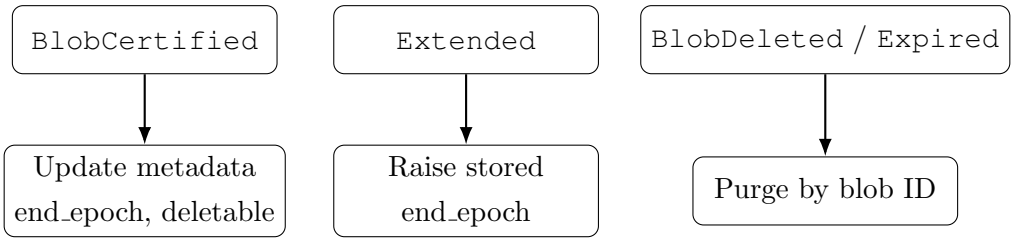

Figure 5.1: Main cache side-effects triggered by lifecycle events. Certification updates metadata, extension updates stored retention state, and deletion- or expiry-like events trigger purging.

5.15 Metrics Implementation

Metrics are defined in `metrics.rs` and exposed through a small Axum HTTP server. The implementation includes counters for RAM hits, disk hits, origin misses, purges, cache writes, bytes served, and bytes written. It also exposes a histogram vector labelled by serving source:

Listing 5.7: Latency histogram labelled by serving source.

```
pub static REQUEST_LATENCY_MS: Lazy<HistogramVec> = Lazy::new(|| {
    let opts = HistogramOpts::new("request_latency_ms", "Get latency by source (
   ms)")
        .buckets(vec![1.0, 2.0, 5.0, 10.0, 20.0, 50.0, 100.0, 200.0,
                      500.0, 1000.0, 2000.0, 5000.0, 10000.0]);
    let h = HistogramVec::new(opts, &["source"]).unwrap();
    REGISTRY.register(Box::new(h.clone())).ok();
    h
});
```

This implementation detail matters because it lets the dissertation distinguish RAM, persistent-store, and origin latency directly rather than inferring serving source indirectly. It is therefore central not only to evaluation, but also to the credibility of the architectural claims made earlier.

5.16 Demonstration Flow

One of the most useful implementation features for this project is the guided `demo-flow`. This command: uploads a blob, performs a first get that should miss and store, performs a second get that should hit, purges by blob ID, gets again to demonstrate miss-after-purge, purges by tag, and finally sets epoch metadata and purges by epoch threshold.

The value of this command is that it keeps the entire miss/hit/invalidate sequence inside one running process, so the metrics and logs remain consistent and easy to observe.

5.17 Sharding Helper

The implementation includes a small rendezvous-style node-selection helper:

Listing 5.8: Rendezvous-style node selection for future multi-node extension.

```
pub fn pick_node<'a>(key: &str, nodes: &'a [String]) -> Option<&'a str> {
    if nodes.is_empty() { return None; }
    let mut best = None;
    let mut best_score = u64::MIN;
    for n in nodes {
        let mut h = AHasher::default();
        key.hash(&mut h);
        n.hash(&mut h);
        let score = h.finish();
        if score > best_score {
            best_score = score;
            best = Some(n.as_str());
        }
    }
    best
}
```

At present this is only lightly integrated, but it is still worth highlighting because it shows that the implementation already anticipates future distributed placement.

5.18 Evaluation Harness Implementation

The dissertation's quantitative evaluation is supported not only by the live prototype but also by the synthetic framework implemented in `eval.rs`. This module generates reproducible workloads, simulates policy variants, records latency distributions, computes summary statistics, and exports CSV, JSON, trace, and plotting artefacts.

This part of the implementation is especially important because it separates two kinds of evidence. The first is **live-system evidence**, which shows that the actual prototype works end to end. The second is **controlled synthetic evidence**, which makes it possible to study

policy behaviour under a wider range of workloads than would be convenient to reproduce repeatedly against live infrastructure.

The evaluation harness also contains automated checks that strengthen the implementation chapter because they show that the evaluator itself encodes meaningful behavioural expectations. These include tests for warm-up improvement, WTinyLFU-style scan resistance, AdaptSize-lite competitiveness, baseline-versus-cache gains, and disk survival or recovery round-trips. This means that `eval.rs` is not merely a plotting script; it is an important part of the project's verification and evidence-producing software.

5.19 Implementation Limitations

The implementation is intentionally practical rather than exhaustive. Several limitations remain, but they are mostly limitations of deployment scope rather than conceptual weakness.

First, the cache uses process-local locking rather than a distributed control plane. Second, the in-memory tier is a bounded hot window rather than a full segmented cache with strong theoretical policy guarantees. Third, the WTinyLFU-inspired path is intentionally lightweight: it uses a Count-Min sketch and frequency-aware comparison, but it is not a full industrial W-TinyLFU implementation. Fourth, live event integration depends on timely and decodable Sui event visibility and is available only when the `sui-live` feature is enabled. Fifth, sharding support is only an extension point, not a complete multi-node deployment mechanism.

These limitations are acceptable for the dissertation because the goal is to build a convincing lifecycle-aware edge prototype rather than a production CDN. What matters is that the implemented system is coherent, measurable, and faithful to the main design claims.

5.20 Implementation Summary

The implementation directly realises the project's core system idea. It combines a Walrus-aware retrieval client, a hybrid cache, structured metadata, multiple invalidation paths, event-driven lifecycle maintenance, metrics exposure, reproducible demos, and a synthetic evaluation harness. The result is not merely a performance tweak but a small, coherent edge system whose behaviour can be measured, explained, and evaluated.

Chapter 6

Evaluation

6.1 Evaluation Goals

The evaluation has three main aims:

1. determine whether the cache/CDN-like layer improves performance relative to a no-cache baseline;
2. determine whether admission policy choice materially affects behaviour under different workload patterns;
3. determine whether lifecycle-aware cache maintenance behaves in a way that is operationally sensible for a Walrus-facing edge layer.

These goals follow directly from the design in Chapter 4 and the implementation in Chapter 5. The evaluation is therefore not only asking whether caching helps in the abstract, but whether **this particular design** – a hybrid RAM-plus-RocksDB cache with policy-aware admission and event-informed invalidation – behaves as intended.

6.2 Methodology

The evaluation combines two complementary forms of evidence.

First, the implemented prototype is used to demonstrate concrete end-to-end behaviours such as miss → store → hit transitions, purge-by-ID and purge-by-tag invalidation, and epoch-threshold purging. These behaviours validate that the control-plane mechanisms described earlier are present in the real system.

Second, the reproducible synthetic evaluation harness implemented in `eval.rs` generates controlled workloads and computes summary metrics including hit-rate, byte-hit-rate, average latency, p95 latency, and throughput for different policy and workload configurations. Note that, when this chapter refers to concurrency, it refers to the number of requests issued in parallel within the evaluation harness. The harness also exports the figures used in this chapter.

The figures in this section are not measurements from a globally deployed multi-node CDN. They are controlled evaluations of the systems prototype and its policy logic under a stylised but reproducible workload model. Where relevant, the synthetic evaluator uses simplified policy models to isolate behavioural differences more clearly; these results should therefore be interpreted as controlled evidence about design behaviour rather than as exact reproductions of every runtime code path. This is appropriate for a prototype, but it defines the scope of the conclusions.

6.3 Scenario Set

The evaluation is organised around six scenario families. The first is a baseline-versus-CDN A/B comparison, which contrasts a no-cache baseline with a cache-enabled WTinyLFU configuration. The second is admission-policy comparison, where WTinyLFU, LRU, and a no-admission variant are evaluated against one another. The third is a scan-resistance workload of the form hotset-scan-hotset, intended to stress recency-only policies with a burst of low-reuse accesses. The fourth examines size-aware admission by comparing the simplified AdaptSize-style gate in its enabled and disabled forms. The fifth studies cold-start warm-up by tracing hit-rate over successive windows as the cache fills from empty state. The sixth examines concurrency scaling, where **concurrency** refers to the number of retrieval requests issued in parallel by the synthetic evaluation harness. The purpose of this scenario is to test whether the cache-enabled design remains effective under increasing parallel request-level demand, rather than to claim a full production study of internal cache-thread scalability or multi-node deployment behaviour.

Together, these scenario families cover the main architectural claims of the dissertation: that the cache improves repeated retrieval performance, that policy choice matters, that frequency-aware admission is more robust under scan pressure, and that the cache becomes useful quickly rather than only after a long burn-in period.

6.4 How to Read Curated Labels

Some figures use short readable x-axis labels rather than full scenario names in order to keep the plots legible. Table 6.1 defines them.

Short Label	Meaning
`ab:base`	A/B baseline: no cache
`ab:cdn`	A/B cache-enabled CDN variant (WTinyLFU)
`pol:w`	Admission policy: WTinyLFU
`pol:l`	Admission policy: LRU
`pol:none`	Admission policy: no admission filter
`scan:w`	Scan-resistance workload with WTinyLFU
`scan:l`	Scan-resistance workload with LRU
`size:on`	Size-aware admission gate enabled
`size:off`	Size-aware admission gate disabled
`conc:1`	Concurrency-scaling run at concurrency 1
`conc:64`	Concurrency-scaling run at concurrency 64

Table 6.1: Legend for curated short labels used in readability-optimised charts.

6.5 Headline Results

The most important overall result is that the cache-enabled path substantially outperforms the no-cache baseline in the synthetic A/B comparison. In this run, hit-rate increases from 0.0000 to 0.8170, average latency decreases from 85.019 ms to 20.461 ms, and throughput increases from 11.762 req/s to 48.873 req/s.

This corresponds to an approximately 4.16x improvement in throughput and an approximately 4.16x improvement in average-latency speedup when measured relative to the no-cache baseline. The result is consistent with the architectural intent of the system: repeated accesses should increasingly be served from cache rather than from origin, which lowers latency and correspondingly raises effective request throughput.

6.6 Overall Cache Effectiveness

Figure 6.1 provides a concise overview of hit-rate across a readability-optimised subset of representative variants.

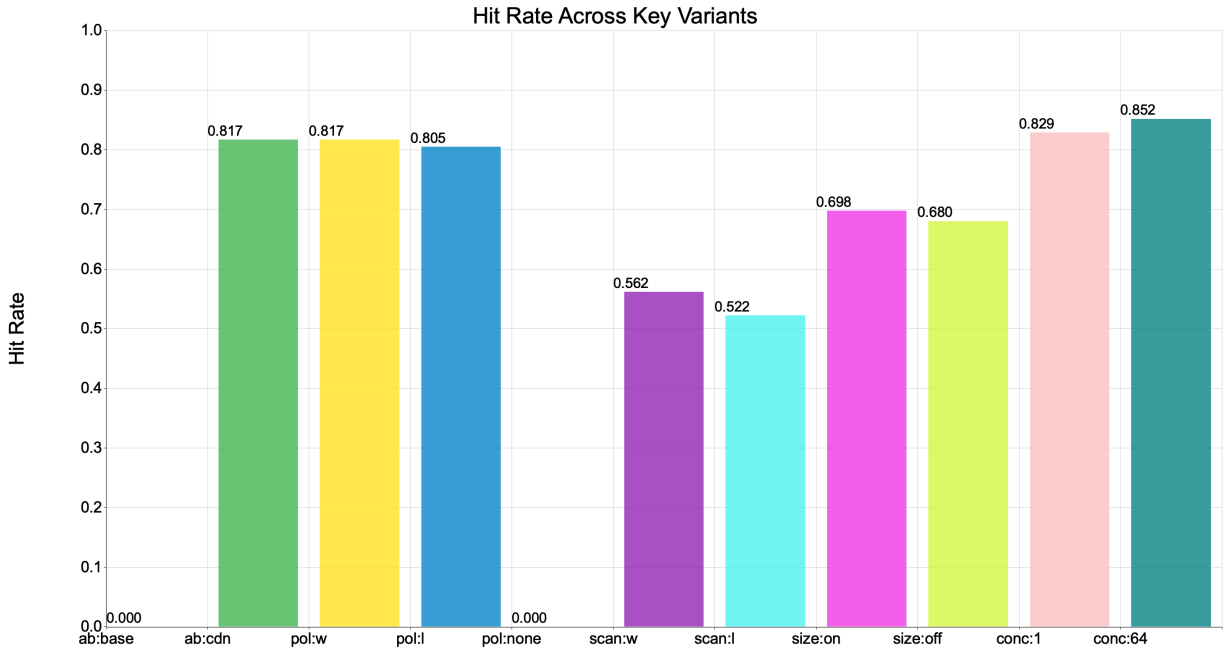

Figure 6.1: Overview hit-rate for a readability-optimised key set. Labels are defined in Table 6.1.

This figure supports four useful observations. First, the no-cache baseline has zero hit-rate, as expected. Second, the cache-enabled A/B variant reaches approximately 0.817 hit-rate, showing that the prototype is actually reusing previously fetched objects rather than merely inserting them without later benefit. Third, the WTinyLFU policy slightly outperforms LRU in the main policy comparison, while the no-admission mode provides no useful reuse in the

synthetic evaluator. Fourth, the concurrency points remain high, which is consistent with the workload still containing substantial reuse under parallel access.

Figure 6.2 complements this overview by focusing on p95 latency rather than hit-rate.

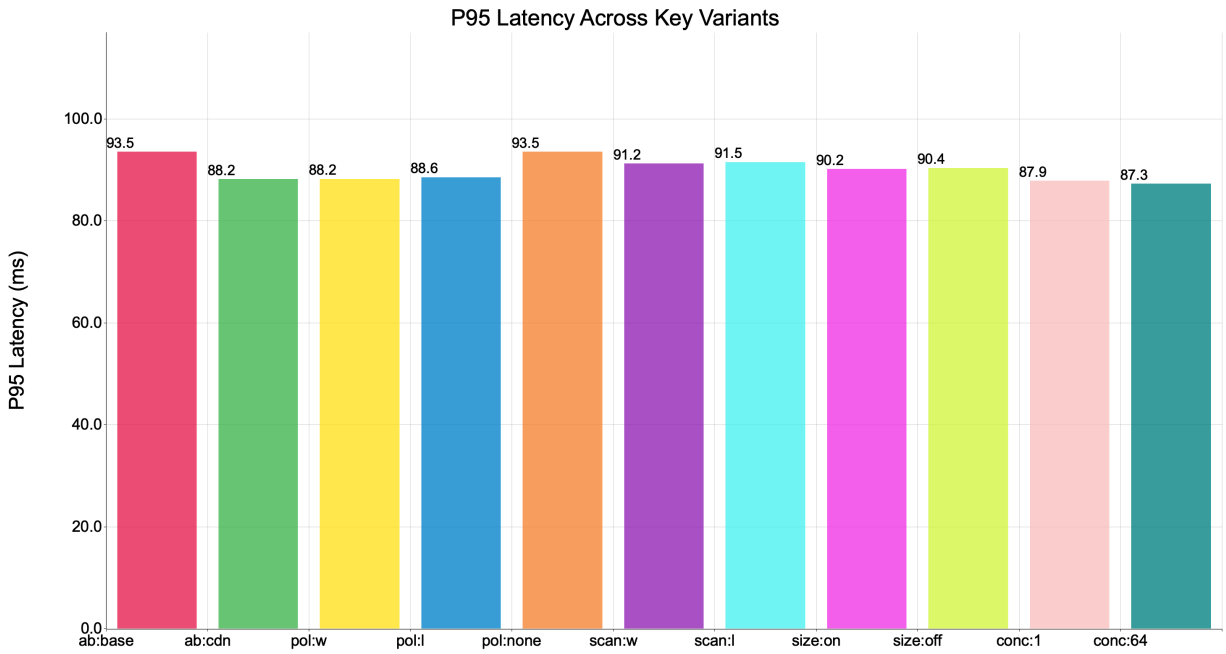

Figure 6.2: Overview p95 latency for the same readable key set. Lower values indicate better tail responsiveness.

The p95 figure shows that tail-latency differences are present but smaller than the average-latency improvements. In the A/B comparison, p95 latency falls from 93.547 ms to 88.178 ms. This modest movement is sensible because the workload model still includes origin misses, and those misses continue to influence the upper tail. In other words, caching substantially improves the bulk of requests, but p95 remains shaped by the remaining misses.

6.7 Baseline-Relative A/B Speedups

The most direct performance comparison is the A/B baseline-versus-CDN evaluation. Figure 6.3 shows throughput speedup relative to the no-cache baseline, and Figure 6.4 shows latency speedup relative to the same baseline.

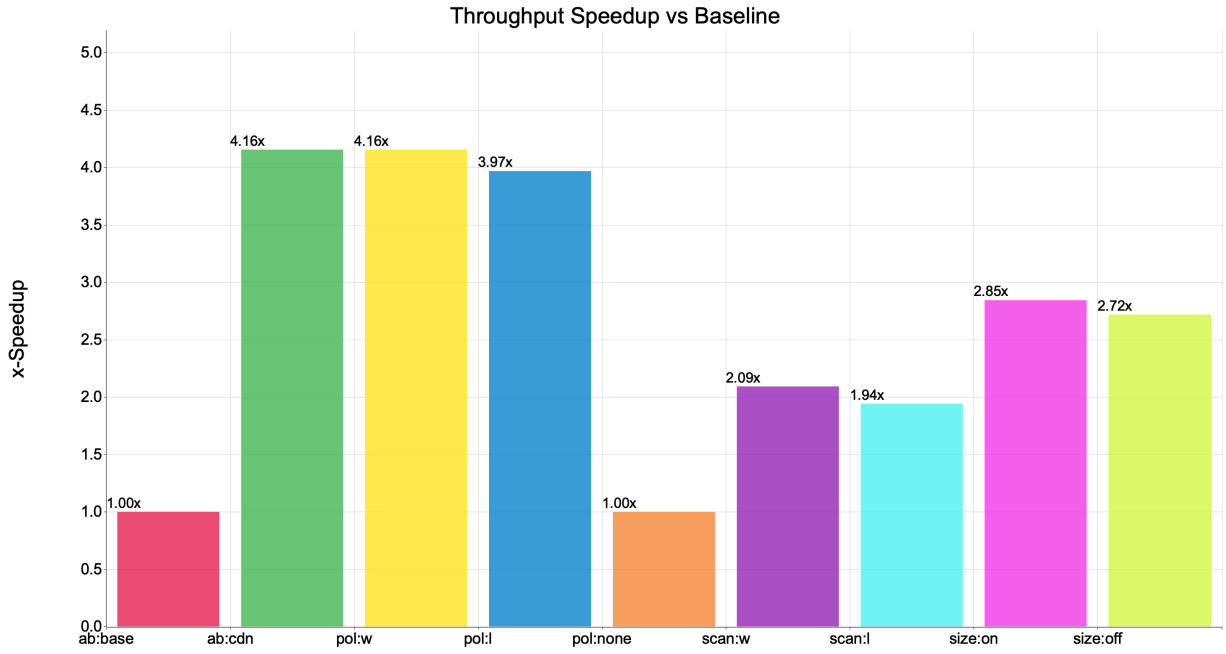

Figure 6.3: Throughput speedup relative to baseline. Curated x-axis labels are fully defined in Table 6.1.

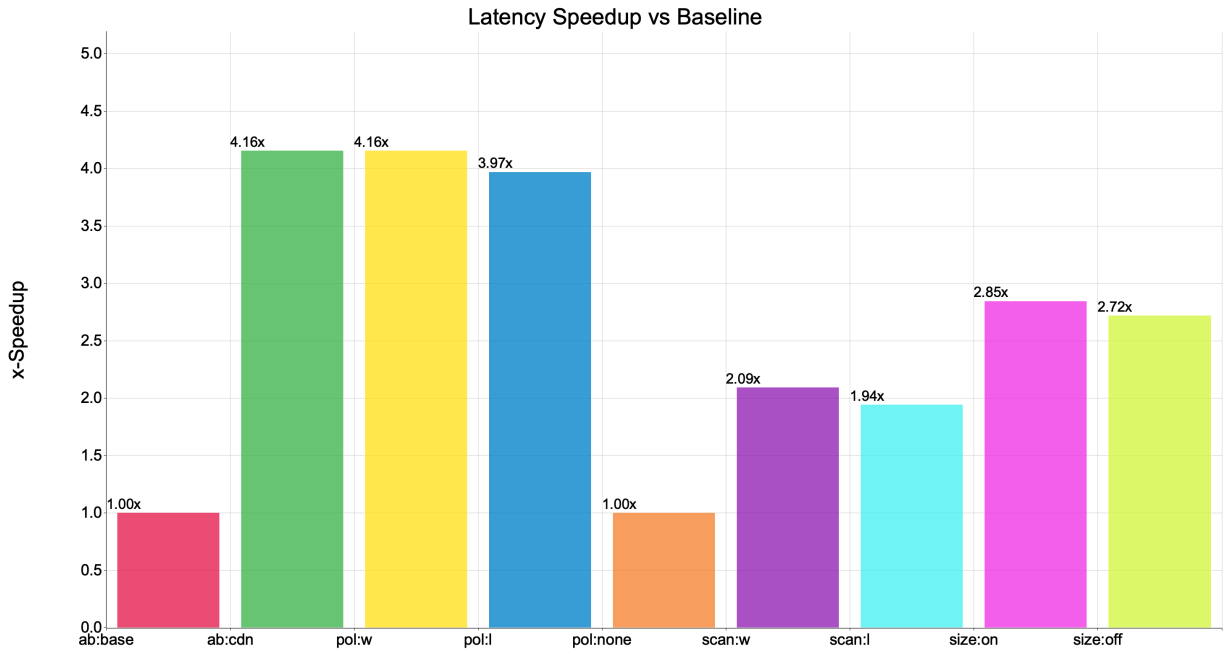

Figure 6.4: Latency speedup relative to baseline (baseline latency divided by variant latency). Higher is better. Curated labels are defined in Table 6.1.

These figures show that the cache-enabled CDN path and the WTinyLFU policy deliver the strongest improvements among the main A/B and policy-comparison variants. LRU also

improves strongly over the no-cache baseline, while the scan-resistance and size-aware variants occupy intermediate positions because they evaluate more specialised workload conditions.

In the synthetic harness, throughput is computed from total simulated request time, so throughput and average latency are mathematically linked. The two speedup figures are therefore not independent measurements in the same way they would be in a live deployment with external bottlenecks. Instead, they are two readable views of the same central outcome: reducing average request cost leads directly to higher effective request throughput.

6.8 Admission Policy Comparison

Figure 6.5 compares hit-rate across the three admission modes: WTinyLFU, LRU, and no-admission mode.

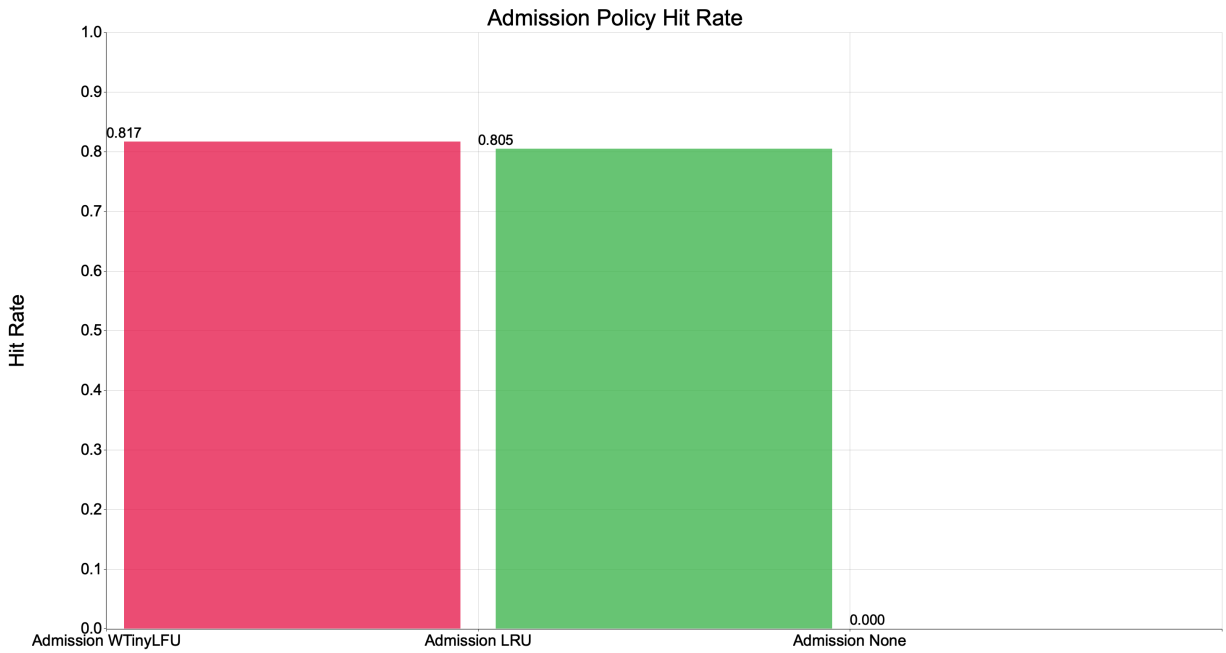

Figure 6.5: Hit-rate comparison across admission policies. WTinyLFU slightly outperforms LRU, while the deliberately weak synthetic no-admission baseline provides no useful reuse.

This is one of the most important results in the dissertation because it justifies the inclusion of policy-aware admission in the architecture. WTinyLFU reaches a hit-rate of 0.8170, LRU reaches 0.8049, and the deliberately weak synthetic no-admission baseline remains at 0.0000. The gap between WTinyLFU and LRU is not enormous in this particular workload, but it is consistent and positive.

This result should be interpreted carefully. In the runtime cache implementation described in Chapter 5, `AdmissionMode::None` is a minimal admission baseline subject to hard limits, whereas in the synthetic evaluator the `None` policy is intentionally modelled as a much weaker comparison point. The purpose of the synthetic `None` variant is therefore not to reproduce the

runtime cache path exactly, but to show how much policy-aware admission matters relative to an intentionally unhelpful baseline.

6.9 Scan Resistance

Figure 6.6 evaluates the hotset-scan-hotset workload, which is intended to stress recency-only policies with a large one-off scan.

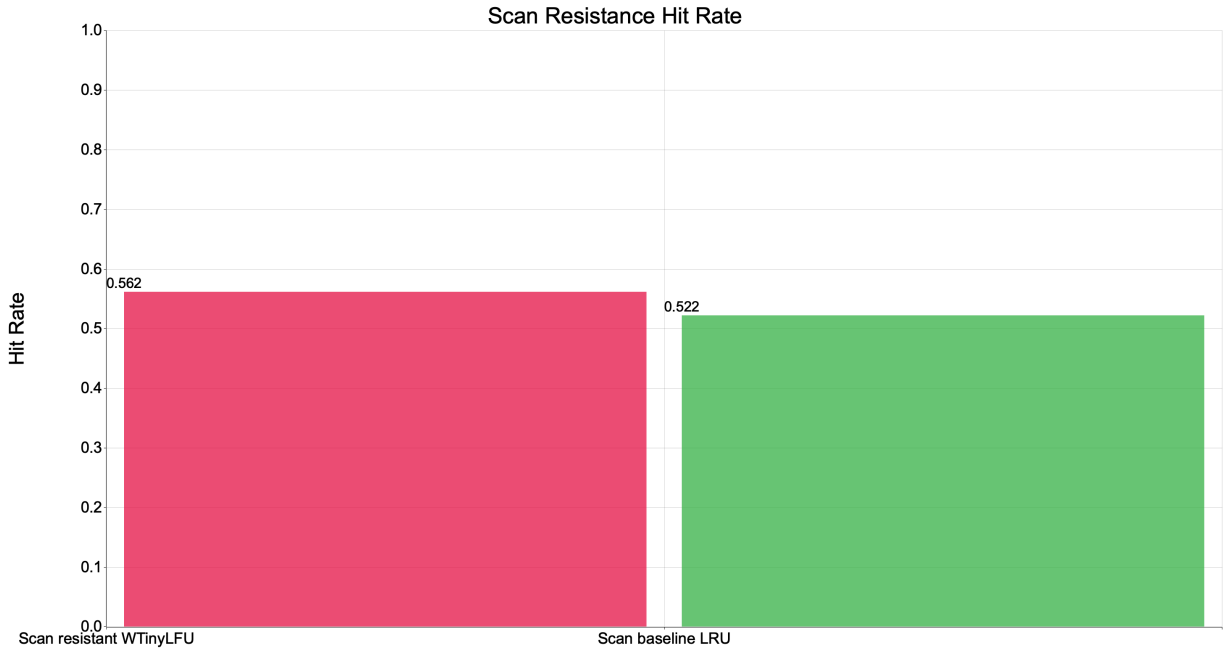

Figure 6.6: Scan-resistance scenario (hotset-scan-hotset). WTinyLFU retains more useful objects than LRU under scan pressure.

The WTinyLFU variant achieves a hit-rate of 0.5617, whereas LRU reaches 0.5222. This is the expected pattern. During the scan phase, LRU is more easily polluted by recently touched but low-future-value objects, whereas WTinyLFU's frequency-aware admission is better able to preserve objects that have demonstrated longer-term value. This figure therefore provides concrete evidence for one of the dissertation's core policy claims: frequency-aware admission is especially useful under workloads designed to expose the weaknesses of simpler recency-only approaches.

6.10 Size-Aware Admission Discussion

Although size-aware admission is not shown in its own stand-alone figure here, it appears in the overview and policy-frontier views through the `size:on` and `size:off` variants. In this run, enabling the size-aware gate improves hit-rate from 0.6804 to 0.6978, reduces average latency from 31.274 ms to 29.875 ms, and increases throughput from 31.976 req/s to 33.473

req/s. Byte-hit-rate decreases slightly from 0.4407 to 0.4338, which means the gain is mainly in object-level hit-rate and latency rather than in byte-level offload. This suggests that the size-aware gate improves object-level reuse and latency by favouring smaller reusable objects, even if that comes at a slight cost in the proportion of total bytes served from cache.

This behaviour is plausible in the context of the prototype. Large objects are more expensive to admit because they consume a disproportionate amount of cache capacity. A lightweight size-aware gate can therefore improve the effective use of cache space by biasing against oversized insertions that would evict many smaller, more reusable objects. The result is not dramatic, but it is consistent with the intended purpose of the feature and with the literature discussed in Chapter 2.

6.11 Cold-Start Warm-up Behaviour

Figure 6.7 shows the cache warm-up trajectory over 80 windows.

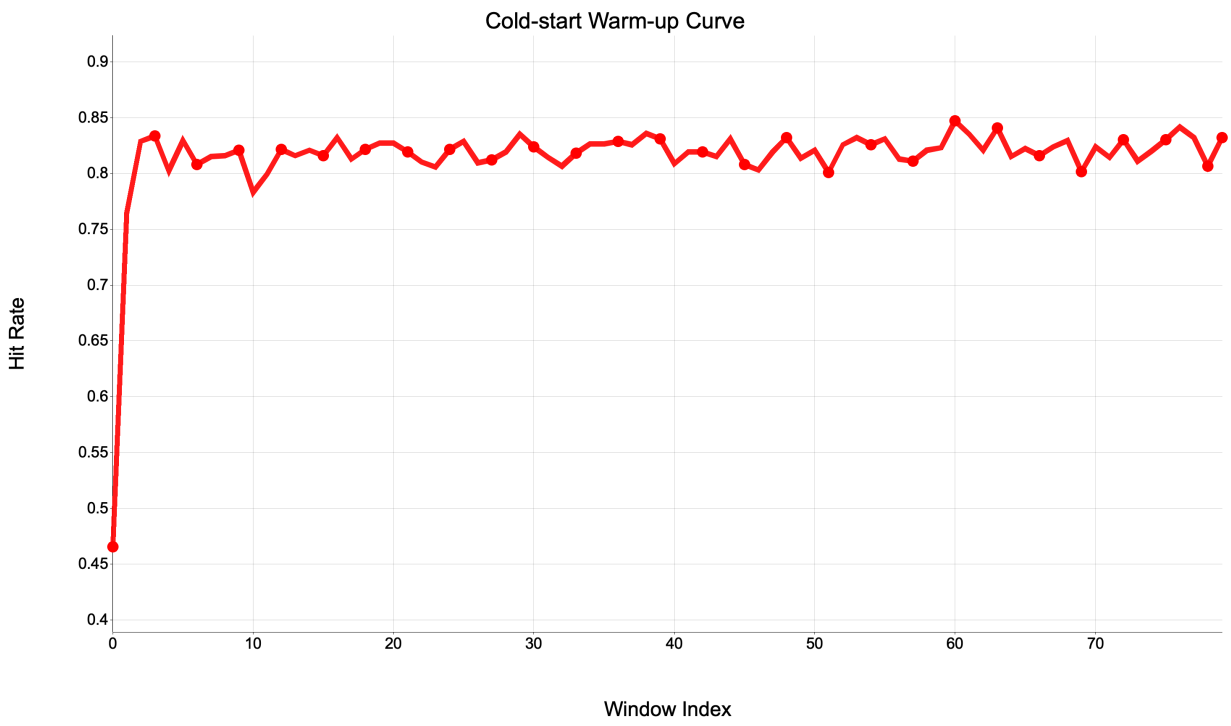

Figure 6.7: Cold-start warm-up trajectory over 80 windows. Hit-rate rises from 0.4656 in the first window and stabilises around a high-hit regime, ending at 0.8320 in this run.

This figure answers a practical question: how quickly does the cache become worthwhile after starting empty? The plot shows a steep initial rise, moving from 0.4656 in the first window to above 0.82 by the third window, after which it oscillates around a high-hit regime. These oscillations reflect normal variation in the stochastic workload rather than instability in the design. In practical terms, the result means the prototype does not require a long burn-in period before delivering useful reuse.

6.12 Concurrency Scaling

Figure 6.8 shows throughput as concurrency increases.

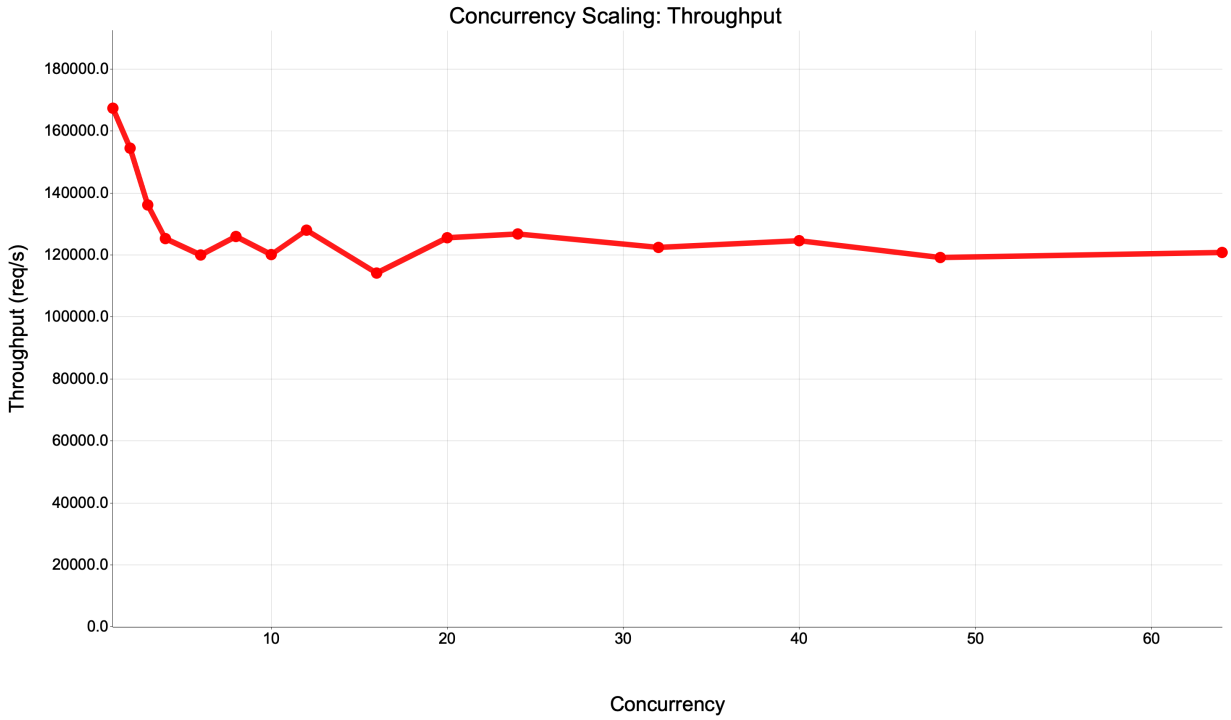

Figure 6.8: Throughput under increasing concurrency. Here, concurrency denotes the number of retrieval requests issued in parallel by the synthetic evaluation harness. The initial drop is followed by a broad high-throughput region, indicating that throughput remains substantial rather than collapsing as the level of parallel request demand increases.

In this scenario, concurrency means parallel request-level demand generated by the evaluation harness. More concretely, the harness issues multiple retrieval operations at the same time against the same cache-enabled design and records the resulting aggregate throughput. It should therefore be interpreted as a test of how the prototype behaves when many requests are active simultaneously, rather than as an isolated measurement of the cache module's internal locking behaviour or as a direct model of a geographically distributed CDN.

The most obvious feature of this plot is that throughput is highest at concurrency 1 and then settles into a broad high-throughput region as concurrency increases. In the current run, concurrency 1 reaches approximately 174k requests per second, while higher-concurrency points remain roughly in the 112k–128k requests-per-second range. This shape is sensible in the context of the synthetic harness. The harness models concurrent work using local threaded execution rather than a full real-world networked system, so the curve should not be interpreted as a literal production scaling law.

The useful conclusion is more modest: cached serving remains effective under parallel request demand in this synthetic framework and does not collapse as the number of simultaneously active requests rises. This supports the claim that the cache benefit is not restricted to purely serial access, while avoiding an over-strong claim about production CDN scalability.

6.13 Latency Distribution

Figure 6.9 shows empirical latency CDFs for the key behaviours. Closely overlapping variants are combined in the legend so that the distinct latency patterns remain visible.

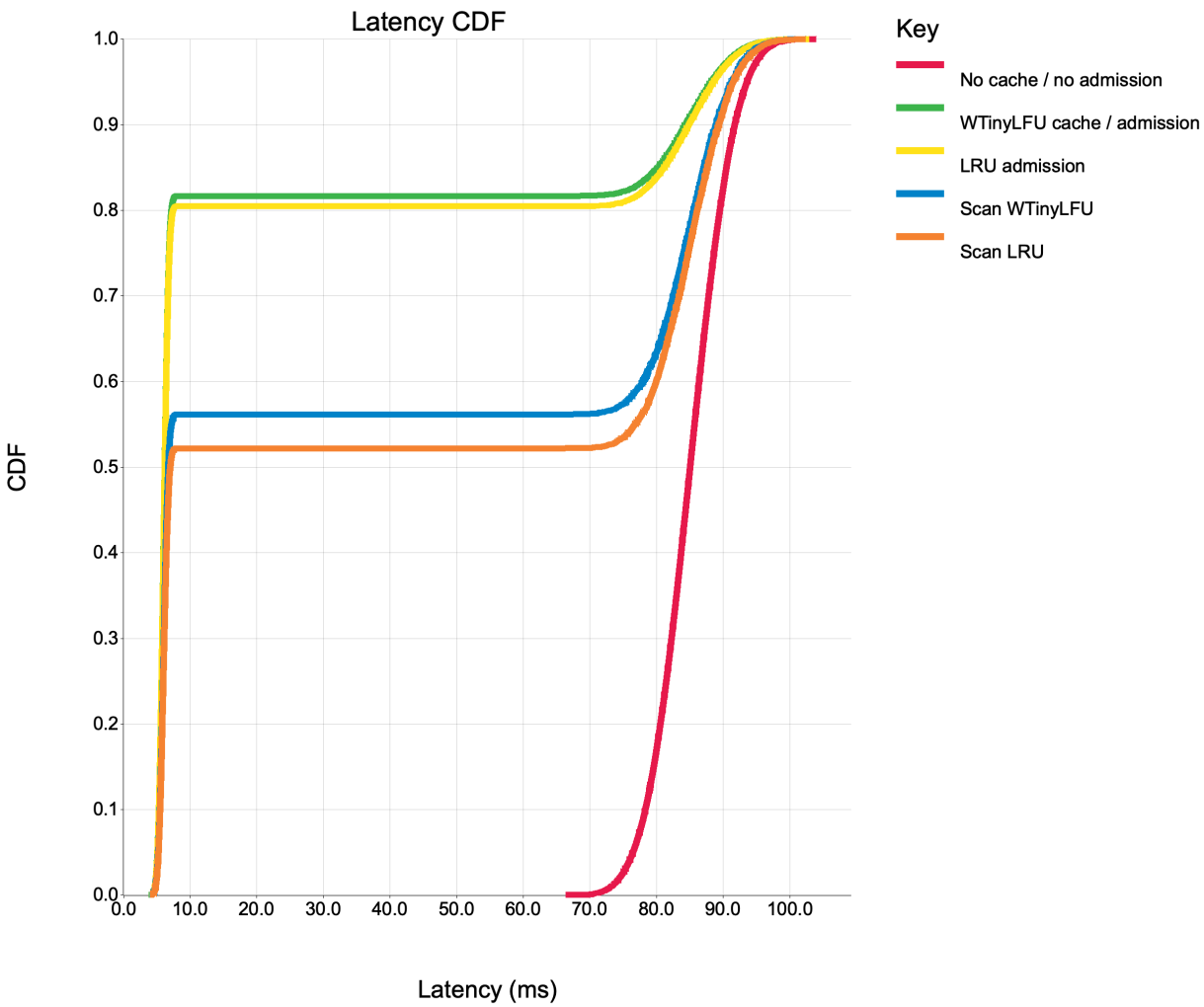

Figure 6.9: Empirical latency CDFs for distinct key behaviours. Curves that lie further left indicate lower latency for a larger fraction of requests.

This is one of the most informative figures in the chapter because it shows more than a single summary statistic. The cache-enabled and WTinyLFU-style curves shift markedly left relative to the no-cache baseline, meaning that a much larger proportion of requests complete at low latency. The baseline curve remains concentrated in the high-latency region because every request effectively incurs origin cost. By contrast, the cache-enabled variants show a substantial mass of low-latency requests corresponding to cache hits.

The scan-resistance curves sit between these extremes. In the scan workload, the cache is still useful, but the workload intentionally disrupts locality and therefore reduces the proportion of low-latency hits. The CDF therefore gives a more nuanced picture than average latency alone: the cache changes the overall distribution of user-visible response times, not only one aggregate number.

6.14 Policy Trade-off Frontiers

Figures 6.10 and 6.11 summarise the main policy trade-offs using latency-versus-hit-rate scatter plots.

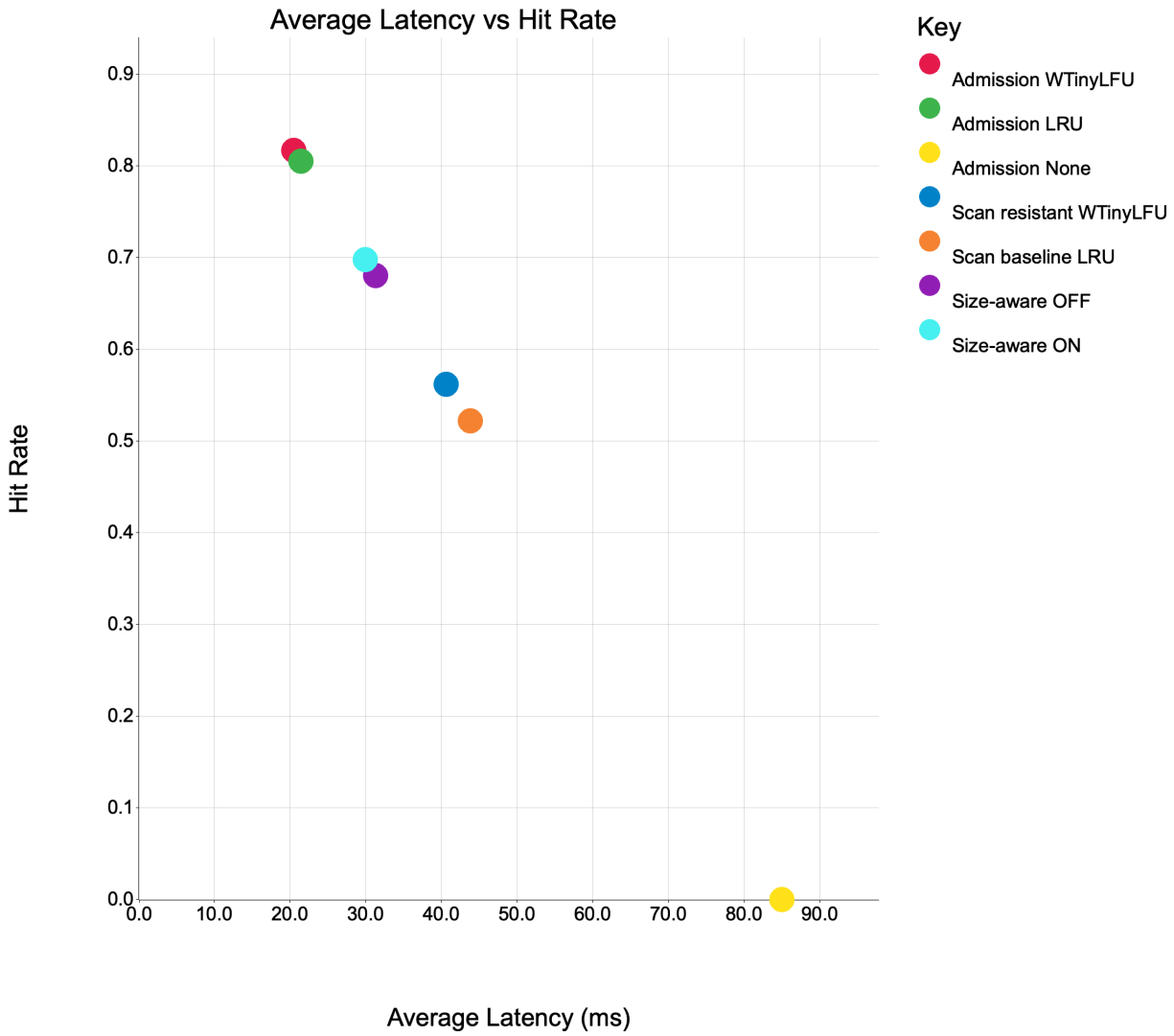

Figure 6.10: Average-latency versus hit-rate frontier with a full-text legend key. Points nearer the upper-left are preferable because they combine lower latency with higher hit-rate.

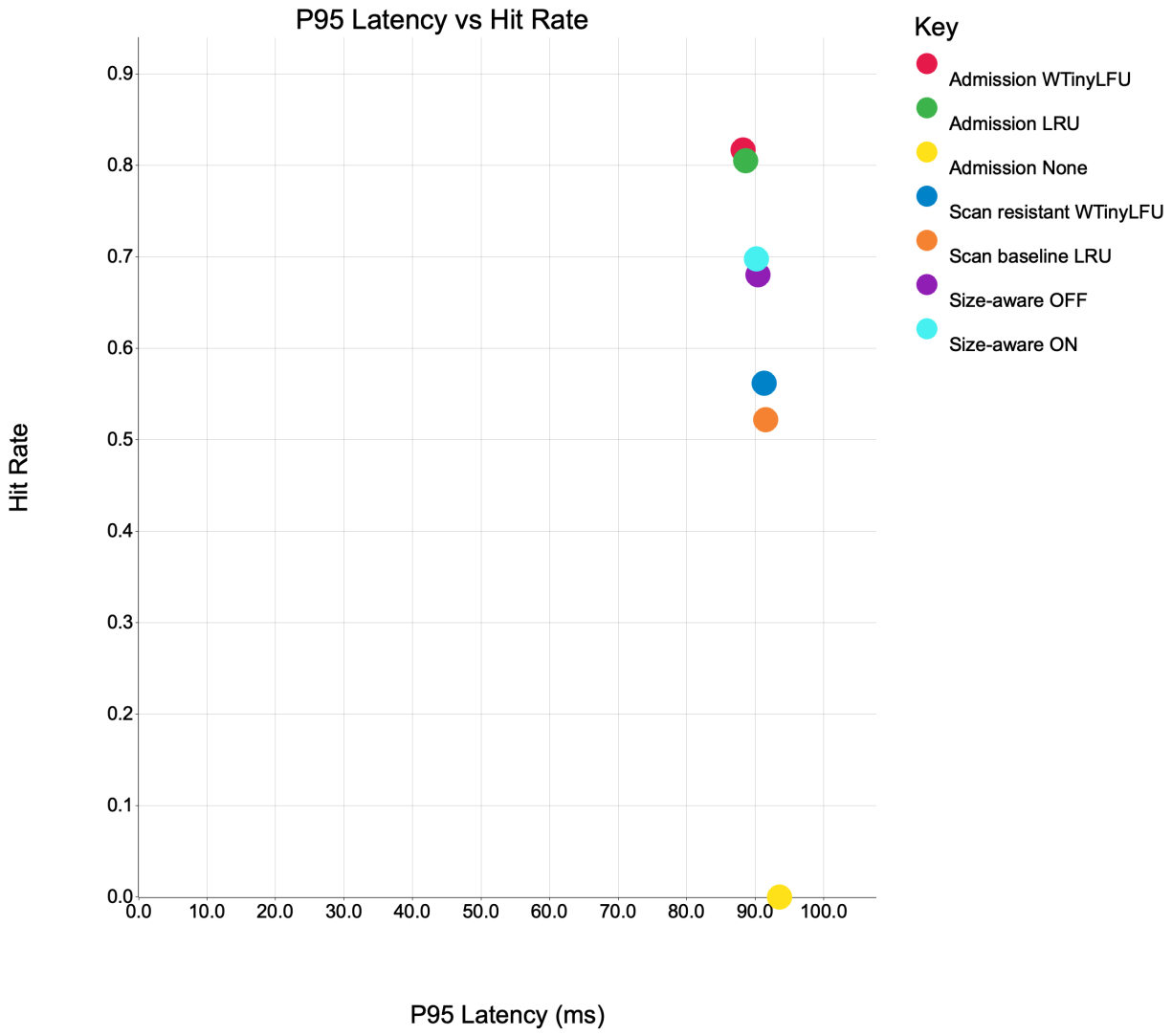

Figure 6.11: P95-latency versus hit-rate frontier with a full-text legend key. This complements Figure 6.10 by showing the same trade-off using tail latency rather than average latency.

The average-latency frontier makes the main comparisons visually explicit. WTinyLFU combines high hit-rate with low average latency in the default policy comparison. LRU is close, but slightly worse on both hit-rate and average latency in this run. The no-admission mode sits far to the right with zero hit-rate, making it the least attractive operating point. The scan and size-aware variants occupy intermediate regions because they test specific workload sensitivities rather than serving as the overall best operating point.

The p95 frontier should be interpreted with more caution. Because p95 latency remains strongly influenced by the misses that still go to origin, the points are more tightly clustered than in the average-latency frontier. This does not make the figure unhelpful; rather, it shows that the main policy separation appears more clearly in average behaviour and hit-rate than in upper-tail latency alone.

6.15 Correctness and Invalidation

The performance figures above evaluate the data plane. Correctness, however, also depends on the control plane. In this project, invalidation scenarios are interpreted as correctness checks rather than as hit-rate maximisation tests.

The required behaviour is straightforward: once an object is invalidated – whether by explicit purge, derived expiry, or lifecycle event processing – it should no longer be served as a stale cache hit. The next relevant request should instead miss and re-fetch from origin, or observe updated metadata-consistent state. This aligns directly with the architecture and implementation chapters, where cache correctness depends on metadata updates and purge logic rather than on TTL alone.

In practical terms, the correctness evidence for the prototype comes from controlled invalidation demonstrations rather than from a single scalar metric. The most important observable behaviours are that a previously cached object is served as a hit before invalidation, that purge-by-ID and purge-by-tag remove that local serving path, and that epoch-threshold maintenance prevents continued reuse once stored lifecycle conditions are no longer satisfied. These behaviours do not maximise hit-rate, but they are essential to the claim that the cache remains lifecycle-aware rather than merely aggressive.

Although the graphs in this chapter focus mainly on performance, they should be read together with the demo-flow and indexer behaviour described in Chapter 5. The dissertation's claim is therefore not that caching is always maximised, but that useful reuse can be achieved while still preserving an explicit pathway for lifecycle-aware invalidation.

6.16 Why the Results Make Sense for This Project

Taken together, the figures make sense in the specific context of Walrus-Cache-CDN.

The hit-rate improvements make sense because the workload contains substantial repeated access and the architecture is explicitly designed to exploit that reuse through a hybrid cache. The average-latency improvements make sense because cache hits replace repeated origin fetches with much cheaper local service. The advantage of WTinyLFU over LRU makes sense because approximate frequency-aware admission protects the cache against pollution more effectively than recency-only logic. The warm-up curve makes sense because the cache starts empty and quickly fills with the most reusable objects. The CDF makes sense because caching changes the full latency distribution. The relatively modest p95 movement also makes sense because the upper tail continues to include origin misses. Finally, the concurrency plot should be understood as a synthetic stress indicator rather than as a production deployment curve.

6.17 Threats to Validity

Several limitations affect how these results should be interpreted. The prototype is primarily single-node and does not model full multi-region CDN placement. Most of the quantitative figures come from the synthetic evaluation harness rather than from a live geographically distributed Walrus deployment. The workload family is deliberately broad but still not exhaustive, meaning that different access distributions or blob-size distributions could shift the relative performance of policies. Throughput in the synthetic harness is derived from simulated or local measured execution time depending on the scenario, and the concurrency experiments model parallel request issuance within the harness rather than a full production study of internal cache-thread scalability, queueing behaviour, or distributed edge deployment. Throughput and latency should therefore not be over-interpreted as independent production observables. In addition, live event timing may differ from simulated event timing, which means that correctness responsiveness in practice may vary with event visibility. More broadly, correctness in this dissertation is practical and event-informed rather than supported by a formal proof of strong distributed consistency.

These limitations do not invalidate the evaluation. They define its scope. The dissertation provides credible prototype-level evidence, not a claim to production-wide CDN benchmarking.

6.18 Evaluation Summary

The evaluation supports the dissertation's central claim. The cache/CDN-like layer improves reuse substantially relative to a no-cache baseline, lowers average latency, and correspondingly increases effective throughput. Admission policy choice matters: WTinyLFU slightly outperforms LRU in the main policy comparison and is more robust under scan pressure. The cache warms up quickly from cold start, the latency distribution shifts in the expected direction, and the system retains explicit lifecycle-aware invalidation behaviour through its metadata and event-processing mechanisms.

Overall, the results suggest that Walrus-Cache-CDN is both technically coherent and practically useful as a dissertation-scale prototype of a lifecycle-aware edge retrieval layer for Walrus.

Chapter 7

Conclusions

7.1 Achievements

This dissertation has presented and evaluated Walrus-Cache-CDN, a lifecycle-aware cache and CDN-like retrieval layer for Walrus objects. The project achieved its main objectives by implementing a working Rust prototype with a hybrid RAM-plus-RocksDB cache, policy-aware admission, event ingestion in both simulated and live modes, explicit invalidation controls, and Prometheus-compatible observability.

The evaluation showed that the prototype is not only functionally complete but also practically beneficial for the workload family studied. In the central baseline-versus-cache comparison, the cache-enabled path increased hit-rate from 0.0000 to approximately 0.8170, reduced average latency from about 85.019 ms to 20.461 ms, and increased effective throughput from 11.762 req/s to 48.873 req/s. These results support the core claim that a lightweight edge layer can materially improve repeated retrieval performance without modifying the underlying Walrus protocol.

7.2 Evaluation of the Work

A central reason the project is successful is that it does not treat caching as a purely local optimisation. Instead, it recognised that correctness in this setting depends on lifecycle-aware metadata and explicit invalidation. This led to a more realistic prototype in which a fast read path is combined with a control plane for metadata maintenance, tag-based purging, epoch-based purging, and event-driven lifecycle updates.

The evaluation also showed that policy choice matters. Across the evaluated workload family, WTinyLFU generally occupied the strongest operating point, combining the highest or near-highest hit-rate with the lowest or near-lowest average latency, while showing a clearer advantage under scan pressure. The warm-up curve further showed that the cache becomes useful quickly from cold start, rising rapidly to a stable high-hit regime rather than requiring a long burn-in period.

At the same time, the results should be interpreted within their intended scope. Tail-latency improvements were more modest than the average-latency gains, which is sensible because the p95 metric remains influenced by the remaining origin misses. Similarly, the concurrency-scaling results show robustness within the synthetic framework rather than proving linear scalability in a production deployment. Most of the quantitative graphs come from the reproducible synthetic harness in `eval.rs`, while the live prototype demonstrations primarily validate end-to-end behaviour and lifecycle-aware invalidation paths.

7.3 Limitations

The main limitations of the work are architectural scope rather than conceptual weakness. The system is still primarily single-node, and it does not implement a full distributed control plane or globally coherent multi-edge deployment. Correctness is practical and event-informed rather than a formal proof of strong distributed consistency. In addition, the synthetic evaluator uses a stylised workload model, so the precise numerical results should not be read as production benchmarks for a live Walrus deployment.

There is also an important distinction between the runtime implementation and the synthetic evaluator: some policy variants in the evaluator are intentionally simplified in order to create clearer comparison baselines. This does not undermine the usefulness of the evaluation, but it does mean that the synthetic results should be interpreted as controlled evidence about design behaviour rather than as exact reproductions of every runtime code path.

7.4 Future Work

Several natural extensions remain. These directions follow directly from the present prototype's limits. Multi-node deployment would address the currently single-node architecture. Stronger live-event integration would improve confidence in correctness under delayed or incomplete event visibility. Richer workload evaluation would test whether the observed policy advantages persist under more varied access and object-size distributions. More advanced policy tuning would build on the encouraging but still lightweight WTinyLFU-inspired and size-aware results reported in Chapter 6. Stronger persistence and replication guarantees would move the system closer to production-style edge operation.

These would deepen the CDN-like character of the system, but they do not undermine the central contribution already demonstrated here: a practical Walrus-facing edge cache can be built, instrumented, and evaluated, and it can provide meaningful performance gains while still preserving explicit pathways for lifecycle-aware correctness.

Bibliography

[1] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf, 2008.

[2] Ethereum.org. Technical Introduction to Ethereum. https://ethereum.org/developers/docs/intro-to-ethereum/, accessed March 2026.

[3] Ethereum.org. Introduction to Smart Contracts. https://ethereum.org/developers/docs/smart-contracts/, accessed March 2026.

[4] Mysten Labs. Sui Object Model. https://docs.sui.io/guides/developer/objects/object-model, accessed March 2026.

[5] Mysten Labs. Using Events. https://docs.sui.io/guides/developer/accessing-data/using-events, accessed March 2026.

[6] Mysten Labs. Epochs, Equivocation, and Reconfiguration. https://docs.sui.io/concepts/sui-architecture/epochs, accessed March 2026.

[7] J. Wang. A Survey of Web Caching Schemes for the Internet. *ACM SIGCOMM Computer Communication Review*, 29(5):36–46, 1999.

[8] Mysten Labs. *Walrus: An Efficient Decentralized Storage Network*. https://docs.wal.app/walrus.pdf, accessed March 2026.

[9] Walrus Foundation. Walrus Fundamentals. https://docs.wal.app/docs/system-overview/core-concepts, accessed March 2026.

[10] Walrus Foundation. Off-Chain Operations. https://docs.wal.app/docs/design/operations-off-chain, accessed March 2026.

[11] Protocol Labs. IPFS Documentation. https://docs.ipfs.tech/, accessed March 2026.

[12] Protocol Labs. Content Identifiers (CIDs). https://docs.ipfs.tech/concepts/content-addressing/, accessed March 2026.

[13] Protocol Labs. The Lifecycle of Data in IPFS. https://docs.ipfs.tech/concepts/lifecycle/, accessed March 2026.

[14] Protocol Labs. Filecoin Documentation. https://docs.filecoin.io/, accessed March 2026.

[15] Protocol Labs. Filecoin Storage Model. https://docs.filecoin.io/basics/what-is-filecoin/storage-model, accessed March 2026.

[16] Protocol Labs. Retrieve Data. https://docs.filecoin.io/builder-cookbook/data-storage/retrieve-data, accessed March 2026.

[17] Protocol Labs. Filecoin Proofs / Storage Proving. https://docs.filecoin.io/basics/the-blockchain/proofs, accessed March 2026.

[18] S. Williams et al. *The Permanent Information Storage Protocol.* Arweave Lightpaper, 2023. https://arweave.org/files/arweave-lightpaper.pdf

[19] Arweave. Arweave Documentation. https://docs.arweave.org/, accessed March 2026.

[20] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Caching.* RFC 9111, IETF, 2022.

[21] M. Kleppmann. *Designing Data-Intensive Applications.* O'Reilly Media, 2017.

[22] Redis. Key Eviction. https://redis.io/docs/latest/develop/reference/eviction/, accessed March 2026.

[23] N. Megiddo and D. Modha. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *FAST*, 2003.

[24] S. Jiang and X. Zhang. LIRS: An Efficient Low Inter-reference Recency Set Replacement Policy to Improve Buffer Cache Performance. *Proceedings of SIGMETRICS*, 2002.

[25] G. Einziger, R. Friedman, and B. Manes. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage*, 13(4), 2017.

[26] B. Manes. Caffeine Cache Design Notes. https://github.com/ben-manes/caffeine/wiki/Design, accessed March 2026.

[27] B. Manes. Caffeine Cache Efficiency Notes. https://github.com/ben-manes/caffeine/wiki/Efficiency, accessed March 2026.

[28] moka-rs authors. Moka Cache Documentation. https://docs.rs/moka/latest/moka/, accessed March 2026.

[29] Dgraph Labs. Ristretto Documentation. https://github.com/dgraph-io/ristretto, accessed March 2026.

[30] D. S. Berger, R. K. Sitaraman, and M. Harchol-Balter. AdaptSize: Orchestrating the Hot Object Memory Cache in a Content Delivery Network. In *Proceedings of NSDI*, 2017.

[31] Meta Platforms, Inc. RocksDB Column Families. https://github.com/facebook/rocksdb/wiki/column-families, accessed March 2026.

[32] Meta Platforms, Inc. RocksDB Overview. https://github.com/facebook/rocksdb/wiki/RocksDB-Overview, accessed March 2026.

[33] S. Dong, M. Callaghan, L. Galanis, D. Borthakur, A. Savor, and M. Strum. RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications. *ACM Transactions on Storage*, 17(4), 2021.

[34] Prometheus Authors. Prometheus: Monitoring System and Time Series Database. `https://prometheus.io/docs/introduction/overview/`, accessed February 2026.

[35] Prometheus Authors. Histograms and Summaries. `https://prometheus.io/docs/practices/histograms/`, accessed March 2026.

[36] M. Nottingham. *The Cache-Status HTTP Response Header Field.* RFC 9211, IETF, 2022.

Appendix A

System Manual

A.1 Purpose

This appendix provides the technical information needed for a later student to rebuild, run, inspect, and extend the project. It is written against the workspace snapshot submitted with this dissertation rather than against an external deployment. The canonical repository URL for the submitted workspace is as follows:

```
https://github.com/raghav2005/Walrus-Cache-CDN-FYP
```

The README file located in the root of the repository covers most of the same content as the appendices below.

A.2 Repository Layout

The repository root contains a single Rust crate, `walrus-cache-cdn`, which contains the prototype itself. The most important paths are summarised in Table A.1.

Table A.1: Main repository locations relevant to maintenance and extension.

Path	**Purpose**
`walrus-cache-cdn/src/main.rs`	CLI entry point and command dispatch
`walrus-cache-cdn/src/walrus.rs`	Walrus aggregator/publisher client and cached retrieval path
`walrus-cache-cdn/src/cache.rs`	RocksDB-backed cache, RAM hot tier, admission, metadata, and invalidation
`walrus-cache-cdn/src/indexer.rs`	Simulated and optional live Sui-backed event ingestion
`walrus-cache-cdn/src/metrics.rs`	Prometheus-style metrics registry and HTTP exporter
`walrus-cache-cdn/src/config.rs`	Environment-driven configuration
`walrus-cache-cdn/src/eval.rs`	Synthetic evaluation harness, plot generation, and unit tests
`walrus-cache-cdn/scripts/`	Demo launcher, demo panes, and evaluation helper scripts
`walrus-cache-cdn/DEMO_RUNBOOK.md`	Human-readable demo instructions
`walrus-cache-cdn/evaluation_output/`	Generated CSV, JSON, Markdown, and PNG evaluation artefacts

A.3 Build and Test Procedure

The implementation is a standard Cargo project. Build and test from the crate root:

Listing A.1: Core build and test commands.

```
cd walrus-cache-cdn
cargo build
cargo test
```

If live Sui-backed indexing is required, the optional feature can be enabled:

Listing A.2: Build and test with live Sui support enabled.

```
cd walrus-cache-cdn
cargo build --features sui-live
cargo test --features sui-live
```

The current workspace snapshot contains five unit tests in `src/eval.rs`. These cover warm-up improvement, scan resistance, size-aware admission competitiveness, baseline-versus-cache improvement, and disk survival/recovery behaviour. The repository root also includes `rust-toolchain.toml`, pinning the checked toolchain for reproducible builds.

A.4 Configuration and Environment

The executable reads configuration from environment variables. The project uses `dotenvy`, so a local `.env` file is loaded automatically when present. The repository already includes several presets: `.env.example`, `.env.localnet`, `.env.testnet-local`, and `.env.testnet-public`.

Table A.2: Configuration variables used by the prototype.

Variable	**Meaning**
`NETWORK`	selects `localnet` or `testnet` defaults
`WALRUS_AGGREGATOR_URL`	read-side Walrus endpoint
`WALRUS_PUBLISHER_URL`	upload-side Walrus endpoint
`INDEXER_MODE`	`simulated` or `sui` event source
`SUI_RPC_URL`	Sui RPC endpoint for live event mode
`WALRUS_PACKAGE_ID`	package filter used in live event mode
`WALRUS_EVENT_MODULE`	preferred module filter used in live event mode
`WALRUS_EVENT_MODULES`	ordered fallback live-event module candidates
`INDEXER_POLL_MS`	live-indexing polling interval in milliseconds
`INDEXER_START_AT_HEAD`	controls whether live indexing tails directly from the head
`CACHE_ENABLED`	enables or disables the cache path entirely
`CACHE_DB_PATH`	RocksDB directory used for persistent cache state
`CACHE_BLOCK_CACHE_MB`	RocksDB block-cache budget

Continued on next page

Variable	Meaning
CACHE_ADMISSION	wtinylfu, lru, or none
CACHE_ADAPTSIZE	enables probabilistic size-aware gating
METRICS_ADDR	bind address for the metrics exporter
CACHE_NODES	optional node list for rendezvous-style sharding experiments

For a fresh clone, the built-in runtime defaults are intentionally safe: they use simulated indexing together with public Walrus testnet endpoints so that the project can be built and exercised without first standing up local infrastructure. If an explicit local config file is preferred, the included `.env.example` can be copied to `.env`.

A.5 Extending the System

The implementation was deliberately split by responsibility. In practical terms, future extensions would normally begin in one of the following places:

- `main.rs` for new CLI commands or orchestration flows;
- `walrus.rs` for changes to the read/write interaction with Walrus endpoints;
- `cache.rs` for new admission, promotion, metadata, or invalidation behaviour;
- `indexer.rs` for new lifecycle event types or new live-source decoding logic;
- `eval.rs` for new synthetic workload families or additional validation checks.

Appendix B

User Manual

B.1 Overview

Walrus-Cache-CDN is operated through a command-line interface rather than through a graphical user interface. The main commands are listed in Table B.1.

Because this is a CLI-driven prototype rather than a GUI application, command examples and output-oriented workflows are more useful here than screen shots. The repository therefore includes both a deterministic demo flow and a four-pane tmux walkthrough for live demonstrations.

Table B.1: Main user-facing commands provided by the prototype.

Command	**Purpose**
`cargo run -- index --limit 5`	Print events from the configured event source. In the built-in default configuration this uses simulated mode, so it works immediately after cloning.
`cargo run -- get --id <blob_id>`	Retrieve a blob by Walrus blob ID through the cache-enabled path.
`cargo run -- get-by-object --object-id <object_id>`	Retrieve a blob by Sui object ID. This path is supported but is not the main cached path.
`cargo run -- put --data "hello"`	Upload inline data through the configured publisher endpoint.
`cargo run -- put --file ./file.bin --epochs 5`	Upload a file while requesting an epoch horizon.
`cargo run -- index --follow --demo-view`	Follow lifecycle events continuously in a compact demo-oriented view.
`cargo run -- cache-purge --id <blob_id>`	Purge one cached object by blob ID.
`cargo run -- cache-purge --tag module:walrus`	Purge every cached object carrying a specific tag.
`cargo run -- cache-purge --epoch-leq 55`	Purge objects whose stored end epoch is less than or equal to a threshold.
`cargo run -- cache-set-epoch --id <blob_id> --end-epoch 55`	Set deterministic epoch metadata for invalidation demonstrations.
`cargo run -- demo-flow --data "prof-demo"`	Run the single-process miss→hit→invalidate demo.

Continued on next page

Command	Purpose
`cargo run -- evaluate --out-dir ./evaluation_output --seed 42 --scale 1`	Run the synthetic dissertation evaluation suite and export artefacts.

B.2 Metrics and Observability

The prototype starts a metrics server automatically when the process starts. By default the metrics endpoint is:

`http://127.0.0.1:9001/metrics`

The most useful metrics during demonstrations are: `cache_origin_misses`, `cache_disk_hits`, `cache_ram_hits`, `cache_puts`, `cache_purges`, `cache_bytes_served`, `cache_bytes_written`, and `request_latency_ms{source=...}`.

B.3 Demo Workflow

For a supervised demonstration, the easiest entry point is the provided tmux launcher:

Listing B.1: Recommended multi-pane demo launcher.

```
cd walrus-cache-cdn
./scripts/demo_tmux.sh
```

This creates four panes for mode selection, continuous indexing, live metrics, and a deterministic cache walkthrough. The details are documented in `walrus-cache-cdn/DEMO_RUNBOOK.md`.

Appendix C

Project Plan and Interim Report

C.1 Project Plan

Executive Summary:
Walrus is a decentralised storage and data availability network built on top of the Sui blockchain, and it is designed to efficiently store large files off-chain while still having on-chain verification. Walrus aims to provide robust, censorship-resistant, and fault-tolerant data storage. In the architecture of Walrus, data is uploaded and split into slivers based on an erasure encoding scheme called Red Stuff, and these slivers are then distributed across various storage nodes. To retrieve a single file/blob, an aggregator (which is a specialised node on the network) collects the slivers from multiple storage nodes and reconstructs the file/blob.

Due to the distributed nature of the storage, Walrus can tend to have higher latency for reads – especially when direct interactions with these networks are considered. It can also be inefficient for applications that necessarily require low-latency data delivery, such as web services and distributed applications. That is where this project comes in: the project aims to build a CDN-like caching layer on top of Walrus in order to improve the performance of data delivery by indexing content, caching popular data, and geographically distributing cached copies. The goal is to combine the low-latency and high-speed access of traditional CDNs (which improve performance by serving frequently accessed content from locations closer to the client) with Walrus' decentralised trust and verifiability guarantees. The outcome should be a prototype Walrus Cache/CDN layer that demonstrates how such a decentralised storage system can be augmented to provide fast and reliable data access globally.

Objectives:
Build a working prototype: develop a functional prototype of a Cache/CDN-like layer integrated with the Walrus network. This includes designing and implementing components for content indexing, distributed caching nodes, and integration with Walrus' HTTP API for data retrieval. The prototype should be able to fetch content from the Walrus network and serve it to clients via cache nodes transparently. It should also implement basic content delivery features such as prefetching and request routing.

Implement Indexing, Caching, and Cache Invalidation: create an event indexing mechanism that monitors Walrus (via on-chain events) for new/updated blobs, enabling the system to keep track of available content. Implement caching strategies where regional nodes store copies of popular or recently accessed Walrus content. Develop a cache invalidation policy to handle content updates/expirations – for example, automatically refreshing or purging cached data when a blob's availability period ends or when updated content is detected, ensuring consistency with the Walrus source.

Performance Improvement: achieve measurable improvements in data delivery performance. The goal is to significantly reduce content retrieval latency and improve throughput when using the system compared to accessing Walrus without caches. In other words, measure the latency in test scenarios once the system is built, and the latency should be reduced by the system compared to the baseline.

Guarantee Decentralised Integrity: ensure the solution still preserves Walrus' decentralised guarantees while providing low-latency access. This means that cached content must be verifiable – end-users should be able to verify data against Walrus' proofs of availability (to ensure authenticity).

Evaluation and Documentation: documentation detailing the system architecture, design decisions, and integration points with Walrus should be extensive and comprehensive. This should include a performance analysis comparing the baseline vs the prototype, and should consider various situations and scenarios; this also means measuring performance across different geographical regions, analysing cache hit/miss rates, and evaluating the overhead of indexing and invalidation.

Timeline:

Dec 2025 – Research and Design: conduct a thorough literature review on decentralised storage, CDN architectures, and relevant indexing strategies. Have an initial system design and architecture plan for the Cache/CDN layer for Walrus (defining how aggregators, caches, and the index will interact). Additionally, become familiar with the Walrus HTTP API and begin prototyping parts of certain system components – in particular, the indexing system and the event listener (e.g., a daemon that listens to Walrus blockchain events or periodically queries Walrus metadata to discover new blobs).

Jan 2026 – Core Implementation: implement the full feature set – indexing service, caching (using RocksDB for fast data retrieval) with policies for eviction (e.g., time-to-live and event-triggered invalidation), and exposing a simple HTTP API for clients to request data, backed by the cache or fetched from Walrus as needed. Conduct initial performance tests on a very small scale to verify that the caching actually provides latency benefits.

Feb 2026 – Advanced Features and Extensive Testing: advanced features would be anything that goes above the extremely basic functionality of the cache – this could include things such as prefetching, request coalescing, and load-balancing. Testing includes measuring retrieval latency for various content sizes and distances, comparing Walrus direct access vs via the cache, and recording metrics like cache hit rates and network overhead – this should be extremely rigorous, considering as many situations as possible. In the background, work on the write-up should also begin.

Mar 2026 – Write-up and Optimisation: if testing reveals any redundancies or areas of improvement, this is when the prototype should be iterated on and revisited. In addition, the majority of the final dissertation should be complete by the end of this month.

Apr 2026 – Full Write-up and Submission: ideally, should submit the final dissertation to be reviewed by the supervisor during the first week of the month. Should also clean up the project code repository and ensure that it is extensively well-documented.

Note that throughout, notes should be taken to make the final dissertation much easier to write. Additionally, bi-weekly meetings will be set up as progress and sanity checks.

Resources:
Infrastructure: the project will need to utilise some sort of cloud-based virtual private servers to emulate geographically distributed cache nodes; servers from providers such as AWS or GCP could be used for this. This will allow realistic latency measurements over a wide-area network environment. The cache nodes will obviously require CPU, memory, and network bandwidth, but no significant GPU resources are anticipated at this time, as the workload involves primarily I/O and network operations.

Software and Tools: the prototype will be built using the Rust programming language and RocksDB for seamless integration with Walrus (as this is used by the latter). Docker will also be used for containerising the cache node application, simplifying the deployment during the cloud server phase. Other standard web/developer/devops tools will be used for monitoring and testing. The Walrus testnet will be used for development and experimentation in order to avoid costs on the mainnet. Ping, traceroute, httperf, JMeter, and custom scripts will be used as tools for the performance measurements.

Relevant Works:
- Walrus whitepaper (Mysten Labs, 2024) – official technical paper describing Walrus' design
- CacheCash (Almashaqbeh, G., 2019) – PhD dissertation proposing a blockchain-incentivised CDN
- Walrus & Sui vs IPFS & Filecoin (Zero Knowledge Collective, 2025) – blog post comparing Walrus with other decentralised storage systems; highlights the importance of caches/CDNs in Walrus
- The Graph Protocol (The Graph, 2020) – decentralised indexing protocol for blockchain data

C.2 Interim Report

Name (Candidate Number): VYSW4

Project title as given in October project plan / Current project title: Building a Cache / Content Delivery Service-like Layer on Top of the Walrus Decentralised Storage Network

Internal supervisor: Prof. Philipp Jovanovic

External supervisor: Alberto Sonnino

Progress made to date:
Since the start, I have worked on various aspects of this project. To begin with, I focused on understanding Walrus' architecture and the motivation for a CDN-like layer. Walrus stores blobs off-chain and relies on an aggregator to collect erasure-coded slivers from storage nodes and reconstruct content on demand; while this is robust and decentralised, it can introduce high read latency, which is not ideal for certain applications that rely on low-latency delivery (obviously). The project goal was therefore to augment Walrus with a cache/CDN-like layer that indexes blob metadata/events and caches popular content in order to provide fast and reliable access globally, all while preserving verifiability and integrity guarantees.

So far, I have completed majority of the main implementation required for this project – in fact, I have almost completed the implementation of the caching layer (it just needs to be tested together with the indexer and polling strategy to make sure they work seamlessly). In particular, I have finished building a Walrus HTTP client that can retrieve blobs via an aggregator and upload blobs via a publisher; an indexing subsystem with both a simulated event source (for deterministic local testing) and a Sui-backed event source (for testnet integration); and a polling-based tailing mechanism so the indexer can continuously query events rather than exiting after a single page. These things all establish the end-to-end integration path (Sui RPC to event stream to internal event model) that will drive cache population and invalidation.

The cache is designed as a RocksDB-backed storage layer that persists blob payloads and the metadata needed to enforce correctness and cache policy decisions. This includes tracking blob identifiers (blob_id/object_id), expiry information (e.g., end_epoch where available), insertion time, last-access time, access frequency counters, and object size. On top of this storage layer, I implemented and compared multiple eviction strategies suited to CDN-like workloads, including frequency-based (LFU) and recency-based policies with weighting (e.g., a weighted-LRU variant that biases toward popular items while still adapting to changing demand). Additionally, I also integrated eviction and invalidation mechanisms that reflect the Walrus lifecycle – for example, entries can be removed via TTL-based eviction, event-driven invalidation (e.g., with expiry signals), and I also implemented update handling where certification/extension events imply changes to the cached metadata and retention policy. At this stage, most of the remaining work (for the main part of the cache/CDN-like layer) is primarily end-to-end integration testing of the indexer-driven invalidation and the polling/tailing behaviour under realistic workloads, rather than adding missing core features.

Alongside implementation, I have written most of the literature review for the final dissertation. This covers Walrus' architecture and data retrieval flow, motivations for introducing caching/CDN-like layers in decentralised storage systems, as well as the relevant background on caching strategies, eviction theory, and consistency/invalidation mechanisms. The intent is to directly connect the related work and theory to the concrete design decisions made in this prototype (e.g., why certain policies are more appropriate under skewed access patterns, and why event-driven invalidation is important for correctness in an epoch-based retention model).

Finally, I have worked on the evaluation plan and even started implementing some of the experiments (and experiment frameworks) needed to demonstrate the benefit of the cache/CDN-like layer. To be more specific, I

have begun implementing measurement for cold versus warm retrieval latency (first request miss versus subsequent cache hits) and have identified a broader experimental matrix that will quantify cache hit rate, origin load reduction, and performance under different workloads (e.g., varying blob sizes, different request distributions, and concurrency). The next step is to finalise the integrated pipeline (indexer to cache policy to retrieval path) and then run the experiments in a controlled and repeatable way suitable to gain displayable results for the actual dissertation/write-up.

Remaining work to be done before the final report deadline:
The remaining work is now mostly integration, validation, and evaluation, followed by write-up completion. The core modules of the project (the Walrus client, indexer/tailer, and cache layer with eviction/invalidation policies) are largely implemented, so the key risk is ensuring they behave correctly together under continuous operation and producing rigorous experimental evidence. Additionally, I will also need to look into features that can extend this project more to satisfy my curiosity (given the time limit).

The end-to-end integration tests should hopefully only take another 1-2 weeks, and include:

- Integrating the indexer tailing loop with cache state updates so that event-driven invalidation/updates are exercised continuously.
- Adding targeted correctness tests (e.g., entries removed on expiry, extensions update retention metadata, and no stale reads after invalidation).
- Adding structured logs and basic metrics (hit/miss counts, evictions, bytes served, origin fetches, and per-request latency).

The experimental evaluation and benchmarking (including creating graphs, plots, etc.) is one of the main contributions of my project. Hence, I should take around 4-6 weeks working on this, and even longer if necessary. So far, my planned experiments include the following:

- Latency (cold vs warm): measure miss vs hit response times.
- Testing with workload variation – blob sizes and concurrency in particular.
- Effectiveness metrics: hit rate, byte hit ratio, eviction rate, origin load reduction (requests and bytes), and overhead (storage growth, write amplification if measured).
- Stability under load: stampede scenarios (many concurrent misses for the same blob) and (potentially) exploring behaviour with request coalescing.

In parallel, I will also continue to explore extensions beyond the basic cache, and this could take anywhere from 2 weeks up to 4, but these are mostly just extra, and I will only do these if I have time to spare. Luckily, I managed to make really good progress on the main caching aspect of the project, so there should be enough time to go as deep into this part as I want. Some of the features I am looking at implementing (but I am still undecided on) are request coalescing, trying different prefetching strategies, implementing admission control, and looking into multi-tier caching. Once again, it is important to note that the *extensions are not explicitly necessary as part of the main contributions for the project*, so I will not sacrifice time for the other sections in order to do this – these are just things that would be nice to have, but if, for example, the evaluation section takes longer, then I will focus more time on that and ensure it is completely comprehensive, as opposed to working on the extensions.

Finally, the final dissertation needs to be written up and reviewed with enough time to spare so that I can make any and all changes necessary. This will be continuously worked on in the background but will be the particular focus from late February / early March onwards.

To reiterate, I am hoping to finish the first draft of my dissertation by around the last week of March to give me ample time to refine the final version.

Appendix D

Selected Code Listing

D.1 Selection Rationale

The full Rust source tree is far too long to include in its entirety within this dissertation. Hence, to remain within a reasonable appendix length, this appendix includes the most architecturally significant listings rather than the entire codebase. In particular, the full evaluator implementation in `src/eval.rs` is very large, so only its verification section is reproduced here. The omitted material consists mainly of the bulk of `src/eval.rs`, the short `src/sharding.rs` helper, the configuration and metrics files (`src/config.rs` and `src/metrics.rs`), and the shell scripts under `walrus-cache-cdn/scripts/`. The omitted code is still present in the repository and is referenced throughout Chapter 5.

D.2 CLI Surface and Runtime Wiring

Listing D.1: CLI command surface in `walrus-cache-cdn/src/main.rs`.

```
#[derive(Parser)]
#[command(name = "walrus-cdn")]
#[command(about = "Walrus CDN prototype: indexer + HTTP retrieval")]
struct Cli {
    #[command(subcommand)]
    cmd: Commands,
}

#[derive(Subcommand)]
enum Commands {
    // print events from chosen source (simulated or sui)
    // by default this exits after `--limit` events
    // use `--follow` to keep tailing
    Index {
        // no. of events to pull before exiting (simulated ignores this)
        #[arg(long, default_value_t = 2)]
        limit: usize,
        // keep running and print an updated feed until Ctrl+C
        #[arg(long, default_value_t = false)]
        follow: bool,
        // demo-friendly compact view: print only key lifecycle events in one line each
        #[arg(long, default_value_t = false)]
        demo_view: bool,
    },
    // download a blob by its Walrus blob ID
    Get {
        #[arg(long)]
        id: String,
        #[arg(long)]
        out: Option<String>,
    },
    // download a blob by Sui object ID
    GetByObject {
        #[arg(long)]
        object_id: String,
        #[arg(long)]
        out: Option<String>,
    },
    // upload bytes via a publisher (local or public) - prints blobId
    Put {
        #[arg(long)]
        data: Option<String>,
        #[arg(long)]
        file: Option<String>,
        #[arg(long)]
        epochs: Option<u64>,
        #[arg(long, default_value_t = false)]
        permanent: bool,
    },
    // simple load generator: replay blob IDs from a file
    Bench {
        #[arg(long)]
        file: String,
        #[arg(long, default_value_t = 1000)]
        requests: usize,
        #[arg(long, default_value_t = 16)]
```

```
        conc: usize,
    },
    // force cache invalidation for demos (by blob id, tag, or epoch)
    #[command(group(
        ArgGroup::new("target")
            .required(true)
            .multiple(false)
            .args(["id", "tag", "epoch_leq"])
    ))]
    CachePurge {
        // purge one blob from cache
        #[arg(long)]
        id: Option<String>,
        // purge all blobs with this tag (e.g. module:walrus)
        #[arg(long)]
        tag: Option<String>,
        // purge all blobs whose end_epoch <= this value
        #[arg(long = "epoch-leq")]
        epoch_leq: Option<u64>,
    },
    // set/overwrite cached blob end-epoch metadata for deterministic epoch-purge demos
    CacheSetEpoch {
        #[arg(long)]
        id: String,
        #[arg(long = "end-epoch")]
        end_epoch: u64,
        #[arg(long, default_value_t = true, action = ArgAction::Set)]
        deletable: bool,
    },
    // run deterministic cache demo in one process so metrics remain consistent
    DemoFlow {
        #[arg(long, default_value = "prof-demo")]
        data: String,
        #[arg(long, default_value = "/tmp/prof-demo.bin")]
        out: String,
        // run without pressing Enter between steps
        #[arg(long, default_value_t = false)]
        auto: bool,
        // seconds to wait between steps in --auto mode
        #[arg(long, default_value_t = 6)]
        step_sleep_secs: u64,
    },
    // run dissertation-grade synthetic evaluation suite and export CSV/JSON artifacts
    Evaluate {
        // output directory for summary.csv, warmup_curve.csv, evaluation.json, README.md
        #[arg(long, default_value = "./evaluation_output")]
        out_dir: String,
        // reproducibility seed
        #[arg(long, default_value_t = 42)]
        seed: u64,
        // scale factor (1=default, 2=2x workload, ...)
        #[arg(long, default_value_t = 1)]
        scale: usize,
    },
```

Listing D.2: Application start-up, cache construction, and command dispatch in `walrus-cache-cdn/src/main.rs`.

```
#[tokio::main]
async fn main() -> Result<()> {
    dotenv().ok();
```

```
    tracing_subscriber::fmt()
        .with_env_filter(EnvFilter::try_from_default_env().unwrap_or_else(|_| "info".into()))
        .with_target(false)
        .compact()
        .init();

    let cfg = AppConfig::from_env();

    // start /metrics
    let maddr = cfg.metrics_addr.clone();
    tokio::spawn(async move {
        if let Err(e) = metrics::serve(&maddr).await {
            eprintln!("metrics server error: {e}");
        }
    });

    // build one shared cache (optional)
    let cache_opt: Option<Arc<Mutex<cache::RocksCache>>> = if cfg.cache_enabled {
        let admission = cfg.cache_admission;
        let adaptsize_k = if cfg.cache_adaptsize {
            Some(128_000.0)
        } else {
            None
        };
        Some(Arc::new(Mutex::new(cache::RocksCache::open(
            &cfg.cache_db_path,
            Some(cfg.cache_block_cache_mb * 1024 * 1024),
            admission,
            adaptsize_k,
        )?)))
    } else {
        None
    };

    // expose to indexer for purge/update side-effects
    if let Some(c) = &cache_opt {
        let _ = indexer::CACHE_HANDLE.set(c.clone());
    }

    let cli = Cli::parse();

    match cli.cmd {
        Commands::Index {
            limit,
            follow,
            demo_view,
        } => run_index(cfg, limit, follow, demo_view).await?,
        Commands::Get { id, out } => {
            let client = {
                let base = WalrusClient::new(&cfg.aggregator_base, cfg.publisher_base.as_deref())?;
                if let Some(c) = &cache_opt {
                    base.with_cache(c.clone())
                } else {
                    base
                }
            };
            if let Some(n) = sharding::pick_node(&id, &cfg.cache_nodes) {
                tracing::debug!("(shard) {} -> {}", id, n);
            }
            let bytes = client.get_blob_by_id(&id).await?;
            write_output(&bytes, out.as_deref())?;
        }
```

```
        Commands::GetByObject { object_id, out } => {
            let client = WalrusClient::new(&cfg.aggregator_base, cfg.publisher_base.as_deref())?;
            let bytes = client.get_blob_by_object_id(&object_id).await?;
            write_output(&bytes, out.as_deref())?;
        }
        Commands::Put {
            data,
            file,
            epochs,
            permanent,
        } => {
            let client = {
                let base = WalrusClient::new(&cfg.aggregator_base, cfg.publisher_base.as_deref())?;
                if let Some(c) = &cache_opt {
                    base.with_cache(c.clone())
                } else {
                    base
                }
            };
            let bytes = if let Some(path) = file {
                std::fs::read(path)?
            } else {
                data.unwrap_or_else(|| "hello walrus".to_string())
                    .into_bytes()
            };
            let blob_id = client.put_blob(bytes, epochs, Some(permanent)).await?;
            println!("{blob_id}");
        }
        Commands::Bench {
            file,
            requests,
            conc,
        } => {
            use rand::{SeedableRng, rngs::StdRng, seq::SliceRandom};
            let ids: Vec<String> = std::fs::read_to_string(&file)?
                .lines()
                .map(|s| s.trim().to_string())
                .filter(|s| !s.is_empty())
                .collect();
            anyhow::ensure!(!ids.is_empty(), "no blob IDs in {}", file);
            let client = {
                let base = WalrusClient::new(&cfg.aggregator_base, cfg.publisher_base.as_deref())?;
                if let Some(c) = &cache_opt {
                    base.with_cache(c.clone())
                } else {
                    base
                }
            };
            let client = Arc::new(client);
            let mut rng = StdRng::seed_from_u64(0xC0FFEE);
            let chosen: Vec<String> = (0..requests)
                .map(|_| ids.choose(&mut rng).unwrap().clone())
                .collect();
            let start = Instant::now();
            let mut tasks = Vec::new();
            for chunk in chosen.chunks((requests + conc - 1) / conc) {
                let client = client.clone();
                let chunk = chunk.to_owned();
                tasks.push(tokio::spawn(async move {
                    for id in chunk {
                        let _ = client.get_blob_by_id(&id).await;
                    }
```

```
            }));
        }
```

D.3 Walrus Retrieval and Upload Paths

Listing D.3: Walrus client structure and cached retrieval path in `walrus-cache-cdn/src/walrus.rs`.

```
pub struct PutBlobResult {
    pub blob_id: String,
    pub object_id: Option<String>,
    pub tx_digest: Option<String>,
}

#[derive(Clone)]
pub struct WalrusClient {
    http: Client,
    base: Url,
    publisher: Option<Url>,
    pub cache: Option<Arc<Mutex<RocksCache>>>,
}

impl WalrusClient {
    pub fn new(aggregator_base: &str, publisher_base: Option<&str>) -> Result<Self> {
        let base = Url::parse(aggregator_base)
            .with_context(|| format!("Invalid WALRUS_AGGREGATOR_URL: {aggregator_base}"))?;
        let publisher = match publisher_base {
            Some(p) => {
                Some(Url::parse(p).with_context(|| format!("Invalid WALRUS_PUBLISHER_URL: {p}"))?)
            }
            None => None,
        };
        Ok(Self {
            http: Client::builder().build()?,
            base,
            publisher,
            cache: None,
        })
    }

    pub fn with_cache(mut self, cache: Arc<Mutex<RocksCache>>) -> Self {
        self.cache = Some(cache);
        self
    }

    // GET /v1/blobs/{blobId} -> Vec<u8>
    pub async fn get_blob_by_id(&self, blob_id: &str) -> Result<Vec<u8>> {
        let t0 = Instant::now();

        if let Some(c) = &self.cache {
            match c.lock().unwrap().get(blob_id)? {
                CacheGet::RamHit(b) => {
                    info!("cache_hit source=ram blob_id={} bytes={}", blob_id, b.len());
                    return Ok(b.to_vec());
                }
                CacheGet::DiskHit(b) => {
                    info!(
                        "cache_hit source=disk blob_id={} bytes={}",
```

```
                    blob_id,
                    b.len()
                );
                return Ok(b.to_vec());
            }
            CacheGet::Miss => {
                info!("cache_miss blob_id={}", blob_id);
            }
        }
    }

    let url = self.base.join(&format!("v1/blobs/{blob_id}"))?;
    let res = self
        .http
        .get(url)
        .send()
        .await?
        .error_for_status()
        .context("Aggregator returned an error")?;
    let body = res.bytes().await?;
    CACHE_ORIGIN_MISSES.inc();
    CACHE_BYTES_SERVED.inc_by(body.len() as u64);
    REQUEST_LATENCY_MS
        .with_label_values(&["origin"])
        .observe(t0.elapsed().as_millis() as f64);

    if let Some(c) = &self.cache {
        let meta = BlobMeta {
            size: body.len() as u64,
            end_epoch: None,
            ts_ms: None,
            deletable: None,
            tags: vec!["module:walrus".into()],
        };
        let admitted = c
            .lock()
            .unwrap()
            .put_if_admit(blob_id, body.clone(), &meta)?;
        info!(
            "cache_store blob_id={} bytes={} admitted={}",
            blob_id,
            body.len(),
            admitted
        );
        CACHE_BYTES_WRITTEN.inc_by(body.len() as u64);
    }
    Ok(body.to_vec())
}

// GET /v1/blobs/by-object-id/{object-id} -> Vec<u8>
// left uncached by default - can add a separate object-id cache if needed
pub async fn get_blob_by_object_id(&self, object_id: &str) -> Result<Vec<u8>> {
    let url = self
        .base
        .join(&format!("v1/blobs/by-object-id/{object_id}"))?;
    let res = self
        .http
        .get(url)
        .send()
        .await?
        .error_for_status()
        .context("Aggregator returned an error")?;
```

```
        Ok(res.bytes().await?.to_vec())
    }
```

Listing D.4: Upload path and publisher-response decoding in `walrus-cache-cdn/src/walrus.rs`.

```
    // PUT /v1/blobs[...] -> returns blobId
    pub async fn put_blob(
        &self,
        data: impl Into<Vec<u8>>,
        epochs: Option<u64>,
        permanent: Option<bool>,
    ) -> Result<String> {
        Ok(self
            .put_blob_with_details(data, epochs, permanent)
            .await?
            .blob_id)
    }

    // PUT /v1/blobs[...] -> returns blobId + optional object/tx identifiers when present
    pub async fn put_blob_with_details(
        &self,
        data: impl Into<Vec<u8>>,
        epochs: Option<u64>,
        permanent: Option<bool>,
    ) -> Result<PutBlobResult> {
        let Some(pub_base) = &self.publisher else {
            bail!("No publisher configured. Set WALRUS_PUBLISHER_URL to use uploads.");
        };
        let mut url = pub_base.join("v1/blobs")?;
        {
            let mut qp = url.query_pairs_mut();
            if let Some(e) = epochs {
                qp.append_pair("epochs", &e.to_string());
            }
            if let Some(perma) = permanent {
                qp.append_pair("permanent", if perma { "true" } else { "false" });
            }
        }

        let body_vec: Vec<u8> = data.into();
        let res = self
            .http
            .put(url)
            .body(body_vec)
            .send()
            .await?
            .error_for_status()
            .context("Publisher returned an error")?;
        let v: serde_json::Value = res.json().await?;
        let blob_id = v
            .pointer("/newlyCreated/blobObject/blobId")
            .or_else(|| v.pointer("/alreadyCertified/blobObject/blobId"))
            .and_then(|x| x.as_str())
            .ok_or_else(|| anyhow::anyhow!("blobId not found in publisher response: {v}"))?
            .to_string();

        let object_id = v
            .pointer("/newlyCreated/blobObject/objectId")
            .or_else(|| v.pointer("/newlyCreated/blobObject/id"))
            .or_else(|| v.pointer("/alreadyCertified/blobObject/objectId"))
```

```
            .or_else(|| v.pointer("/alreadyCertified/blobObject/id"))
            .or_else(|| v.pointer("/blobObject/objectId"))
            .or_else(|| v.pointer("/blobObject/id"))
            .and_then(|x| x.as_str())
            .map(|s| s.to_string());

        let tx_digest = v
            .pointer("/newlyCreated/txDigest")
            .or_else(|| v.pointer("/alreadyCertified/txDigest"))
            .or_else(|| v.pointer("/txDigest"))
            .and_then(|x| x.as_str())
            .map(|s| s.to_string());

        Ok(PutBlobResult {
            blob_id,
            object_id,
            tx_digest,
        })
    }
}
```

D.4 Cache Core, Metadata, and Invalidation

Listing D.5: Admission modes, metadata, RocksDB opening logic, and core get/put behaviour in `walrus-cache-cdn/src/cache.rs`.

```
#[derive(Clone, Copy, Debug)]
pub enum AdmissionMode {
    WTinyLFU,
    Lru,
    None,
}

#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct BlobMeta {
    pub size: u64,
    pub end_epoch: Option<u64>,
    pub ts_ms: Option<u64>,
    pub deletable: Option<bool>,
    pub tags: Vec<String>,
}

pub enum CacheGet {
    RamHit(Bytes),
    DiskHit(Bytes),
    Miss,
}

pub struct RocksCache {
    db: DBWithThreadMode<SingleThreaded>,
    window_lru: VecDeque<(String, Bytes)>,
    disk_hits: HashMap<String, u8>,

    sketch: CountMin8,
    admission: AdmissionMode,
    adaptsize_k: Option<f64>,
}
```

```
impl RocksCache {
    pub fn open(
        path: impl AsRef<Path>,
        block_cache_bytes: Option<usize>,
        admission: AdmissionMode,
        adaptsize_k: Option<f64>,
    ) -> Result<Self> {
        let mut opts = Options::default();
        opts.create_if_missing(true);
        opts.create_missing_column_families(true);
        opts.set_max_open_files(1024);
        opts.optimize_for_point_lookup(1024);

        let mut table = BlockBasedOptions::default();
        let cap = block_cache_bytes.unwrap_or(DEFAULT_BLOCK_CACHE_BYTES);
        let lru: RdbCache = RdbCache::new_lru_cache(cap);
        table.set_block_cache(&lru);
        table.set_bloom_filter(10.0, false);
        table.set_cache_index_and_filter_blocks(true);
        opts.set_block_based_table_factory(&table);

        let cf_blobs = ColumnFamilyDescriptor::new("blobs", Options::default());
        let cf_meta = ColumnFamilyDescriptor::new("meta", Options::default());
        let cf_tags = ColumnFamilyDescriptor::new("tags", Options::default());

        let db = DB::open_cf_descriptors(&opts, path, vec![cf_blobs, cf_meta, cf_tags])?;

        Ok(Self {
            db,
            window_lru: VecDeque::with_capacity(WINDOW_LRU_CAP),
            disk_hits: HashMap::new(),
            sketch: CountMin8::new(1 << 14, 4),
            admission,
            adaptsize_k,
        })
    }

    #[inline]
    fn cf_blobs(&self) -> &rocksdb::ColumnFamily {
        self.db.cf_handle("blobs").expect("missing cf: blobs")
    }

    #[inline]
    fn cf_meta(&self) -> &rocksdb::ColumnFamily {
        self.db.cf_handle("meta").expect("missing cf: meta")
    }

    #[inline]
    fn cf_tags(&self) -> &rocksdb::ColumnFamily {
        self.db.cf_handle("tags").expect("missing cf: tags")
    }

    #[inline]
    fn sketch_add(&mut self, k: &str) {
        self.sketch.add(k);
    }
    #[inline]
    fn sketch_est(&self, k: &str) -> u32 {
        self.sketch.estimate(k)
    }

    fn admit(&mut self, key: &str, size: usize, victim_freq: u32) -> bool {
```

```
        match self.admission {
            AdmissionMode::None => size <= MAX_ADMIT_BYTES,
            AdmissionMode::Lru => size <= MAX_ADMIT_BYTES && self.adaptsize_pass(size),
            AdmissionMode::WTinyLFU => {
                let cand = self.sketch_est(key);
                size <= MAX_ADMIT_BYTES && cand >= victim_freq && self.adaptsize_pass(size)
            }
        }
    }

    fn adaptsize_pass(&self, size: usize) -> bool {
        if let Some(k) = self.adaptsize_k {
            let p = (k / (size as f64)).clamp(0.0, 1.0);
            return thread_rng().gen_bool(p);
        }
        true
    }

    pub fn get(&mut self, blob_id: &str) -> Result<CacheGet> {
        let t0 = Instant::now();
        self.sketch_add(blob_id);

        if let Some((_, v)) = self.window_lru.iter().find(|(k, _)| k == blob_id) {
            CACHE_RAM_HITS.inc();
            CACHE_BYTES_SERVED.inc_by(v.len() as u64);
            REQUEST_LATENCY_MS
                .with_label_values(&["ram"])
                .observe(t0.elapsed().as_millis() as f64);
            return Ok(CacheGet::RamHit(v.clone()));
        }

        if let Some(v) = self.db.get_cf(self.cf_blobs(), blob_id.as_bytes())? {
            let b = Bytes::from(v);
            CACHE_DISK_HITS.inc();
            CACHE_BYTES_SERVED.inc_by(b.len() as u64);
            REQUEST_LATENCY_MS
                .with_label_values(&["disk"])
                .observe(t0.elapsed().as_millis() as f64);
            let e = self.disk_hits.entry(blob_id.to_string()).or_default();
            *e = e.saturating_add(1);
            if *e >= 2 || self.sketch_est(blob_id) >= 3 {
                self.promote(blob_id, &b);
                self.disk_hits.remove(blob_id);
            }
            return Ok(CacheGet::DiskHit(b));
        }

        Ok(CacheGet::Miss)
    }

    fn promote(&mut self, blob_id: &str, bytes: &Bytes) {
        if self.window_lru.len() >= WINDOW_LRU_CAP {
            self.window_lru.pop_back();
        }
        self.window_lru
            .push_front((blob_id.to_string(), bytes.clone()));
    }

    pub fn put_if_admit(&mut self, blob_id: &str, bytes: Bytes, meta: &BlobMeta) -> Result<bool> {
        let size = bytes.len();
        let victim_freq = self
            .window_lru
```

```
            .back()
            .map(|(k, _)| self.sketch_est(k))
            .unwrap_or(0);
        let admit = self.admit(blob_id, size, victim_freq);

        self.write_kv(blob_id, &bytes, meta)?;
        CACHE_PUTS.inc();
        CACHE_BYTES_WRITTEN.inc_by(bytes.len() as u64);

        if admit {
            self.promote(blob_id, &bytes);
        }

        if !meta.tags.is_empty() {
            self.add_tags(blob_id, &meta.tags)?;
        }
        Ok(admit)
    }
```

Listing D.6: Invalidation methods and epoch-metadata maintenance in `walrus-cache-cdn/src/cache.rs`.

```
    pub fn purge_by_blob_id(&mut self, blob_id: &str) -> Result<()> {
        self.db.delete_cf(self.cf_blobs(), blob_id.as_bytes())?;
        self.db.delete_cf(self.cf_meta(), blob_id.as_bytes())?;
        // delete tag entries with suffix |blob_id
        let entries = self.scan_prefix(self.cf_tags(), b"tag|");
        for (k, _) in entries {
            if k.ends_with(format!("|{blob_id}").as_bytes()) {
                self.db.delete_cf(self.cf_tags(), &k)?;
            }
        }
        if let Some(pos) = self.window_lru.iter().position(|(k, _)| k == blob_id) {
            self.window_lru.remove(pos);
        }
        CACHE_PURGES.inc();
        Ok(())
    }

    pub fn add_tags(&self, blob_id: &str, tags: &[String]) -> Result<()> {
        let mut wo = WriteOptions::default();
        wo.set_sync(false);
        for t in tags {
            let key = format!("tag|{t}|{blob_id}");
            self.db
                .put_cf_opt(self.cf_tags(), key.as_bytes(), &[], &wo)?;
        }
        Ok(())
    }

    pub fn purge_by_tag(&mut self, tag: &str) -> Result<()> {
        let prefix = format!("tag|{tag}|");
        let pfx = prefix.as_bytes().to_vec();
        let entries = self.scan_prefix(self.cf_tags(), &pfx);
        for (k, _) in entries {
            if let Some(bid) = k
                .split(|&b| b == b'|')
                .last()
                .and_then(|s| std::str::from_utf8(s).ok())
            {
                let _ = self.purge_by_blob_id(bid);
```

```
        }
        self.db.delete_cf(self.cf_tags(), &k)?;
    }
    CACHE_PURGES.inc();
    Ok(())
}

pub fn purge_epoch_leq(&mut self, epoch: u64) -> Result<()> {
    let prefix = b"tag|epoch:";
    let entries = self.scan_prefix(self.cf_tags(), prefix);
    for (k, _) in entries {
        let s = std::str::from_utf8(&k).unwrap_or("");
        // tag|epoch:{n}|{blob}
        if let Some(rest) = s.strip_prefix("tag|epoch:") {
            if let Some((n, bid)) = rest.split_once('|') {
                if let Ok(v) = n.parse::<u64>() {
                    if v <= epoch {
                        let _ = self.purge_by_blob_id(bid);
                        self.db.delete_cf(self.cf_tags(), &k)?;
                    }
                }
            }
        }
    }
    Ok(())
}

pub fn update_meta_epoch(
    &self,
    blob_id: &str,
    end_epoch: u64,
    deletable: Option<bool>,
    ts_ms: Option<u64>,
) -> Result<()> {
    let existing = self.db.get_cf(self.cf_meta(), blob_id.as_bytes())?;
    let mut meta = if let Some(v) = existing {
        serde_json::from_slice::<BlobMeta>(&v).unwrap_or(BlobMeta {
            size: 0,
            end_epoch: None,
            ts_ms: None,
            deletable: None,
            tags: vec![],
        })
    } else {
        BlobMeta {
            size: 0,
            end_epoch: None,
            ts_ms: None,
            deletable: None,
            tags: vec![],
        }
    };
    meta.end_epoch = Some(end_epoch);
    meta.deletable = deletable.or(meta.deletable);
    meta.ts_ms = ts_ms.or(meta.ts_ms);
    if !meta.tags.iter().any(|t| t.starts_with("epoch:")) {
        meta.tags.push(format!("epoch:{end_epoch}"));
    }
    let v = serde_json::to_vec(&meta)?;
    self.db.put_cf(self.cf_meta(), blob_id.as_bytes(), v)?;
    self.add_tags(blob_id, &meta.tags)?;
    Ok(())
```

```
    }

    fn write_kv(&self, blob_id: &str, bytes: &Bytes, meta: &BlobMeta) -> Result<()> {
        let mut wo = WriteOptions::default();
        wo.set_sync(false);
        self.db
            .put_cf_opt(self.cf_blobs(), blob_id.as_bytes(), bytes, &wo)?;
        let v = serde_json::to_vec(meta)?;
        self.db
            .put_cf_opt(self.cf_meta(), blob_id.as_bytes(), &v, &wo)?;
        Ok(())
    }

    fn scan_prefix(&self, cf: &rocksdb::ColumnFamily, prefix: &[u8]) -> Vec<(Vec<u8>, Vec<u8>)> {
        let mut out = Vec::new();
        let mut it = self
            .db
            .iterator_cf(cf, IteratorMode::From(prefix, Direction::Forward));
        while let Some(Ok((k, v))) = it.next() {
            if !k.starts_with(prefix) {
                break;
            }
            out.push((k.to_vec(), v.to_vec()));
        }
        out
    }
}
```

Listing D.7: Count-Min sketch used by the WTinyLFU-style path in `walrus-cache-cdn/src/cache.rs`.

```
/* --------- Count-Min Sketch (8-bit counters) --------- */
struct CountMin8 {
    width: usize,
    depth: usize,
    table: Vec<u8>,
    salts: Vec<u64>,
}
impl CountMin8 {
    fn new(width: usize, depth: usize) -> Self {
        assert!(width.is_power_of_two());
        let table = vec![0u8; width * depth];
        let salts = (0..depth)
            .map(|i| 0x9e3779b97f4a7c15u64 ^ (i as u64).wrapping_mul(0xBF58476D1CE4E5B9))
            .collect();
        Self {
            width,
            depth,
            table,
            salts,
        }
    }
    #[inline]
    fn idx(&self, h: u64, row: usize) -> usize {
        ((h ^ self.salts[row]) as usize) & (self.width - 1)
    }
    #[inline]
    fn hash(s: &str) -> u64 {
        use ahash::AHasher;
        use std::hash::{Hash, Hasher};
        let mut hasher = AHasher::default();
```

```
        s.hash(&mut hasher);
        hasher.finish()
    }
    fn add(&mut self, key: &str) {
        let h = Self::hash(key);
        for r in 0..self.depth {
            let i = self.idx(h, r);
            let cell = &mut self.table[r * self.width + i];
            if *cell < u8::MAX {
                *cell += 1;
            }
        }
    }
    fn estimate(&self, key: &str) -> u32 {
        let h = Self::hash(key);
        let mut m = u8::MAX;
        for r in 0..self.depth {
            let i = self.idx(h, r);
            m = m.min(self.table[r * self.width + i]);
        }
        m as u32
    }
}
```

D.5 Lifecycle Event Handling

Listing D.8: Event kinds, event representation, and the simulated event source in `walrus-cache-cdn/src/indexer.rs`.

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum EventKind {
    // walrus blob lifecycle (on-chain) events
    BlobRegistered,
    BlobCertified {
        end_epoch: u64,
        deletable: bool,
    },
    BlobDeleted,
    InvalidBlobId,

    // sui / system epoch signals
    EpochChangeStart {
        epoch: u64,
    },
    EpochChangeDone {
        epoch: u64,
    },
    ShardsReceived,
    EpochParametersSelected,
    ShardRecoveryStart,

    // synthetic (derived) signals
    Extended {
        old_end_epoch: u64,
        new_end_epoch: u64,
    },
    Expired,
}
```

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct WalrusEvent {
    pub kind: EventKind,
    pub blob_id: Option<String>,
    pub object_id: Option<String>,
    pub end_epoch: Option<u64>,
    pub tx_digest: Option<String>,
    pub event_seq: Option<String>,
    pub timestamp_ms: Option<u64>,
}

pub trait EventSource: Send {
    fn stream(&mut self) -> Pin<Box<dyn Stream<Item = Result<WalrusEvent>> + Send + '_>>;
}

/* ---------------------- simulated source ---------------------- */
pub struct SimulatedEventSource;

impl SimulatedEventSource {
    pub fn new() -> Self {
        Self
    }
}

impl EventSource for SimulatedEventSource {
    fn stream(&mut self) -> Pin<Box<dyn Stream<Item = Result<WalrusEvent>> + Send + '_>> {
        let s = async_stream::try_stream! {
            let samples = vec![
                WalrusEvent {
                    kind: EventKind::BlobRegistered,
                    blob_id: Some("M4hsZGQ1oCktdzeg...".into()),
                    object_id: Some("0xabc...".into()),
                    end_epoch: None,
                    tx_digest: Some("A1...".into()),
                    event_seq: Some("0".into()),
                    timestamp_ms: Some(1_700_000_000_000),
                },
                WalrusEvent {
                    kind: EventKind::BlobCertified { end_epoch: 50, deletable: true },
                    blob_id: Some("M4hsZGQ1oCktdzeg...".into()),
                    object_id: Some("0xabc...".into()),
                    end_epoch: Some(50),
                    tx_digest: Some("A1...".into()),
                    event_seq: Some("1".into()),
                    timestamp_ms: Some(1_700_000_100_000),
                },
                WalrusEvent {
                    kind: EventKind::BlobCertified { end_epoch: 80, deletable: true },
                    blob_id: Some("M4hsZGQ1oCktdzeg...".into()),
                    object_id: Some("0xabc...".into()),
                    end_epoch: Some(80),
                    tx_digest: Some("A1...".into()),
                    event_seq: Some("2".into()),
                    timestamp_ms: Some(1_700_000_200_000),
                },
                WalrusEvent {
                    kind: EventKind::EpochChangeDone { epoch: 81 },
                    blob_id: None, object_id: None, end_epoch: None,
                    tx_digest: None, event_seq: None, timestamp_ms: Some(1_700_000_300_000),
                },
            ];
```

```
        let mut last_end_epoch: HashMap<String, u64> = HashMap::new();

        for ev in samples {
            if let (EventKind::BlobCertified { end_epoch: new, .. }, Some(bid)) =
                (&ev.kind, ev.blob_id.clone())
            {
                if let Some(old) = last_end_epoch.insert(bid.clone(), *new) {
                    if *new > old {
                        yield WalrusEvent {
                            kind: EventKind::Extended { old_end_epoch: old, new_end_epoch: *new
},
                            blob_id: Some(bid), object_id: None, end_epoch: Some(*new),
                            tx_digest: ev.tx_digest.clone(), event_seq: ev.event_seq.clone(),
                            timestamp_ms: ev.timestamp_ms,
                        };
                    }
                }
            }

            if let EventKind::EpochChangeDone { epoch } = ev.kind {
                let current_epoch = epoch;
                for (bid, ee) in last_end_epoch.clone() {
                    if ee <= current_epoch {
                        yield WalrusEvent {
                            kind: EventKind::Expired,
                            blob_id: Some(bid), object_id: None, end_epoch: Some(ee),
                            tx_digest: None, event_seq: None, timestamp_ms: ev.timestamp_ms,
                        };
                    }
                }
            }

            yield ev;
            sleep(Duration::from_millis(150)).await;
        }
    };
    Box::pin(s)
}
```

Listing D.9: Representative live-event decoding logic in `walrus-cache-cdn/src/indexer.rs`.

```
    fn decode(type_str: &str, parsed: &Value) -> Result<WalrusEvent> {
        if type_str.ends_with("::BlobRegistered") || type_str.contains("BlobRegistered") {
            return Ok(WalrusEvent {
                kind: EventKind::BlobRegistered,
                blob_id: Self::get_str(parsed, &["blob_id", "blobId"]),
                object_id: Self::get_str(parsed, &["object_id", "objectId"]),
                end_epoch: None,
                tx_digest: None,
                event_seq: None,
                timestamp_ms: None,
            });
        }
        if type_str.ends_with("::BlobCertified") || type_str.contains("BlobCertified") {
            let end_epoch = Self::get_u64(parsed, &["end_epoch", "endEpoch"])
                .ok_or_else(|| anyhow!("BlobCertified missing end_epoch"))?;
            let deletable = Self::get_bool(parsed, &["deletable"]).unwrap_or(true);
            return Ok(WalrusEvent {
                kind: EventKind::BlobCertified {
```

```
                    end_epoch,
                    deletable,
                },
                blob_id: Self::get_str(parsed, &["blob_id", "blobId"]),
                object_id: Self::get_str(parsed, &["object_id", "objectId"]),
                end_epoch: Some(end_epoch),
                tx_digest: None,
                event_seq: None,
                timestamp_ms: None,
            });
        }
        if type_str.ends_with("::BlobDeleted") || type_str.contains("BlobDeleted") {
            return Ok(WalrusEvent {
                kind: EventKind::BlobDeleted,
                blob_id: Self::get_str(parsed, &["blob_id", "blobId"]),
                object_id: Self::get_str(parsed, &["object_id", "objectId"]),
                end_epoch: None,
                tx_digest: None,
                event_seq: None,
                timestamp_ms: None,
            });
        }
        if type_str.contains("InvalidBlobID") || type_str.contains("InvalidBlobId") {
            return Ok(WalrusEvent {
                kind: EventKind::InvalidBlobId,
                blob_id: Self::get_str(parsed, &["blob_id", "blobId"]),
                object_id: None,
                end_epoch: None,
                tx_digest: None,
                event_seq: None,
                timestamp_ms: None,
            });
        }
        if type_str.contains("EpochChangeStart") {
            let epoch = Self::get_u64(parsed, &["epoch"]).unwrap_or_default();
```

D.6 Selected Verification Code

Listing D.10: Unit tests from `walrus-cache-cdn/src/eval.rs`. These listings show how the synthetic evaluator is checked for warm-up improvement, policy behaviour, and persistence correctness.

```
    #[test]
    fn cold_warm_curve_improves() {
        let out = run_evaluation_suite(1, 42);
        let mut curve: Vec<&WarmupRow> = out
            .warmup
            .iter()
            .filter(|r| r.scenario == "cold_start_warmup_curve")
            .collect();
        curve.sort_by_key(|r| r.window_index);
        assert!(
            curve.len() >= 6,
            "expected enough windows for a warm-up curve"
        );
        let early: f64 = curve.iter().take(3).map(|r| r.hit_rate).sum::<f64>() / 3.0;
        let late: f64 = curve.iter().rev().take(3).map(|r| r.hit_rate).sum::<f64>() / 3.0;
```

```
    assert!(
        late > early,
        "late hit-rate should exceed early hit-rate: early={early}, late={late}"
    );
}

#[test]
fn wtinylfu_beats_lru_on_scan_resistance() {
    let out = run_evaluation_suite(1, 7);
    let w = out
        .rows
        .iter()
        .find(|r| r.scenario == "scan_resistance" && r.variant == "wtinylfu")
        .expect("missing wtinylfu scan result");
    let l = out
        .rows
        .iter()
        .find(|r| r.scenario == "scan_resistance" && r.variant == "lru")
        .expect("missing lru scan result");
    assert!(
        w.hit_rate >= l.hit_rate,
        "WTinyLFU should be at least as scan-resistant as LRU"
    );
}

#[test]
fn adaptsize_improves_byte_efficiency() {
    let out = run_evaluation_suite(1, 9);
    let off = out
        .rows
        .iter()
        .find(|r| {
            r.scenario == "size_aware_admission_adaptsize_lite" && r.variant == "adaptsize_off"
        })
        .expect("missing adaptsize_off row");
    let on = out
        .rows
        .iter()
        .find(|r| {
            r.scenario == "size_aware_admission_adaptsize_lite" && r.variant == "adaptsize_on"
        })
        .expect("missing adaptsize_on row");
    assert!(
        on.byte_hit_rate >= off.byte_hit_rate * 0.95,
        "adapt-size should be competitive on byte hit rate (on={}, off={})",
        on.byte_hit_rate,
        off.byte_hit_rate
    );
}

#[test]
fn baseline_vs_cdn_has_latency_and_hit_gains() {
    let out = run_evaluation_suite(1, 123);
    let base = out
        .rows
        .iter()
        .find(|r| r.scenario == "baseline_vs_cdn_ab_toggle" && r.variant == "baseline_no_cache")
        .expect("missing baseline row");
    let cdn = out
        .rows
        .iter()
        .find(|r| r.scenario == "baseline_vs_cdn_ab_toggle" && r.variant == "cdn_wtinylfu")
```

```
            .expect("missing cdn row");
        assert!(
            cdn.hit_rate > base.hit_rate,
            "cdn should improve hit rate over baseline"
        );
        assert!(
            cdn.avg_ms < base.avg_ms,
            "cdn should reduce average latency"
        );
    }

    #[test]
    fn disk_survival_recovery_roundtrip() {
        let db_path = test_temp_dir("disk_recovery");
        fs::create_dir_all(&db_path).expect("create temp db dir");

        let blob_id = "disk-survival-blob";
        let payload = Bytes::from(vec![7u8; 128 * 1024]);
        let meta = BlobMeta {
            size: payload.len() as u64,
            end_epoch: Some(55),
            ts_ms: Some(1_700_000_000_000),
            deletable: Some(true),
            tags: vec!["module:walrus".to_string(), "dataset:alpha".to_string()],
        };

        {
            let mut cache = RocksCache::open(
                &db_path,
                Some(32 * 1024 * 1024),
                AdmissionMode::WTinyLFU,
                None,
            )
            .expect("open initial cache");
            cache
                .put_if_admit(blob_id, payload.clone(), &meta)
                .expect("put to cache");
            let first = cache.get(blob_id).expect("first get");
            assert!(matches!(first, CacheGet::RamHit(_) | CacheGet::DiskHit(_)));
```